

Università degli Studi di Udine

Corso di Sistemi Operativi

Appunti di Lezione

Sbobinati dalle registrazioni audio e appunti

A.A. 2025 - 2026

Docente: Mirko Loghi

1 - Fondamenti dei Sistemi Operativi e Architettura delle System Call	8
1.1 Architettura Generale del Sistema Operativo	8
1.1.1 Accesso al Sistema: Autenticazione, Identificatori e Shell	8
1.2 Funzioni e Componenti Interne del Kernel	9
1.2.1 Programma e Processo: una Distinzione Fondamentale	10
1.3 Livelli di Privilegio e Meccanismi di Protezione	10
1.4 Le System Call: L'Interfaccia Hardware/Software	11
1.4.1 Le API (Application Programming Interface)	11
1.5 Implementazione Architetturale delle Syscall (Casi Studio)	12
1.5.1 Architettura ARM a 32 bit	12
1.5.2 Architetture Moderne	12
2 – Evoluzione delle Architetture Hardware e dei Sistemi Operativi	13
2.1 La Prima Generazione (1945–1955): Tubi a Vuoto e Assenza di Software	13
2.2 La Seconda Generazione (1955–1965): Transistor, Mainframe e Batch Processing	13
2.3 La Terza Generazione (1965–1980): Circuiti Integrati e Famiglie di Macchine	14
2.4 Il Progetto Multics e la Nascita di UNIX	14
2.4.1 La Filosofia di UNIX	15
2.5 Il Linguaggio C e la Distribuzione di UNIX	15
2.6 La Quarta Generazione (1980–Presente): L'Era dei Personal Computer	16
3 – Architettura di Sistema, I/O e Standard POSIX	17
3.1 Livelli Architettureali e Utilità di Sistema	17
3.2 Input/Output e File Descriptor	17
3.3 Il File System: Percorsi, Tipi e Lock	18
3.4 Gestione degli Errori e Segnali	19
3.5 Tempo e Comunicazione Inter-Processo (IPC)	19
3.6 Standardizzazione: Libreria C e POSIX	20
3.7 Limiti di Sistema: Caratteristiche Generali	20
3.8 Limiti Numerici Principali (POSIX)	20
3.9 Limiti Operativi e File System	21
3.10 Garanzie POSIX e Consultazione dei Manuali	21
4 – Unix: Panoramica per l'Utente	23
4.1 Identità, Accesso e Gestione Utenti/Gruppi	23
4.2 Risoluzione dei Nomi e Servizi di Rete	23
4.3 Informazioni sui File, Proprietà e Permessi Standard	23
4.4 Permessi Specifici: Access Control List (ACL)	24
4.5 Gestione dello Spazio: Sparse File	25
4.6 Redirezione dell'I/O	25
4.7 Systemd: L'Inizializzazione Moderna	27
4.8 La Gerarchia del File System Unix	27
4.9 I Dispositivi (Device Files in /dev)	28
4.10 Panoramica dei Comandi Principali	29
4.11 Gestione dello Storage e dei File System	29
4.12 Controllo dei Processi e Informazioni di Sistema	30

5 – Processi e Thread	31
5.1 Definizione e Natura del Processo	31
5.2 Modelli di Concorrenza e Multiprogrammazione	31
5.3 Supporto Hardware, Protezione e Memoria Virtuale	32
5.4 Il Modello degli Stati del Processo	32
5.5 Ciclo di Vita: Creazione e Gerarchia	33
5.6 Ciclo di Vita: Terminazione, Orfani e Zombie	33
5.7 Il Process Control Block (PCB) e il Context Switch	34
5.8 Scheduling e Overhead del Context Switch	34
5.9 Architetture del Kernel	35
5.9.1 Kernel Monolitico	35
5.9.2 Microkernel	35
5.9.3 Kernel Ibrido	36
5.10 Gestione dei Thread e Concorrenza	36
5.10.1 Modelli di Mappatura dei Thread	36
5.10.2 Framework e Modelli Concorrenti	37
5.10.3 Cancellazione e Pulizia	37
5.11 Segnali	37
5.12 Comunicazione Inter-Processo (IPC)	38
5.12.1 Scambio di Messaggi	38
5.12.2 Memoria Condivisa	38
5.13 Comunicazione Inter-Processo (IPC) tramite Socket	39
6 – Virtualizzazione	40
6.1 Architetture di Virtualizzazione	40
6.1.1 Hypervisor di Tipo 1 (Bare-Metal)	40
6.1.2 Hypervisor di Tipo 2 (Hosted)	40
6.1.3 Supporto Hardware alla Virtualizzazione	41
6.2 Ciclo di Vita del Processo: Esecuzione e Terminazione	41
6.3 Ambiente di Esecuzione e Parametrizzazione	42
6.4 Layout della Memoria del Processo	42
6.5 Librerie Dinamiche	43
6.6 Limiti e Restrizioni dei Processi	44
6.7 Identificatori di Processo e Identità Utente	44
6.8 Creazione dei Processi: fork() e Copy-On-Write	45
6.9 Il Flag close-on-exec	45
6.10 Esecuzione di Programmi: La Famiglia exec	45
6.11 Alterazione dell'Identità e Sicurezza	46
6.12 Attesa e Terminazione	46
6.13 La Funzione di Libreria system()	47
6.14 Terminali, Console e Procedure di Login	47
6.15 Gruppi di Processi, Sessioni e Terminale di Controllo	47
6.15.1 La Gerarchia a Due Livelli	47
6.15.2 Il Terminale di Controllo	48
6.15.3 Foreground, Background e Gestione I/O	48

6.15.4 Generazione e Inoltro dei Segnali	48
6.15.5 API per Gruppi e Terminali	49
6.15.6 Gruppi di Processi Orfani	49
6.16 Processi Demone (Daemon)	49
6.17 Registrazione degli Errori e Sottosistema Syslog	50
7 – Input/Output di Base nei Sistemi Unix (I/O Non Bufferizzato)	51
7.1 File Descriptor e I/O di Basso Livello	51
7.2 Apertura e Creazione di File (open e creat)	51
7.3 Relatività, Sicurezza e Protezione Concorrenziale (openat)	52
7.4 Lettura e Scrittura (read, write, close)	52
7.5 Modifica del Punto d'Accesso e Sparse File (lseek)	53
8 – Architettura Interna dell'I/O, Concorrenza e Libreria Standard	54
8.1 Strutture Dati del Kernel per la Gestione dei File	54
8.2 Condivisione dei File e Viste Indipendenti	55
8.3 Operazioni Atomiche e Prevenzione delle Race Condition	55
8.4 Duplicazione dei Descrittori e Redirezione dell'I/O	56
8.5 Sincronizzazione dei Buffer del Kernel (Flushing)	56
8.6 Manipolazione dei Descrittori a Runtime (fcntl)	57
8.7 Il Flag O_NONBLOCK e l'Errore EAGAIN	57
8.8 Lock sui File (File Locking) e Sincronizzazione	58
8.9 Posizionamento nel File: Astrazioni della Libreria stdio.h	58
8.9.1 Interazione Diretta con il Dispositivo (ioctl)	59
8.9.2 L'I/O Bufferizzato: Gli Stream della Libreria Standard	59
Politiche di Bufferizzazione	59
Apertura Avanzata degli Stream (freopen e fdopen)	60
Interazione con i File Descriptor (fileno)	60
Funzioni di I/O Orientate agli Stream	60
I/O Formattato	61
Posizionamento negli Stream	61
File Temporanei	62
Memory Streams (Stream in Memoria)	62
9 - Consistenza della Memoria e Riordinamento delle Istruzioni	63
9.1 Il Livello Software: Il Compilatore e volatile	63
9.2 Il Livello Hardware: Store Buffer, Cache e Invalidazione	64
9.2.1 Concorrenza Hardware	64
9.3 I Modelli di Consistenza della Memoria	65
9.3.1 Consistenza Locale (Local Consistency)	65
9.3.2 Consistenza Sequenziale (Sequential Consistency - SC)	66
9.3.3 Consistenza Causale (Causal Consistency)	66
9.3.4 Consistenza P-RAM (Pipelined RAM)	66
9.3.5 Consistenza di Cache (Cache Consistency)	66
9.3.6 Consistenza di Processore (Processor Consistency)	66
9.3.7 Slow Consistency (SC)	67
9.4 Modelli ibridi	67

9.4.1 Il Modello di Consistenza "Weak" (Debole)	67
9.4.2 Release consistency	68
9.4.3 Entry consistency	68
9.5 Barriere del Compilatore	68
9.5.1 Sincronizzazione Software: Le Barriere di Memoria (Memory Fences)	69
9.5.2 Esempi di architetture	69
10 - Sincronizzazione	72
10.1 Problemi Fondamentali della Concorrenza	72
10.2 Il Problema della Sezione Critica	72
10.3 Supporto Hardware di Basso Livello	73
10.4 Primitive di Sincronizzazione di Basso Livello	73
10.4.1 Lo Spinlock (Lock ad Attesa Attiva)	73
10.4.2 Lock Lettori-Scrittori (Reader-Writer Locks)	74
10.4.3 Lock Sequenziali (Sequential Locks / Seqlocks)	74
10.5 Strutture Dati Lock-Free (Cenni)	75
10.6 Granularità dei Lock nei Sistemi Database	75
10.7 Strategie di Lock	75
10.8 Problemi di locking	76
10.8.1 Il Deadlock	76
10.8.2 L'Inversione di Priorità (Priority Inversion)	76
10.8.3 Lock ed Esecuzione in Spazio Interruzione (Interrupt Handlers)	77
10.8.4 Prestazioni e Contesa	77
10.9 Primitive di Sincronizzazione di Alto Livello	77
10.9.1 I Semafori	78
10.9.2 Mutex	78
10.9.3 Le Variabili di Condizione (Condition Variables)	78
La Semantica Mesa e il Rischio di Risveglio Spurio	79
10.10 Pattern Architettureali di Sincronizzazione	79
10.11 Condizioni di deadlock	80
11 – I/O Avanzato	82
11.1 Locking sui File (File Locking)	82
11.2 I/O Bloccante e Non Bloccante	82
11.3 Multiplexing dell'I/O (select e poll)	83
11.3.1 La funzione select	83
11.3.2 La funzione poll (e varianti)	83
11.4 I/O Vettoriale (Scatter-Gather I/O)	84
11.5 Memory Mapping (La Funzione mmap)	84
12 – Segnali	86
12.1 Ciclo di Vita	86
13 – Gestione della Memoria Fisica	87
13.1 Allocazione a Partizioni e Frammentazione Esterna	87
13.2 Compattazione e Rilocalizzazione	87
13.3 Introduzione al Buddy System	88
13.3.1 Implementazione tramite Albero Binario	88

13.4 Il Vincolo di Contiguità Fisica per le Periferiche	88
14 – Gestione della Memoria Logica	90
14.1 Il Concetto di Segmento come Unità Logica	90
14.2 Meccanismo di Traduzione e Tabella dei Segmenti	90
14.3 Protezione, Isolamento e Condivisione	91
14.4 Frammentazione Esterna	92
15 – Paginazione	93
15.1 Pagine e Frame	93
15.2 Struttura dell'Indirizzo e Page Table	93
15.3 Il Ruolo della MMU e il TLB	94
15.3.1 Gestione di TLB Hit e TLB Miss	94
15.3.2 Overhead nel Cambio di Contesto e ASID	95
15.4 Tabella delle Pagine Multi-livello	95
15.5 Tabella delle Pagine Invertita (Inverted Page Table)	96
15.6 Architetture Ibride: Segmentazione Paginata	96
16 – Memoria Virtuale, Gestione dei Page Fault e Algoritmi di Sostituzione	98
16.1 Fondamenti della Memoria Virtuale e Page Fault	98
16.2 Il Working Set e Tempo Effettivo di Accesso (EAT)	99
16.3 Page Replacement	100
16.3.1 Algoritmo FIFO (First-In, First-Out)	100
16.3.2 Algoritmo LRU (Least Recently Used)	101
16.3.3 Algoritmo NRU, Second Chance e Clock (Approssimazioni di LRU)	101
16.4 Allocazione dei Frame e Thrashing	102
16.4.1 Il Problema del Thrashing (Iperpaginazione)	102
16.5 Copy-On-Write e Page Pinning	102
17 – Pipe Anonime	104
17.1 Definizione e Proprietà Fondamentali	104
17.2 Meccanica dei Buffer e Atomicità delle Scritture	104
17.3 Creazione e Invocazione della System Call pipe	105
17.4 Comportamento delle Primitive read e write	105
17.4.1 Dinamiche della Chiamata read	105
17.4.2 Dinamiche della Chiamata write	106
17.5 Pattern d'Uso Canonico tramite fork	106
18 – Pipe con Nome (FIFO)	107
18.1 Le Pipe con Nome (FIFO) come File Speciali	107
18.2 Creazione, Apertura e Sincronizzazione delle FIFO	107
18.3 Astrazioni di Alto Livello: popen e pclose	108
18.4 Altre Primitive di IPC POSIX	108
19 – Esercizi	110
19.1 Atomicità nei Gestori di Segnale	110
19.2 Struttura di una Pipeline Circolare	110
20 – Gestione dei Dischi, Partizionamento e Mapping Logico dei Volumi	112
20.1 Basso Livello vs Alto Livello	112
20.2 MBR (Master Boot Record) (schema vecchio)	112

20.2.1 Estensione del Limite delle 4 Partizioni	112
20.2.2 Il Limite dei 32 bit	113
20.3 GPT (GUID Partition Table) (schema moderno)	113
20.3.1 Allocazione Protettiva (Protective MBR)	114
20.4 Mapping Logico dei Volumi (LVM)	114
20.5 Gestione Blocchi Difettosi	115
20.6 Analisi dei Tempi di Accesso nei Dischi Magnetici	116
20.7 Algoritmi di Scheduling	116
20.7.1 FCFS (First-Come, First-Served)	116
20.7.2 SSTF (Shortest Seek Time First)	117
20.7.3 Algoritmo SCAN (o dell'Ascensore)	117
20.7.4 Algoritmo C-SCAN (Circular SCAN)	117
20.7.5 Algoritmi LOOK e C-LOOK	118
20.7.6 Algoritmi N-Step SCAN e F-SCAN	118
21 – Architettura SSD	119
21.1 Pagine e Blocchi	119
21.2 Scrittura Fuori Posto (Out-of-Place Write)	119
21.3 Flash Translation Layer (FTL) e Garbage Collection	120
21.4 Il Livellamento dell'Usura (Wear Leveling)	120
21.4.1 Impatto sulle Metriche di Scheduling	120
22 – File System	121
22.1 Struttura Logica delle Directory	121
22.2 Layout del Disco per il File System	121
22.2.1 Il Superblocco (Superblock / Boot Sector)	121
22.2.2 L'Unità di Allocazione: I Cluster	122
22.3 Strategie di Allocazione dei File	122
22.3.1 Allocazione Contigua	122
22.3.2 Allocazione Concatenata (Linked Allocation / File Allocation Table)	122
22.3.3 Allocazione Indicizzata (I-node con Blocchi Indiretti)	123
22.4 Implementazione delle Directory e Spazio Libero	124
23 – File System Avanzati	125
23.1 Consistenza dei Metadati: I File System Journaled	125
23.2 Astrazione del Kernel: Il Virtual File System (VFS)	125
24 - Dispositivi di I/O	127
24.1 Accesso	127
25 - Interrupt	128
25.1 Gestione delle Interruzioni (Interrupt Handling)	128
25.1.1 Il Conflitto Tempestività ed Elaborazione Lunghe	128
25.1.2 Suddivisione in Top Half e Bottom Half	128

1 - Fondamenti dei Sistemi Operativi e Architettura delle System Call

[Il corso si avvale della piattaforma Teams; le credenziali per le aree ad accesso ristretto sono fornite separatamente per evitarne l'indicizzazione sui motori di ricerca. Il docente è disponibile via email o di persona per eventuali chiarimenti. Il programma tratterà la struttura astratta dei sistemi operativi, la programmazione in ambiente Unix (il quale rappresenta lo standard de facto, base di Linux, macOS, iOS, Android e modello verso cui converge anche Windows), la gestione di processi e thread, la consistenza della memoria nei sistemi paralleli, la sincronizzazione, la gestione dell'I/O e i file system. Vengono richiesti come prerequisiti una conoscenza fluente della programmazione in C e dell'architettura dei calcolatori. Le lezioni registrate fungono esclusivamente da backup, in quanto la qualità audio può variare e le spiegazioni alla lavagna non saranno visibili. La parte relativa alla programmazione a livello kernel non verrà trattata per questioni di tempo.]

1.1 Architettura Generale del Sistema Operativo

Un sistema operativo non è un'entità monolitica, bensì un complesso strato software (*stack*) progettato per astrarre l'hardware sottostante e fornire un ambiente di esecuzione sicuro e controllato per i programmi applicativi. Esso si articola in tre componenti fondamentali.

- **Il Kernel.** Costituisce il nucleo privilegiato del sistema. Si occupa della gestione diretta dell'hardware, dell'allocazione delle risorse fisiche (memoria, CPU, periferiche) e della coordinazione delle entità astratte quali i processi.
- **Le Librerie di Base.** Codice non privilegiato che funge da supporto per le applicazioni. L'esempio canonico è la libreria standard del C (`libc`) o la libreria matematica: esse implementano funzioni ricorrenti (come `printf` o le funzioni trigonometriche) evitando che ogni sviluppatore ne riscriva l'implementazione da zero.
- **Le Applicazioni di Base.** Programmi forniti a corredo del sistema per permetterne l'utilizzo immediato. Includono editor di testo, interpreti di comandi (Shell) e strumenti per la gestione degli utenti, dei dischi e delle reti.

1.1.1 Accesso al Sistema: Autenticazione, Identificatori e Shell

Se il kernel deve garantire protezione e isolamento, è fondamentale che il sistema sappia con esattezza quale entità stia eseguendo un determinato processo. In architettura UNIX, l'accesso (*login*) e la gestione degli utenti sono regolati da identificatori numerici rigorosi.

- **User ID (UID):** Ogni utente è rappresentato dal sistema operativo tramite un numero intero non negativo. È questo numero, e non il nome testuale (*login name*), a determinare i permessi su file e processi.

- **L'utente Root (UID 0):** L'identificatore 0 è universalmente riservato al *superuser* (amministratore di sistema), un'utenza speciale che scavalca i normali controlli di accesso disponendo di privilegi assoluti sulla macchina.
- **Group ID (GID):** Gli utenti possono essere raggruppati logicamente tramite i GID, permettendo la condivisione di risorse e file tra team di lavoro.

Una volta superata l'autenticazione, l'utente interagisce con il sistema tramite una **Shell** (interprete dei comandi). L'ecosistema UNIX offre diverse varianti storiche, ognuna con propria sintassi:

- **sh:** la storica Bourne shell;
- **bash:** la Bourne-Again Shell, standard de facto sulla maggior parte dei sistemi GNU/Linux;
- **csh:** la C shell, la cui sintassi richiama il linguaggio C;
- **ksh:** la Korn shell;
- **tcsh:** variante avanzata della C shell.

1.2 Funzioni e Componenti Interne del Kernel

Il kernel ha il compito di nascondere all'utente la complessità e l'eterogeneità dell'hardware. Indipendentemente dal produttore di un dispositivo di storage, il kernel traduce una richiesta generica (es. "leggi il blocco 5") in istruzioni elettriche specifiche per quel dispositivo. Lo stesso principio si applica alle interfacce di rete e a tutte le altre periferiche.

Per orchestrare questo livello di astrazione, il kernel si avvale dei seguenti moduli interni:

- **Device Driver:** Moduli software incaricati di comunicare direttamente con l'hardware, inviare comandi e gestire gli *interrupt* (segnali asincroni inviati dalle periferiche al processore).
- **Scheduler:** L'algoritmo che determina quale processo, tra i tanti in attesa, debba ottenere l'accesso al processore e per quanto tempo.
- **Gestione della Memoria:** Il kernel alloca e dealloca la memoria richiesta dai processi, isola gli spazi di indirizzamento per prevenire interferenze (*protezione*) e gestisce la memoria condivisa quando i processi devono cooperare.
- **Gestione del Tempo:** Regolata da clock interni (per i processi) e *wall time* (per l'ora di sistema, conteggiata in secondi a partire dall'*Epoch* Unix: 1 gennaio 1970, ore 00:00 UTC); è fondamentale per implementare timeout di rete, pause (**sleep**) e per consentire allo scheduler di sottrarre la CPU a un task che la monopolizza.
- **Gestione del File System:** Organizzazione strutturata e gerarchica dei dati persistenti su dispositivi di storage.
- **Gestione di Cache e Buffer:** Meccanismo critico per colmare il divario prestazionale tra la CPU (ordine dei nanosecondi) e le periferiche di I/O (ordine dei microsecondi o millisecondi). Il kernel precarica i dati in RAM per evitare che il processore rimanga inattivo in attesa di periferiche lente.
- **Gestione della Sincronizzazione:** Ogni processo che condivide risorse o comunica con altri processi deve essere coordinato. Ad esempio, se il processo B necessita del

risultato prodotto dal processo A, deve attenderne il completamento prima di procedere.

L'interazione con l'esterno avviene tramite un'interfaccia di I/O estremamente astratta e uniforme: che si tratti di un file su disco o di una porta seriale, il programmatore utilizzerà sempre il medesimo paradigma: `open` per iniziare, `read` per acquisire dati, `write` per inviare dati, `close` per terminare. Per operazioni specifiche e non standardizzabili (es. variare il baud rate di una porta seriale), si ricorre alla funzione `ioctl` (*Input/Output Control*).

1.2.1 Programma e Processo: una Distinzione Fondamentale

È essenziale distinguere formalmente due concetti frequentemente confusi:

- **Programma:** Entità statica; un file eseguibile salvato fisicamente su storage.
- **Processo:** Entità dinamica; l'istanza in esecuzione di quel programma, caricata in memoria e gestita dallo scheduler.

Ogni processo è identificato univocamente da un **PID** (*Process ID*), un numero intero non negativo. I processi UNIX sono organizzati in una **gerarchia ad albero** basata sulla relazione padre-figlio (*parent-child*), gestita tramite specifiche System Call:

- `fork()`: duplica il processo padre creando un processo figlio quasi identico;
- `exec()`: sostituisce il codice del processo figlio con quello di un nuovo programma;
- `waitpid()`: sospende il processo padre in attesa della terminazione del figlio.

I processi possono inoltre essere raggruppati in *Process Groups* per permettere l'invio di segnali a interi blocchi di esecuzione simultaneamente.

All'interno di un singolo processo possono coesistere più flussi di esecuzione paralleli, denominati **Thread**. Ogni flusso è talvolta indicato anche come **Task**, termine generico per indicare un'entità astratta che richiede l'utilizzo del processore per eseguire istruzioni.

- Il **PID 1** è assegnato al processo di inizializzazione del sistema (tradizionalmente `init`), responsabile della configurazione dell'ambiente prima che gli utenti possano accedere.
- Un processo può contenere più **thread**, ciascuno dotato di un proprio identificativo numerico.

1.3 Livelli di Privilegio e Meccanismi di Protezione

L'affidabilità di un sistema richiede una compartimentalizzazione rigorosa. Il kernel deve proteggere il sistema sia da bug accidentali (es. un puntatore dereferenziato in modo errato) sia da attacchi software malevoli. A tal fine, l'hardware supporta due modalità operative distinte:

- **User Mode (Modalità Utente):** Livello a basso privilegio in cui girano le normali applicazioni. Il codice in questa modalità ha accesso ristretto alla memoria e non può

interagire direttamente con i dispositivi hardware. Qualsiasi violazione causa la terminazione forzata del processo, senza compromettere il sistema.

- **Kernel Mode (Modalità Privilegiata / Supervisor):** Livello ad alto privilegio, riservato esclusivamente al kernel, in cui il codice gode di accesso incondizionato all'intero sistema e alla memoria fisica. Un errore in questa modalità può causare un blocco totale del sistema (*kernel panic*).

1.4 Le System Call: L'Interfaccia Hardware/Software

Quando un programma in User Mode necessita di un'operazione privilegiata (come l'apertura di un file), non può invocare direttamente il kernel tramite una normale istruzione di salto, poiché ciò violerebbe la sicurezza: l'utente potrebbe altrimenti specificare arbitrariamente a quale indirizzo del kernel saltare.

Il passaggio avviene tramite una **System Call** (*Syscall*), un meccanismo dipendente dall'architettura del processore, che si articola nei seguenti passi:

1. **Transizione Architetturale:** Il processore commuta dalla modalità utente a quella privilegiata.
2. **Salto a Indirizzo Blindato:** Il flusso di esecuzione viene deviato a un indirizzo predeterminato dai progettisti del kernel, non scrivibile dall'utente.
3. **Validazione Rigorosa:** Il kernel verifica la validità dei puntatori forniti e i permessi dell'utente richiedente.
4. **Ritorno al Chiamante:** I risultati (o i codici di errore) vengono restituiti al processo utente.

Poiché il cambio di privilegio (context switch) ha un costo computazionale (*overhead*), le system call vengono invocate solo quando strettamente necessario. Alcune operazioni semplici (come la lettura dell'ora corrente) vengono ottimizzate in certi sistemi mappando aree di memoria in sola lettura accessibili direttamente a livello utente, evitando la transizione al kernel.

1.4.1 Le API (Application Programming Interface)

L'implementazione tecnica delle System Call differisce profondamente non solo tra sistemi operativi diversi (Unix vs. Windows), ma anche tra architetture hardware distinte per lo stesso sistema operativo (Intel vs. ARM). Per questa ragione, i programmatori non invocano le Syscall direttamente, bensì utilizzano librerie intermedie (come `libc` per il C). Una funzione API come `open()` funge da *wrapper*: nasconde la sintassi assembler specifica e chiama internamente la Syscall appropriata, garantendo la portabilità del codice sorgente su macchine diverse.

1.5 Implementazione Architeturale delle Syscall (Casi Studio)

1.5.1 Architettura ARM a 32 bit

Nei sistemi ARM a 32 bit (diffusi in ambito embedded e mobile), la transizione al kernel avviene tramite l'istruzione **SVC** (*Supervisor Call*, originariamente denominata **SWI**, *Software Interrupt*). Questa porta il processore in modalità privilegiata e forza un salto incondizionato all'indirizzo 8 (seguito da un salto esplicito alla routine interna **Vector SWI**).

L'Ottimizzazione della Cache: I processori ARM usano istruzioni da 32 bit. Inizialmente si pensò di inserire il numero identificativo della Syscall direttamente nei bit liberi dell'istruzione **SVC**. Questa tecnica, pur risparmiando l'uso di registri, si rivelò inefficiente per via dell'architettura della cache. Le CPU moderne dispongono di due cache separate: una per le istruzioni (*I-Cache*) e una per i dati (*D-Cache*). Se il processore dovesse leggere la propria istruzione per estrarne il numero, tale lettura avverrebbe come accesso dati: poiché l'istruzione risiede nella I-Cache, la D-Cache registrerebbe un *miss*, forzando una lettura lenta dalla memoria principale.

La Soluzione Adottata: Per evitare questo stallo, i kernel come Linux inseriscono il valore nullo (**0**) nell'istruzione **SVC** e passano il numero della Syscall tramite il registro **R7**. Gli argomenti vengono allocati nei registri da **R0** a **R6**, e il risultato è restituito in **R0**.

1.5.2 Architetture Moderne

- **ARM a 64 bit:** Il meccanismo logico rimane invariato. I livelli di privilegio sono rinominati: **EL0** (basso privilegio, utente) e **EL1** (alto privilegio, kernel). Il salto avviene a un indirizzo specificato da un registro di sistema non modificabile dall'utente.
- **MIPS a 32 bit:** Si utilizza l'istruzione **syscall**. Il salto avviene a un indirizzo base contenuto in un registro inaccessibile dalla modalità User.
- **Intel (x86/x64):** L'architettura Intel prevede quattro livelli di privilegio denominati *Ring* (da 0 a 3, dove Ring 0 è riservato al kernel e Ring 3 all'utente). Nella variante a 32 bit, la Syscall veniva gestita tramite un interrupt software (**int 0x80**, ossia l'interrupt 128). Nella variante a 64 bit, è stata introdotta l'istruzione **syscall**, che forza il passaggio a Ring 0 e il salto a un indirizzo definito da registri speciali; il registro **RAX** trasporta il numero della Syscall e i risultati, mentre altri sei registri gestiscono gli argomenti.

A riprova dell'importanza dell'astrazione tramite API: la medesima system call **close** è mappata sul numero **6** in ARM 32-bit, sul numero **57** in ARM 64-bit e sul numero **3** nei processori Intel a 64-bit. È il compilatore, unitamente alla libreria di base, a farsi carico della traduzione architeturale, sollevando il programmatore dall'onere di gestire manualmente i codici macchina ai vari livelli sottostanti.

2 – Evoluzione delle Architetture Hardware e dei Sistemi Operativi

[Il corso adotterà UNIX, ed in particolare la sua implementazione open source Linux, come sistema operativo di riferimento per le prove pratiche, in quanto permette di ispezionare e modificare il codice sorgente. Molte delle transizioni storiche aziendali trattate servono primariamente a comprendere come le limitazioni hardware abbiano forzato le scelte architetturali nel software.]

2.1 La Prima Generazione (1945–1955): Tubi a Vuoto e Assenza di Software

I primi calcolatori elettronici, nati a cavallo della Seconda Guerra Mondiale, erano macchine di enormi dimensioni basate sulla tecnologia dei **tubi a vuoto** (valvole).

Caratteristiche Hardware: Le valvole modulavano il flusso di elettroni secondo un principio analogo a quello dei transistor moderni, ma richiedevano tensioni elevatissime e generavano calore estremo, portando a consumi energetici immensi.

Programmazione Manuale: In questa fase non esistevano sistemi operativi né linguaggi di programmazione. La macchina veniva configurata a livello fisico: gli operatori attivavano manualmente interruttori o ricollegavano cavi per impostare le istruzioni.

Schede Perforate: L'input fu successivamente standardizzato tramite schede perforate e nastri di carta. Ogni colonna di una scheda rappresentava un carattere alfabetico o numerico, codificato dalla posizione dei fori.

Destinazione d'uso: Si trattava di macchine costruite ad hoc per calcoli numerici puri, come tabelle trigonometriche o il calcolo di traiettorie balistiche.

2.2 La Seconda Generazione (1955–1965): Transistor, Mainframe e Batch Processing

L'invenzione del transistor rivoluzionò l'architettura dei calcolatori. Questi dispositivi a stato solido operavano a tensioni molto più basse (5–10 V), riducendo drasticamente consumi e dimensioni: i computer passarono dall'occupare interi edifici a occupare singole stanze.

Supercomputer e Mainframe: Si delinearono due linee evolutive distinte. I **Supercomputer** (come il Cray-1 di Seymour Cray) erano focalizzati esclusivamente sulla massima potenza di calcolo numerico. I **Mainframe** (dominati storicamente da IBM) erano orientati alla gestione dati per applicazioni aziendali.

Memorie e I/O: La RAM era costituita da **anelli di ferrite magnetizzati**. L'input/output transitava su nastri magnetici. Il flusso operativo era interamente indiretto: l'operatore caricava le schede perforate in un lettore che generava un nastro magnetico; questo veniva

elaborato dal Mainframe, che scriveva i risultati su un nastro di output, infine stampato su carta da una macchina separata.

Il Monitor e il Batch Processing: Per ridurre i tempi morti tra un'esecuzione e l'altra, nacque il primo rudimentale software di sistema: il **Monitor**. Questo programma controllava l'esecuzione sequenziale di una lista di lavori (*batch*), ripulendo la memoria tra un task e il successivo.

I Primi Linguaggi: Per elevare l'astrazione oltre il codice Assembly, nacquero i primi linguaggi di programmazione: **FORTRAN** (*FORmula TRANslator*, per il calcolo scientifico) e **COBOL** (per applicazioni gestionali).

Un mainframe dell'epoca era composto da armadi metallici (*Cabinets*) interconnessi, contenenti schede a circuito stampato (PCB) con componenti discreti (transistor, resistori, diodi) collegate su un pannello posteriore (*Backplane*). Il cablaggio poteva essere fisicamente modificato per correggere errori logici.

2.3 La Terza Generazione (1965–1980): Circuiti Integrati e Famiglie di Macchine

L'introduzione dei **circuiti integrati** (chip), che raggruppavano più porte logiche su un singolo frammento di silicio (scale SSI, MSI, LSI), portò a un'ulteriore miniaturizzazione e riduzione dei costi.

Compatibilità Architeturale: IBM introdusse il **System/360**: una famiglia di macchine compatibili fra loro che condividevano lo stesso set di istruzioni, permettendo ai clienti di scegliere il modello adatto al budget con la garanzia della piena portabilità del software.

Terminali: Scompare l'uso esclusivo della console centrale. Nacquero i **terminali** (*TTY*, da *TeleTYpewriter*) con tastiera e schermo/stampante, collegabili anche da remoto.

Time-Sharing: Poiché le periferiche di I/O erano molto più lente della CPU, per non sprecare cicli di clock nacque il **Time-Sharing**: il sistema operativo permette a più programmi (e più utenti) di avanzare in modo apparentemente parallelo, sfruttando i tempi morti di ciascun processo.

Primo Sistema Operativo: Nacque l'**IBM OS/360** (con la variante MFT per il multitasking a numero fisso di task).

Problemi di Memoria: L'allocazione di partizioni di memoria ai vari processi generava "buchi" non riutilizzabili (**frammentazione**). Poiché l'hardware non consentiva ancora la rilocalizzazione dinamica degli indirizzi, fu introdotto lo **Swapping**: congelare un processo copiando la sua intera immagine dalla RAM al disco magnetico per liberare spazio.

2.4 Il Progetto Multics e la Nascita di UNIX

Presso i Bell Labs (AT&T), i ricercatori concepirono un sistema operativo rivoluzionario chiamato **MULTICS**, progettato per gestire parallelismo, multiutenza e astrazioni avanzate. Multics introdusse concetti tutt'oggi rilevanti:

- **Memory Mapped Files:** Accesso ai dati su disco trattandoli come indirizzi di memoria RAM, senza operazioni esplicite di lettura/scrittura.
- **File System Virtuale Unificato:** Montare periferiche di storage eterogenee sotto un'unica gerarchia ad albero, mascherando i formati fisici differenti.
- **Dynamic Linking:** Collegare le librerie al programma non in fase di compilazione, ma direttamente in memoria durante l'esecuzione; ciò risparmia spazio su disco ed evita la duplicazione del codice.
- **Paginazione e Segmentazione:** Tecniche avanzate di virtualizzazione della memoria per la gestione efficiente del multitasking.

2.4.1 La Filosofia di UNIX

Unix fu ideato da **Ken Thompson** nei Bell Labs come versione ridotta di MULTICS, progettata per girare su minicomputer. L'architettura UNIX si basa su un approccio minimalista ed elegante:

- **Semplicità:** Costruire programmi piccoli che svolgono una singola funzione in modo efficiente, anziché sistemi monolitici.
- **Combinabilità (Pipe):** I programmi comunicano tramite flussi di testo (*text streams*). Tramite il simbolo `|`, l'output di un programma diventa l'input del successivo. Esempio: `cat` (legge un file) → `grep` (filtra le righe) → `wc` (conta le occorrenze).
- **Tutto è un File:** Dispositivi e periferiche sono astratti come "file virtuali" (es. `/dev/tty1`) accessibili tramite le primitive standard `open`, `read`, `write`, `close`.
- **Gestione Processi (Fork/Exec):** Un processo si duplica tramite `fork()` e il clone sostituisce la propria immagine in memoria con un nuovo eseguibile tramite `exec()`, permettendo la configurazione dinamica dei canali I/O (il meccanismo su cui si basa la Shell per costruire le pipeline).

2.5 Il Linguaggio C e la Distribuzione di UNIX

Inizialmente scritto in Assembly, UNIX richiedeva un linguaggio ad alto livello capace sia di esprimere astrazioni complesse sia di interagire a basso livello con l'hardware. La genealogia del linguaggio C è la seguente:

CPL (1963, *Combined Programming Language*) → **BCPL** (1967, *Basic CPL*) → **B** (1969, Thompson & Ritchie; privo di sistema di tipi, fonte di errori difficili da individuare) → **C** (1972, Dennis Ritchie; introduce un sistema di tipi che, pur permissivo, fornisce avvertimenti sulle conversioni implicite).

UNIX fu interamente riscritto in C, diventando il primo sistema operativo facilmente portabile su architetture diverse.

2.6 La Quarta Generazione (1980–Presente): L'Era dei Personal Computer

La tecnologia VLSI (*Very Large Scale Integration*) rese possibile l'integrazione di un intero microprocessore su un singolo chip. I principali attori (Intel, Zilog, Motorola, MOS Technology) portarono alla nascita dei primi Personal Computer.

Sistemi Operativi del Periodo:

- **CP/M (1974, Digital Research):** Primo software di sistema diffuso per microcomputer. Ogni periferica di storage era identificata con una lettera seguita da due punti (es. **A:**); i nomi file seguivano il formato **8.3**.
 - **MS-DOS / Windows:** Microsoft, a partire da MS-DOS, sviluppò dapprima il sistema a 16 bit con multiprogrammazione cooperativa e segmenti da 64 KB; successivamente, con Windows 95, introdusse il multitasking preemptivo, la paginazione e segmenti a dimensione variabile su processori a 32 bit. Dal 1993, con **Windows NT** (*New Technology*), Microsoft realizzò un kernel completamente nuovo, con spazio di indirizzamento piatto a 32 bit e multitasking preemptivo nativo.
 - **Mac OS:** Apple sviluppò internamente hardware e software, adottando inizialmente un'interfaccia grafica (GUI) con multitasking cooperativo. Nel 2001, con **Mac OS X**, riscrisse il sistema operativo attorno al kernel **XNU**, derivato da UNIX e certificato conforme alla Single UNIX Specification.
 - **Linux:** Nei primi anni '90, lo studente **Linus Torvalds** sviluppò un kernel UNIX-like per i processori Intel 386 domestici. Rilasciandolo in modalità open source e invitando sviluppatori da tutto il mondo a collaborare via Internet, ha dato origine all'ecosistema Linux.
-

3 – Architettura di Sistema, I/O e Standard POSIX

3.1 Livelli Architeturali e Utilità di Sistema

Il sistema operativo è organizzato in strati gerarchici. Il kernel costituisce l'interfaccia verso l'hardware; al di sopra risiedono le System Call, le librerie (API) e le applicazioni.

[Viene esplicitato che il codice appartenente al Kernel viene eseguito principalmente in Kernel Mode (la modalità ad alto privilegio), garantendo così l'accesso diretto e incondizionato all'hardware. Al contrario, tutto ciò che risiede ai livelli superiori – ovvero le librerie (comprese le routine comuni e le astrazioni del sistema operativo) e le applicazioni (interpreti di comandi, utility di sistema, utility per gli utenti e per lo sviluppo) – viene eseguito esclusivamente in User Mode (modalità non privilegiata). Questa barriera architettonica è ciò che rende le System Call l'unico ponte di comunicazione sicuro e controllato per l'accesso alle risorse hardware.]

Le applicazioni possono interfacciarsi con il kernel in due modi:

1. **Tramite librerie (raccomandato):** si sfrutta l'astrazione dell'API, ottenendo portabilità tra versioni e famiglie di sistemi operativi.
2. **Tramite System Call dirette (sconsigliato nella norma):** si perde l'astrazione della libreria; necessario solo per accedere a funzionalità molto recenti non ancora incluse nelle librerie disponibili.

Utilità di Sistema: Le applicazioni incluse nel sistema operativo si suddividono in:

- *Utilità di sistema:* configurazione di rete, partizionamento dello storage, gestione degli utenti (richiedono privilegi elevati).
- *Utilità utente:* listare directory, mostrare il contenuto di file, modificare permessi.
- *Strumenti di sviluppo:* compilatori, linker, gestori di librerie (talvolta opzionali e venduti separatamente; i sistemi Linux li includono tipicamente).

3.2 Input/Output e File Descriptor

I processi comunicano e producono risultati attraverso entità astratte identificate da numeri interi non negativi, i **File Descriptor**, con cui il kernel traccia le risorse aperte da ciascun processo. I File Descriptor possono essere file fisici o canali di comunicazione a cui si accede tramite primitive standard (**open**, **read**, **write**, **close**).

Canali Standard di I/O:

- **0 Standard Input (`stdin`)**: Canale di ingresso (tastiera per default)
- **1 Standard Output (`stdout`)**: Canale di uscita primario
- **2 Standard Error (`stderr`)**: Canale per messaggi diagnostici; tende a bypassare i buffer in user space assicurando che gli avvisi di errore giungano all'utente anche se il programma dovesse subire un blocco critico.

Gestione dei Buffer:

- Le operazioni di I/O invocate direttamente tramite System Call (`read`, `write`) sono dette **non bufferizzate**: ogni chiamata corrisponde esattamente a una system call verso il kernel, senza buffer interposti in user space.
- Le funzioni della libreria C standard (`printf`, `fwrite`, ecc.) sono invece **bufferizzate**: i dati vengono accumulati in un buffer gestito dalla libreria prima che venga emessa una singola system call.
- Lo standard error è tendenzialmente non bufferizzato (o a buffer riga) per garantire la notifica tempestiva degli errori, anche in caso di blocco critico del processo.

3.3 Il File System: Percorsi, Tipi e Lock

Il file system Unix è strutturato come un'unica gerarchia ad albero con radice nel carattere `/`.

Tipi di File:

- **File regolari**: unità logiche di memorizzazione dati persistente.
- **Directory**: contenitori di file regolari o speciali.
- **File speciali**: identificano dispositivi hardware (porte seriali, dischi), socket di rete o pipe per la comunicazione tra processi.
- **Link simbolici**: file speciali contenenti il percorso di un altro file.

Percorsi (Paths): Un percorso che inizia con `/` è **assoluto** (parte dalla radice); un percorso privo di `/` iniziale è **relativo** alla *current working directory* del processo.

Memory-Mapped Files: Il kernel permette di mappare un file direttamente nello spazio di indirizzamento di un processo. Il programmatore accede ai dati del file tramite semplice dereferenziazione di un puntatore in RAM, sfruttando i buffer interni del kernel. Questo meccanismo è estremamente efficiente per accessi casuali.

File Lock: Per garantire la consistenza dei dati all'interno di un file in caso di accesso concorrente da parte di più processi, il sistema supporta l'**applicazione di lock** su specifiche regioni. Il lock è di due tipi:

- **Advisory (suggerito)**: I processi devono cooperare verificando volontariamente lo stato del lock prima di operare. Un processo mal scritto che ignori il controllo può comunque accedere all'area bloccata.
- **Mandatory (obbligatorio)**: Il blocco è imposto a livello di kernel; qualsiasi tentativo di accesso a un'area bloccata da un altro processo fallisce. Storicamente si è rivelato

problematico e implementato in modo lacunoso; la prassi comune preferisce i lock *advisory*.

3.4 Gestione degli Errori e Segnali

La Variabile `errno`: Quando una system call fallisce, restituisce convenzionalmente il valore `-1` e scrive un codice numerico nella variabile globale `errno`. Il valore `errno = 0` indica assenza di errori. Codici comuni:

- `ENOMEM`: memoria insufficiente;
- `EAGAIN`: riprovare l'operazione più tardi;
- `EACCES`: permesso negato.

Funzioni di Traduzione:

- `strerror(errno)`: converte il codice numerico in una stringa leggibile.
- `perror("messaggio")`: stampa direttamente su `stderr` il messaggio specificato concatenato alla descrizione dell'errore corrente.

Segnali: Il kernel notifica eventi ai processi tramite segnali. Essi possono essere:

- **Asincroni:** indipendenti dal punto di esecuzione del processo (es. I/O pronto, invio da un altro processo).
- **Sincroni:** scaturiti da una specifica istruzione del processo (es. `SIGILL` per istruzione illegale, `SIGSEGV` per violazione della protezione della memoria).

`SIGKILL` non può essere né gestito né ignorato dal processo, garantendo al sistema la capacità di forzare la terminazione di qualsiasi processo.

3.5 Tempo e Comunicazione Inter-Processo (IPC)

Tempo di Calendario: Rappresenta l'istante nel mondo reale, conteggiato in secondi dall'Epoch Unix (1° gennaio 1970). È memorizzato nel tipo `time_t`, esteso a 64 bit nei sistemi moderni per evitare l'overflow del 2038.

Tempo di Processo: Misura il tempo di esecuzione di un processo specifico, distinguendo tra tempo in modalità utente e tempo in modalità kernel. L'unità di misura è il *clock tick*, gestito dal tipo `clock_t`.

Comunicazione Inter-Processo (IPC):

- *Scambio di messaggi:* pipe, code di messaggi, socket; richiede l'intervento del kernel per ogni trasferimento.
- *Memoria condivisa:* un'area di memoria fisica è mappata negli spazi di indirizzamento di due o più processi; lo scambio è diretto e veloce, ma richiede sincronizzazione esplicita.
- *Primitive di sincronizzazione:* semafori, mutex, variabili di condizione, barriere.

3.6 Standardizzazione: Libreria C e POSIX

Standard C (C89/C99): Definisce strutture, limiti e tipi. Header principali di interesse:

- `<limits.h>`: Limiti dei tipi base (es. `INT_MAX`, `CHAR_BIT`)
- `<errno.h>`: Codici di errore e variabile `errno`
- `<stdio.h>`: I/O standard (`printf`, `scanf`, `fopen`, ecc.); tipo `FILE`
- `<string.h>`: Manipolazione stringhe; tipo `size_t`
- `<stdlib.h>`: Allocazione memoria, conversioni, `exit`, `abort`, `getenv`
- `<stdint.h>`: Interi a dimensione fissa (es. `int16_t`, `uint32_t`); critico per interfacciarsi con registri hardware
- `<inttypes.h>`: Macro di formato per `printf/scanf` con tipi `<stdint.h>`
- `<time.h>`: Tipi `time_t`, `clock_t`; funzioni temporali
- `<signal.h>`: Definizioni dei segnali

Tipi generici fondamentali:

- `size_t`: intero senza segno per rappresentare la dimensione di oggetti in memoria (usato come argomento di `malloc`).
- `ssize_t`: variante con segno, indispensabile per funzioni come `read` che devono restituire sia un conteggio di byte positivo sia un codice di errore `-1`.

Standard POSIX: L'implementazione di riferimento per i sistemi Unix moderni è **POSIX.1-2008** (equivalente alla *Single UNIX Specification v4*). Estende la libreria C aggiungendo: controllo dei processi, gestione avanzata dei file, memoria condivisa, thread tramite `<pthread.h>`, semafori tramite `<semaphore.h>`.

3.7 Limiti di Sistema: Caratteristiche Generali

Non tutti i parametri operativi possono essere codificati staticamente a compile-time, poiché variano profondamente tra hardware e configurazioni software diverse. Per gestire questa variabilità, POSIX fornisce funzioni per interrogare i limiti direttamente a runtime, ottenendo i valori effettivi della macchina su cui il programma è in esecuzione.

- `sysconf()`: recupera limiti generali del sistema (es. numero di bit in un `long`). Il nome del limite è passato con il prefisso `_SC_` (es. `_SC_LONG_BIT`).
- `pathconf()`: recupera limiti dipendenti dal file system specifico in cui ci si trova (es. lunghezza massima del nome di un file). Il nome del limite è passato con il prefisso `_PC_` (es. `_PC_NAME_MAX`).

3.8 Limiti Numerici Principali (POSIX)

- `LONG_BIT` (`_SC_LONG_BIT`): Indica da quanti bit è costituita una variabile di tipo `long`.

- **SSIZE_MAX** (**_SC_SSIZE_MAX**): Rappresenta il valore massimo che può assumere una variabile di tipo `size_t`, adeguata per passare o ricevere informazioni sulle dimensioni di un oggetto in memoria.
- **WORD_BIT** (**_SC_WORD_BIT**): Dipende dall'architettura e indica quanti bit costituiscono una parola ("word"), ovvero la dimensione corretta per una variabile intera standard.

3.9 Limiti Operativi e File System

- **ARG_MAX** (**_SC_ARG_MAX**): Specifica la lunghezza massima degli argomenti passati alle funzioni `exec` (quanto può essere grande la stringa sulla linea di comando).
- **CHILD_MAX** (**_SC_CHILD_MAX**): Indica il numero massimo di processi figlio che possono essere creati per ID utente reale (tramite la `fork`). È garantito un valore $\geq$ al limite minimo POSIX.
- **PATH_MAX** (**_PC_PATH_MAX**): Stabilisce il numero massimo di byte in un percorso relativo, includendo il carattere terminatore NULL.
- **NAME_MAX** (**_PC_NAME_MAX**): Definisce il numero massimo di byte nel nome di un singolo file (escludendo il NULL finale).
- **OPEN_MAX** (**_SC_OPEN_MAX**): Determina il numero massimo di file che il processo può tenere aperti per processo.
- **PIPE_BUF** (**_PC_PIPE_BUF**): Indica la dimensione massima (in byte) che può essere scritta in modo *atomico* su una pipe. Se si supera questo limite, i dati potrebbero essere trasferiti in più passi separati.
- **ATEXIT_MAX** (**_SC_ATEXIT_MAX**): Numero massimo di funzioni di *clean up* installabili (con la funzione `atexit`) affinché vengano chiamate al termine regolare del processo. Garantito ≥ 32 .
- **PAGESIZE** (≥ 1): È l'unità base per la gestione della memoria. Espresso in byte, indica la dimensione minima di memoria virtuale richiedibile al sistema e definisce la grandezza fissa dei blocchi in cui vengono suddivisi i processi (le *pagine*) e la memoria fisica (i *frame*) per consentirne il corretto mappaggio.

3.10 Garanzie POSIX e Consultazione dei Manuali

Lo standard garantisce valori minimi per i limiti. Ad esempio, `ATEXIT_MAX` è sempre ≥ 32 ; `OPEN_MAX` è sempre ≥ 20 . Quando si vuole sapere il valore effettivo sul sistema corrente, si interroga `sysconf()` o `pathconf()`.

Pagine di Manuale Unix (Man Pages): La sintassi da terminale è: `man [sezione] <nome>`.

Organizzazione delle sezioni su GNU/Linux:

1. **Comandi utente** (User commands)
2. **Chiamate di sistema** (System calls - solitamente racchiuse in chiamate di libreria)

3. **Funzioni di libreria** (Library functions)
4. File di dispositivo e driver
5. Formati di file e convenzioni
6. Giochi e salvaschermi
7. Miscellanea (protocolli, convenzioni, ecc.)
8. Comandi di amministrazione di sistema

Esempi:

- `man strcpy`: Sezione 3 (funzione di libreria). Richiede `#include <string.h>`. Conforme a C89, C99, SVr4.
- `man getuid`: Sezione 2 (system call). Richiede `<unistd.h>` e `<sys/types.h>`. Conforme a POSIX.1-2001 e 4.3BSD. Non fallisce mai.

4 – Unix: Panoramica per l'Utente

4.1 Identità, Accesso e Gestione Utenti/Gruppi

Le informazioni sugli utenti sono memorizzate nei seguenti file di configurazione:

- **/etc/passwd**: Informazioni pubbliche.
Formato: `username:x:UID:GID:descrizione:home_dir:shell`.
Il campo `x` indica che la password effettiva è cifrata in `/etc/shadow`.
- **/etc/shadow**: Password cifrate (es. con MD5 o DES), con `salt` per mitigare attacchi a dizionario. Non leggibile dagli utenti ordinari. Gestisce anche le scadenze delle password (giorni dalla creazione, giorni al cambio obbligatorio, ecc.).
- **/etc/group**: Informazioni sui gruppi.
Formato: `nome_gruppo:x:GID:lista_utenti`.

Comandi utili:

- `id [username]`: mostra UID e GID dell'utente corrente o specificato.
- `newgrp <gruppo>`: cambia il gruppo primario corrente (richiede la password del gruppo se l'utente non ne fa parte).

4.2 Risoluzione dei Nomi e Servizi di Rete

Il sistema operativo utilizza file di testo semplice per mappare nomi a indirizzi o porte, facilitando le comunicazioni di rete.

- **/etc/hosts**: Associa indirizzi IP a nomi di host fisici (es. `127.0.0.1 localhost`).
- **/etc/networks**: Associa indirizzi di rete a nomi logici (es. `loopback 127.0.0.0`).
- **/etc/services**: Elenca i servizi di rete disponibili, mappando il nome del servizio alla porta e al protocollo (es. `ssh 22/tcp`).

Name Service Switch (/etc/nsswitch.conf): Questo file di configurazione determina l'ordine in cui il sistema deve cercare queste informazioni. Ad esempio, la riga `hosts: files dns` indica al sistema di cercare la risoluzione di un hostname prima nei file locali (`/etc/hosts`) e, in caso di fallimento, di interrogare un server DNS.

4.3 Informazioni sui File, Proprietà e Permessi Standard

Il comando `ls -l` produce output strutturato: tipo di file, permessi, numero di link, proprietario, gruppo, dimensione, data di modifica, nome.

Tipo di file (primo carattere):

- -: file regolare
- d: directory
- l: link simbolico
- b: dispositivo a blocchi
- c: dispositivo a caratteri
- p: named pipe
- s: socket

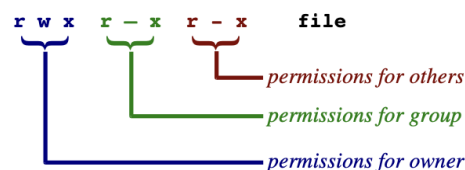
Permessi (caratteri 2-10): Tre gruppi da tre caratteri (**rwX**) **r = lettura**, **w = scrittura**, **x = esecuzione** (o attraversamento per le directory) per proprietario, gruppo e altri. Esprimibili in notazione ottale (es. **0755 = rwxr-xr-x**).

Comandi di modifica:

- **chown <utente>:<gruppo> <file>**: cambia proprietario e gruppo (solo root può cambiare il proprietario).
- **chmod <permessi> <file>**: modifica i permessi (solo il proprietario o root).

Flag speciali:

- **SUID (Set-User-ID)**: il processo acquisisce i privilegi del proprietario del file durante l'esecuzione.
- **SGID (Set-Group-ID)**: il processo acquisisce il gruppo del file.
- **Sticky bit** (su directory): solo il creatore del file o root può cancellarlo dalla directory. Utile ad esempio nella dir con i file temporanei che non possono essere eliminati da altri processi (utenti) se non il proprietario (o root).



4.4 Permessi Specifici: Access Control List (ACL)

Le ACL consentono politiche di accesso più granulari rispetto al modello owner/group/others. Un file con ACL attiva mostra un **+** alla fine della stringa dei permessi in `ls -l` (es. `crw-rw----+`).

Comandi: `getfacl <file>` (visualizza), `setfacl` (modifica).

Esempio: L'utente `user123` può ricevere accesso esplicito `rw-` a `/dev/kvm` tramite `user:user123:rw-`, senza essere proprietario né membro del gruppo `kvm`.

4.5 Gestione dello Spazio: Sparse File

Se un file contiene ampie porzioni vuote (logicamente pari a zero), i file system moderni non allocano spazio fisico per tali aree. L'i-node traccia solo i blocchi con dati reali "saltando" logicamente lo spazio vuoto.

Esempio Pratico in C: È possibile creare uno sparse file spostando in avanti il puntatore del file senza scriverci nulla in mezzo.

1. **Apertura (`open`):** Si apre `sparsefile` con flag `O_RDWR | O_CREAT | O_EXCL` (creazione esclusiva in lettura/scrittura) e i permessi espliciti `S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH` (`rw-r--r--`).
2. **Spostamento (`lseek`):** `lseek(fd, 1000000, SEEK_SET)` sposta il puntatore a 1.000.000 di byte dall'inizio. In questo intervallo non viene allocato spazio.
3. **Scrittura (`write`):** `write(fd, &c, 1)` scrive un singolo byte ('A') alla fine di questo spazio vuoto.

Verifica sul disco: Eseguendo da terminale `ls -ls sparsefile`, noteremo una discrepanza voluta:

- La **dimensione logica** (size) riporterà 1.000.001 byte.
- I **blocchi fisici effettivamente usati** (indicati dal primo numero, ad esempio 8) saranno pochissimi, dimostrando che tutto lo spazio composto da zeri non occupa vera memoria sul disco.

4.6 Redirezione dell'I/O

La shell ci permette di alterare il flusso naturale di questi canali (che di default puntano al terminale) utilizzando appositi operatori:

- **Redirezione dell'Input (`<`):** Permette di far leggere a un programma i dati da un file anziché dalla tastiera (es. `command1 < input_file.txt`).
- **Redirezione dell'Output (`>` e `>>`):** Redirige lo standard output verso un file. Il simbolo `>` sovrascrive il file se esiste, mentre `>>` appende i dati alla fine del file senza cancellarne il contenuto.
- **Redirezione dello Standard Error (`2>`):** Poiché lo stderr corrisponde al descrittore 2, possiamo usare la sintassi `2> file.txt` per separare gli errori dall'output normale.
- **Redirezione combinata:** Per ridirigere sia stdout che stderr nello stesso file, possiamo usare `&>`. [A seconda della shell utilizzata (come bash o csh), la sintassi per queste operazioni avanzate o per l'append dello stderr può variare. Per verificare quale shell si sta utilizzando si può stampare la variabile d'ambiente con `echo $SHELL`.]
[Nella shell bash si ha `2>>` o `&>>` per l'append dello standard error, mentre `>>&` per csh.
Per la ridirezione normale si ha `>` e `&>` unicamente per bash e `<&` per bash e csh]

- **Pipeline (|):** Collega lo stdout di un processo direttamente allo stdin del processo successivo (es. `command1 | command2`). [Se si desidera passare sia l'output che l'errore al processo successivo, si usa `&.`]

All'avvio della macchina, il firmware/ROM carica il kernel in memoria. Dopo aver inizializzato l'hardware, il kernel lancia il primo processo a livello utente: `/sbin/init`. Nel meccanismo tradizionale noto come **System V**, questo programma configura il sistema basandosi su file di testo e sull'esecuzione di script sequenziali. L'uso di file di testo è una scelta progettuale precisa: risultano facilmente accessibili e modificabili dall'amministratore in caso di emergenza, specialmente quando non si ha a disposizione un editor per file binari.

Il cuore pulsante di questo modello è il file `/etc/inittab`, che definisce il comportamento del sistema in base a specifici **Runlevel**. Il concetto di Runlevel rappresenta lo "stato" in cui si trova il sistema. Ad ogni livello sono associati specifici servizi:

- **Runlevel 0:** Spegnimento del sistema (Halt).
- **Runlevel 1 (o S):** Modalità singolo utente (Single-User). Dedicata all'amministratore (root) per interventi di manutenzione a basso livello, solitamente senza rete o altri utenti attivi.
- **Runlevel 2-5:** Modalità operative normali (Multi-User). A seconda del runlevel, includono o meno il supporto alla rete e all'interfaccia grafica.
- **Runlevel 6:** Riavvio del sistema (Reboot).

NB: Durante il normale utilizzo, l'amministratore root può cambiare il runlevel "a caldo", senza riavviare, utilizzando il comando `telinit <numero>`.

In base al runlevel scelto (es. il 2), `init` esegue rigorosamente in ordine alfanumerico una serie di script di avvio. La gestione di questi script ha subito un'evoluzione:

- **Approccio iniziale:** Gli script venivano fisicamente inseriti in specifiche directory di runlevel (es. `/etc/rc2.d/`). Se più runlevel avevano bisogno dello stesso servizio, l'amministratore doveva creare copie fisiche del file per ogni livello, sprecando spazio e complicando la manutenzione.
- **Approccio a Symlink:** Successivamente, si è passati a un'implementazione più intelligente. Gli script effettivi risiedono in una directory centrale di sistema (come `/etc/init.d/`). Nelle directory dedicate ai singoli runlevel (come `/etc/rc2.d/`) vengono invece inseriti solo dei **link simbolici** (collegamenti) che puntano agli script originali.

I link simbolici all'interno delle cartelle dei runlevel seguono una nomenclatura molto rigida:

- Iniziano con **S** (Start) per avviare un servizio quando si entra nel runlevel.
- Iniziano con **K** (Kill) per interrompere un servizio quando si esce dal runlevel.
- La lettera è seguita da un **numero** (es. `S10sshd` o `K20apache`), che indica l'ordine sequenziale di esecuzione.

4.7 Systemd: L'Inizializzazione Moderna

Per risolvere la rigidità sequenziale e la gestione manuale delle dipendenze di System V, che costringeva l'amministratore a calcolare manualmente l'ordine degli script, si è adottato **Systemd**. Le caratteristiche principali di questa architettura includono:

- **Il ruolo di init:** In questo ecosistema, il classico comando `/sbin/init` diventa semplicemente un link simbolico al binario di Systemd.
- **Dai Runlevel ai Target:** Vengono abbandonati i rigidi runlevel numerici in favore di stati logici chiamati **Target**(es. `multi-user.target`).
- **Configurazione e Dipendenze:** Il file principale risiede in `/etc/systemd/system.conf`. Invece di far partire gli script in un cieco ordine alfabetico, le dipendenze sono scritte esplicitamente nei file di configurazione (es. "questo servizio dipende dalla rete").
- **Esecuzione in Parallelo:** Leggendo le configurazioni, Systemd costruisce un grafo delle dipendenze e avvia i servizi in **parallelo** il più possibile. Questo approccio riduce drasticamente i tempi di avvio (boot) della macchina.
- **Gestione Centralizzata:** L'amministratore accende, spegne o interroga i servizi in modo unificato tramite il comando `systemctl` (es. `systemctl start <nome_servizio>` o `systemctl stop <nome_servizio>`).

4.8 La Gerarchia del File System Unix

Il file system Unix è organizzato in una struttura ad albero con un'unica radice (`/`), governato dal Filesystem Hierarchy Standard (FHS). Ecco le directory principali:

- **/bin e /sbin:** Contengono i programmi eseguibili binari. `/bin` contiene i comandi di base a disposizione di tutti gli utenti (es. la shell o `ls`), mentre `/sbin` (System Binaries) contiene strumenti riservati all'amministratore root (es. programmi di partizionamento o configurazione di rete).
- **/usr:** Contiene i programmi e le librerie non indispensabili per il boot minimo del sistema. [Storicamente, per risparmiare spazio, solo `/bin` e `/sbin` risiedevano sul disco locale per avviare la rete, dopodiché l'intera directory `/usr` veniva montata da un server remoto. Oggi, con dischi capienti, questa separazione è meno rilevante e spesso `/bin` e `/sbin` sono solo link simbolici a `/usr/bin` e `/usr/sbin`.]
- **/etc:** È il cuore della configurazione del sistema. Contiene quasi esclusivamente file di testo (es. configurazione di rete, password, SSH), facilmente modificabili con un editor.
- **/home:** Contiene le sottodirectory personali di ciascun utente (es. `/home/user1`). [La home directory del root `/root` è solitamente mantenuta separata nel file system locale per garantire l'accesso amministrativo anche se la directory `/home` di rete fallisce.]
- **/tmp e /var/tmp:** Dedicate ai file temporanei creati dai programmi. Utilizzano un permesso speciale (lo *sticky bit*) che permette a tutti di scriverci, ma impedisce a un utente di cancellare i file temporanei appartenenti a un altro utente.
- **/var:** Contiene dati "variabili", che cambiano costantemente durante il funzionamento del sistema. Sotto-directory importanti includono:
 - `/var/log:` I file di logging che registrano eventi ed errori del sistema.

- `/var/lib`: Database interni generati dalle applicazioni installate.
- `/var/cache`: Dati che possono essere cancellati ma che richiedono tempo per essere ricostruiti.
- `/var/spool`: Gestisce le code per periferiche non condivisibili simultaneamente. Ad esempio, quando più utenti stampano, i file vengono messi in `/var/spool` e il sistema operativo li invia sequenzialmente alla stampante, evitando che un utente debba attendere attivamente la fine della stampa precedente.

4.9 I Dispositivi (Device Files in `/dev`)

In Unix, nel suo paradigma “tutto è un file”, anche i dispositivi hardware sono mappati nel file system sotto la directory “`/dev`” e vengono manipolati tramite gli standard system call (`open`, `read`, `write`, `close`). Funzioni hardware specifiche (es. baud rate seriale) usano invece la funzione `ioctl`. Ovviamente non tutte le syscall di base possono essere usate per tutti i dispositivi come ad esempio la `seek` non si può usare per una porta seriale.

I dispositivi si dividono in due grandi categorie, gestite dal kernel tramite due numeri identificativi:

1. **Dispositivi a Blocchi (b)**: Periferiche di archiviazione come gli hard disk (es. dati letti a blocchi).
2. **Dispositivi a Caratteri (c)**: Periferiche a flusso continuo, come terminali o porte seriali.

All'interno del kernel, un dispositivo è identificato univocamente da:

- **Major Number**: Identifica la classe del dispositivo (es. 8 = hard disk SCSI, 4 = terminale).
- **Minor Number**: Identifica l'istanza specifica di quel dispositivo (es. il disco intero o una singola partizione). Ad esempio, `/dev/sda1` rappresenta la prima partizione (`minor 1`) del primo disco SCSI (`major 8`). L'amministratore può creare manualmente questi "file speciali" usando il comando `mknod`.

Esistono inoltre periferiche puramente "virtuali" fornite dal kernel:

- `/dev/null`: Il "buco nero" del sistema; tutto ciò che vi viene scritto viene scartato. Utile se il risultato delle operazioni non ci interessa e al posto di sprecare spazio per stamparlo su un file temporaneo o visualizzarlo su shell si eliminano direttamente dirottando qui l'output
- `/dev/zero`: Genera infiniti byte nulli (zeri) se letto. Utile per l'inizializzazione a zero di alcune strutture
- `/dev/random`: Genera numeri casuali.

4.10 Panoramica dei Comandi Principali

Identità e Privilegi:

- **su**: acquisisce i privilegi di un altro utente (default: root). Ha il bit SUID attivo permettendo al processo di girare con i privilegi dell'owner (root) per effettuare la transizione di identità.
- **passwd**: cambia la password utente. Richiede SUID per modificare `/etc/shadow`.

Navigazione e Gestione File:

- **ls**, **cd**, **mkdir**: listare, navigare, creare directory.
- **cp**, **mv**, **rm**: copiare, spostare (rinominare se il file di destinazione è nella stessa dir), cancellare. **rm -r** esegue una cancellazione ricorsiva.
- **find**: ricerca file per nome, tipo o permessi (es. **find . -type f -perm /a+x** trova i file eseguibili).
- **ln**: crea hard link (default) o, con **-s**, link simbolici.
- **chmod**, **chown**, **chgrp**: modificano permessi, proprietario e gruppo.

Elaborazione Testi:

- **cat**: stampa il contenuto di un file su **stdout**.
- **more**, **less**: paginatori che permettono di leggere testi lunghi una pagina alla volta; **less** consente anche la navigazione all'indietro.
- **grep**: filtra il testo in input stampando solo le righe che corrispondono a un determinato pattern. Non si limita a cercare parole esatte, ma permette di estrarre righe da un file (o standard input) usando espressioni regolari.
- **sed**: editor di flusso non interattivo; applica trasformazioni/sostituzioni riga per riga.
- **awk**: linguaggio di scripting per l'elaborazione di dati strutturati per campi.

Storage e Archiviazione:

- **dd**: copia a basso livello (*Disk Dump*), opera direttamente sui device file. Usato spesso per copiare il contenuto raw di interi dischi verso file o viceversa.
- **tar**: archivia file/directory in un singolo file, solitamente combinato con **gzip**, **bzip2** o **xz** per la compressione.
- **zip/unzip**: archiviazione e compressione in formato ZIP.

4.11 Gestione dello Storage e dei File System

Comandi Informativi:

- **du**: calcola lo spazio occupato da una directory.
*[L'opzione **-m** restituisce il valore in Megabyte, mentre **-h** (human-readable) formatta l'output con unità dinamiche (K, M, G). Utile per la lettura umana ma meno adatta al parsing automatizzato da script.]*

- **df**: mostra lo spazio libero e occupato sull'intero file system o su periferiche specifiche.

Inizializzazione e Montaggio: *[Questi comandi richiedono permessi di root.]*

- **fdisk**: manipola le tabelle delle partizioni di un disco fisico (creare, rimuovere o ridimensionare le suddivisioni logiche di un disco fisico).
- **mkfs**: inizializza una partizione con la struttura necessaria per archiviare file.
*[È un comando wrapper: per creare un file system **ext2** richiamerà internamente **mke2fs**.]*
- **mount / umount**: innesta o distacca un file system da un punto dell'albero principale.
- **NFS (Network File System)**: permette di montare directory esportate da server remoti.
*[La sintassi tipica è **mount server:/directory_esportata /mnt**. Il kernel del client incapsula trasparentemente ogni richiesta di I/O in pacchetti di rete diretti al server (porta 2049), restituendo i dati come se il file fosse fisicamente locale.]*

4.12 Controllo dei Processi e Informazioni di Sistema

- **uname -a**: Stampa informazioni sul sistema ospite. Con l'opzione **-a** (all) fornisce un quadro completo: tipo di sistema operativo (es. Linux), nome dell'host, versione esatta del kernel, data di compilazione e architettura hardware (es. x86_64).
- **ps**: mostra i processi in esecuzione.
*[Per una visione d'insieme si utilizzano **ps aux** (stile BSD) o **ps -ef** (stile System V), che mostrano tutti i processi con PID, PPID, utente, CPU consumata e comando.]*
- **kill**: invia un segnale a un processo tramite PID. Default: **SIGTERM** (15), terminazione cortese intercettabile. Per cortese si intende una terminazione che dà modo al processo di chiudere tutti i file descriptor e le risorse usate prima di terminare. Con **kill -9** si invia **SIGKILL**, non intercettabile né ignorabile, serve a terminare istantaneamente il processo.
- **nice**: altera la priorità di scheduling. Un utente non privilegiato può solo abbassare la propria priorità.
Niceness: Valori maggiori → "maggiore gentilezza" → priorità più bassa. Intervallo tipico: da -20 (massima priorità) a +19 (minima). Solo root può impostare valori negativi. La system call **nice()** riceve un delta da sommare al valore attuale.
- **time <comando>**: misura il tempo di esecuzione, distinguendo tempo reale (*real*), tempo utente (*user*) e tempo kernel (*sys*).
Funzione times(): Restituisce statistiche cumulative (in *clock tick*) sul tempo CPU in modalità utente e kernel, includendo anche i tempi dei processi figli. La macro **_SC_CLK_TCK** fornisce il fattore di conversione in secondi.

5 – Processi e Thread

5.1 Definizione e Natura del Processo

Un **processo** è formalmente definito come *un programma in esecuzione*. Da un punto di vista architetturale più rigoroso, rappresenta l'entità astratta a cui il sistema operativo assegna risorse fisiche (in primis la CPU) affinché ne esegua le istruzioni. Questa definizione mette in luce la dicotomia tra la natura *statica* del programma (file su disco) e la natura *dinamica* del processo, che progredisce nel tempo.

Un processo è strutturalmente composto da:

- **Codice:** La sequenza di istruzioni macchina derivate dall'eseguibile.
- **Dati:** Le strutture allocate staticamente e dinamicamente durante l'esecuzione.
- **Contesto:** Informazioni di gestione (PID, stato, valore dei registri CPU al momento della sospensione).

5.2 Modelli di Concorrenza e Multiprogrammazione

Tipicamente il numero di processi attivi supera il numero di processori fisici disponibili. Poiché le istruzioni di I/O sono molto più lente rispetto ai tempi della CPU, eseguire un solo processo alla volta lascerebbe il processore inattivo per lunghi intervalli. La soluzione è eseguire più programmi in modo concorrente, avanzando nel programma successivo nei momenti in cui quello corrente è bloccato in attesa di I/O.

[Superando i sistemi storici a singolo task o i sistemi "batch" dei primi elaboratori mainframe, in cui i lavori venivano eseguiti in modo strettamente sequenziale.]

Modelli di multitasking:

- **Multitasking Cooperativo (senza *preemption*):** Il processo detiene la CPU finché non la cede volontariamente o non esegue un'operazione bloccante. Un processo CPU-bound può monopolizzare le risorse, degradando l'interattività del sistema.
- **Multitasking Preemptivo (con *preemption*):** Il sistema operativo impone un limite temporale massimo (*time slice* o *quanto di tempo*) all'esecuzione continua. Allo scadere di un timer hardware, lo scheduler interrompe forzatamente il processo (*preemption*). I processi interattivi ricevono alta priorità ma quanti brevi (si bloccheranno quasi subito per I/O); i processi CPU-bound ricevono priorità inferiore ma quanti più lunghi.
- **Sistemi Time-Sharing:** Estendono il multitasking preemptivo alla multiutenza simultanea, fornendo a più utenti l'illusione dell'accesso esclusivo e interattivo alle risorse.

5.3 Supporto Hardware, Protezione e Memoria Virtuale

La capacità di gestire in modo sicuro la multiprogrammazione dipende dalle caratteristiche architetturali del processore:

- **Sistemi privi di protezione hardware:** Il software viene eseguito in un unico spazio di indirizzamento e il sistema operativo funge da semplice libreria.
- **Sistemi con protezione ma senza Memoria Virtuale:** I processori implementano livelli di privilegio separati (User mode e Kernel mode), proteggendo l'hardware da bug o accessi impropri [come nei microcontrollori avanzati ARM impiegati nell'automotive, supportati da RTOS o versioni embedded di Linux/Windows]. Tuttavia, mancando un'unità di traduzione, gli indirizzi generati dai processi corrispondono fisicamente agli indirizzi in RAM.
- **Sistemi con protezione e Memoria Virtuale:** Il microprocessore possiede un meccanismo hardware di traduzione dinamica degli indirizzi. Il sistema operativo mappa gli indirizzi logici/virtuali del processo in indirizzi fisici. Questo disaccoppiamento garantisce un isolamento totale tra i processi.

[Sistemi privi di protezione hardware sono tipici dei microcontrollori di base per l'automazione o delle prime versioni di Windows per PC. Sistemi con protezione ma senza memoria virtuale sono diffusi nei microcontrollori avanzati ARM impiegati nell'automotive, supportati da RTOS o versioni embedded di Linux/Windows.]

5.4 Il Modello degli Stati del Processo

Durante il suo ciclo di vita, un processo transita attraverso una serie di stati gestiti dallo scheduler. Il modello base prevede:

1. **Running (In Esecuzione):** Il processo detiene il processore e le sue istruzioni vengono eseguite.
2. **Blocked (Bloccato / In Attesa):** Il processo non può proseguire poiché è in attesa di un evento esterno (es. completamento di I/O, dati da un socket o sincronizzazione).
3. **Ready (Pronto):** Il processo ha tutte le risorse necessarie per l'esecuzione ed è in attesa che lo scheduler gli assegni il processore. (Transizione Running → Ready via preemption; transizione Blocked → Ready al verificarsi dell'evento atteso).

A questi si aggiungono stati accessori per la gestione avanzata:

- **New (Nuovo):** Il processo è in fase di creazione e non ha ancora ricevuto tutte le risorse necessarie (es. memoria).
- **Finish / Terminated (Terminato):** Il processo ha concluso l'esecuzione, ma il sistema ne mantiene le informazioni di uscita per fornirle al genitore.
- **Suspended (Sospeso):** In condizioni di esaurimento risorse, il sistema archivia temporaneamente il processo su una periferica di storage. Si divide in *Suspended-Ready* (pronto all'esecuzione non appena riportato in memoria) e

Suspended-Blocked (sospeso e contemporaneamente in attesa di un evento). Questo secondo passa a *Suspended-Ready* nel momento in cui riceve l'evento che sta attendendo.

Transizioni principali:

- Running → Ready: preemption (scadenza time slice).
- Running → Blocked: operazione bloccante (I/O, wait).
- Blocked → Ready: evento atteso si verifica.
- Running/Ready → Suspended: swapping per esaurimento RAM.

5.5 Ciclo di Vita: Creazione e Gerarchia

I processi vengono generati da: avvio del sistema (processo `init`), richieste dell'utente tramite shell, oppure da processi server che generano processi *worker* dedicati alla gestione di singole richieste. *[Definiti **demoni** in Unix o **servizi** in Windows.]*

System Call `fork()`: Genera un processo figlio come copia quasi identica dello spazio di indirizzamento del padre. I processi costituiscono un albero generico rigoroso che deve essere mantenuto consistente anche alla terminazione di uno o più processi.

- Il **padre** riceve come valore di ritorno il **PID del figlio**.
- Il **figlio** riceve il valore **0**.

[In caso di fallimento, `fork()` restituisce un valore negativo al padre e nessun processo viene creato.]

Ogni processo conosce il proprio PID tramite `getpid()` e il PID del padre tramite `getppid()`.

[È importante memorizzare il PID del figlio alla creazione poiché non esistono altre modalità per accedervi successivamente; tale valore è fondamentale per comunicare con il figlio tramite segnali.]

5.6 Ciclo di Vita: Terminazione, Orfani e Zombie

Cause di terminazione:

- **Volontaria:** `exit()` che permette una deallocazione pulita delle risorse e il ritorno di un codice di stato per comunicare al padre delle informazioni (valore da 0 a 255) o `return` dal `main`.
- **Errore fatale:** il kernel abbatte il processo a causa di violazioni (accesso illegale a memoria, istruzione non valida).
- **Terminazione esterna:** `kill()` invia un segnale a un altro processo.

*[Il segnale **SIGKILL** (numero 9) forza la terminazione immediata senza possibilità di intercettazione da parte del processo bersaglio. La system call **kill()** serve per inviare un qualsiasi segnale, non esclusivamente per terminare un processo.]*

Processi Orfani: Un processo il cui padre termina prima di lui diventa *orfano*. In Unix, i processi orfani vengono automaticamente adottati dal processo con PID 1 (**init**), che è programmato per attenderne la futura terminazione. Per adozione si intende che il PPID del processo orfano venga impostato a 1.

Processi Zombie: Quando un processo termina, il kernel dealloca memoria e risorse ma conserva il codice di uscita finché il padre non lo raccoglie. In questo intervallo il processo si trova nello stato di **Zombie**. Il PID e la struttura dati dello zombie vengono liberati definitivamente solo dopo che il padre ha interrogato il sistema per raccogliere il valore di ritorno del figlio.

5.7 Il Process Control Block (PCB) e il Context Switch

Per realizzare l'illusione dell'esecuzione simultanea, il sistema operativo deve poter sospendere un processo, salvarne lo stato e riattivarlo senza che quest'ultimo si accorga dell'interruzione. Tutte le informazioni necessarie risiedono nel **Process Control Block (PCB)**:

- **Identificatori:** PID, PPID, UID proprietario (per i permessi), percorso dell'eseguibile.
- **Contesto del Processore:** Fotografia esatta dei registri hardware (Program Counter, Stack Pointer, registri dati, flag di stato).
- **Informazioni di Scheduling:** Stato corrente, priorità, evento atteso se in stato di block.
- **Gestione Risorse e Memoria:** Strutture della memoria allocata, file descriptor aperti, directory di lavoro corrente.
- **Gestione Segnali:** Segnali ricevuti mentre il processo era sospeso, da consegnare alla riattivazione.

5.8 Scheduling e Overhead del Context Switch

Scheduling Non-Preemptive: Il processo mantiene la CPU fino a terminazione o blocco. Un ciclo infinito può causare il blocco dell'intero sistema.

Scheduling Preemptive: Il processo può essere sospeso anche allo scadere del time slice, alla ricezione di un interrupt hardware o in seguito a chiamate bloccanti come I/O o primitive di sincronizzazione; transisce nello stato *Ready*.

Il **context switch** introduce un costo computazionale inevitabile (*overhead*). Oltre al salvataggio e ripristino dei registri, il costo principale deriva dall'**invalidazione della cache**: quando un nuovo processo subentra, i dati presenti in cache non sono più validi per il nuovo

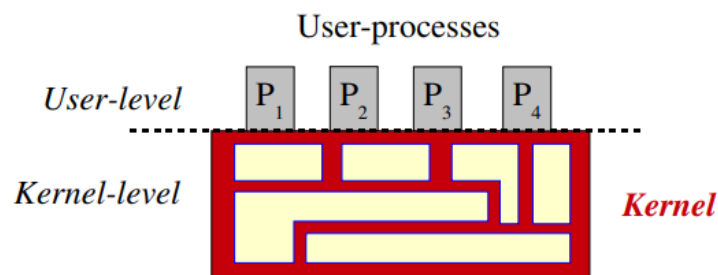
contesto, causando numerosi *cache miss* con accessi alla memoria principale (latenze dell'ordine delle centinaia di cicli di clock rispetto a 1–2 cicli di un *cache hit*).

5.9 Architetture del Kernel

5.9.1 Kernel Monolitico

L'intero kernel (driver, file system, protocolli di rete) opera in un unico spazio di indirizzamento condiviso. Ha come vantaggio che la comunicazione tra sottosistemi avviene tramite semplici chiamate di funzione, minimizzando l'overhead. Un singolo bug, invece, può compromettere l'intero sistema.

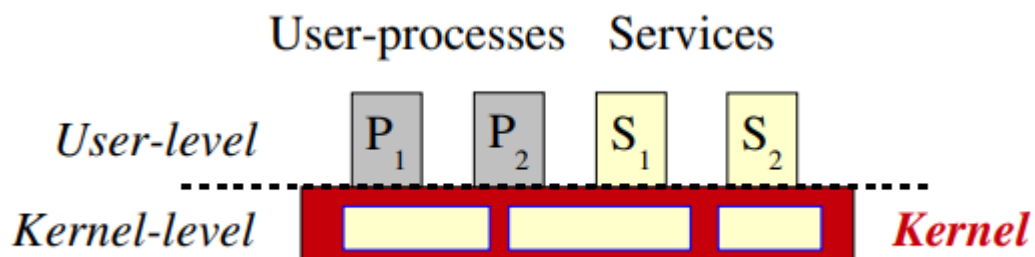
[I sistemi operativi Unix e Linux storici adottano tipicamente questo paradigma.]



5.9.2 Microkernel

Il nucleo è ridotto ai minimi termini (allocazione memoria, scheduling, comunicazione tra processi). I servizi complessi (stack di rete, driver) girano come processi in user space. Alta modularità e robustezza: un servizio che fallisce può essere riavviato senza *kernel panic*. Svantaggio: ogni richiesta a un servizio si traduce nell'invio di un messaggio tramite IPC (Inter-Process Communication), generando continui e costosi context switch.

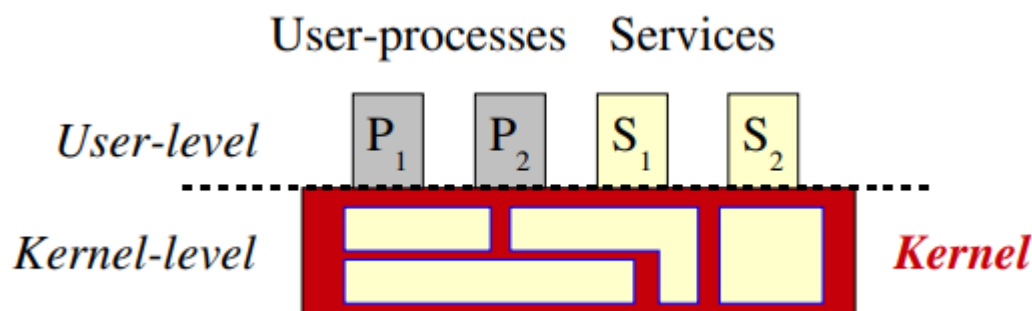
[Esempi: QNX, alcune prime versioni del kernel di macOS.]



5.9.3 Kernel Ibrido

Fonde i due approcci: il nucleo monolitico gestisce i servizi performance-critical e i servizi fondamentali, mentre funzionalità aggiuntive sono delegate a processi in user space.

[Approccio tipico di Windows NT.]



5.10 Gestione dei Thread e Concorrenza

Per favorire l'esecuzione parallela all'interno di una singola applicazione, è possibile suddividere un processo in molteplici flussi di esecuzione, denominati *thread*.

I thread appartenenti allo stesso processo condividono lo spazio di indirizzamento (variabili globali, heap), pur disponendo di stack separati per le variabili locali. Richiedono sincronizzazione per l'accesso concorrente alle risorse condivise.

*[È possibile istanziare variabili globali esclusive per un singolo thread tramite il **Thread Local Storage (TLS)**, utile in pattern di programmazione specifici.]*

Ogni thread ha un **Thread Control Block (TCB)**, analogo al PCB, che raccoglie le informazioni necessarie alla sua sospensione e riattivazione.

5.10.1 Modelli di Mappatura dei Thread

La gestione dei thread può avvenire su due livelli contrapposti (o ibridi):

1. **Gestione a Livello Kernel (1:1):** Ogni thread utente corrisponde a un reale flusso di esecuzione gestito dallo scheduler del kernel. Il kernel può allocare thread differenti dello stesso processo su processori fisici distinti, ottenendo vero parallelismo hardware. Tuttavia, le operazioni di creazione e *switch* di un thread comportano un overhead paragonabile a quello del context switch dei processi (richiedendo *system call*).
2. **Gestione a Livello Utente (N:1):** I molteplici thread sono implementati in *user space* (similmente al concetto di *co-routine*), venendo mappati su un unico flusso di esecuzione gestito dal kernel. Il vantaggio è una drastica riduzione dell'overhead (lo switch equivale a salti fra funzioni), ma il blocco di un singolo thread (es. attesa I/O) causa il congelamento dell'intero processo, vanificando la reale concorrenza su architetture multiprocessore.



Viene implementato il parallelismo con continui salti di funzione tra un thread e l'altro continuando da dove si era rimasti alla chiamata precedente.

*[In ambiente Windows, i flussi leggeri a livello utente prendono il nome di **Fiber**.]*

5.10.2 Framework e Modelli Concorrenti

In POSIX, i thread si creano tramite `pthread_create()`, cui si passa un identificatore opaco (`pthread_t`), una funzione punto di partenza e un argomento di tipo `void*`.

Modelli operativi principali:

- **Fork-Join:** Il thread principale genera dinamicamente un insieme di thread per parallelizzare un problema; al termine si attende il loro completamento (*join*). Alta dinamicità, ma overhead dovuto alla continua creazione/distruzione. Ci si può imbattere nel problema che il lavoro che necessita la creazione dei thread è superiore alla disponibilità delle risorse disponibili.
[Paradigma alla base dello standard OpenMP.]
- **Thread Pool:** Un numero fisso di thread viene generato all'avvio e mantiene uno stato dormiente fino all'accodamento di task. Elimina l'overhead di creazione runtime e previene l'esaurimento delle risorse. Il limite è lo spreco di risorse (memoria associata a TCB e stack) nei periodi di scarsa latenza lavorativa.
[Modello implementato in Grand Central Dispatch (macOS) e nei worker thread interni del kernel Linux.]

5.10.3 Cancellazione e Pulizia

Un thread può terminare fisiologicamente (ritorno dalla funzione principale) o su richiesta esplicita di terzi. La cancellazione può essere:

- **Asincrona (immediata):** Potenzialmente devastante per la consistenza dei dati.
- **Posticipata:** Il thread termina solo ai *punti di cancellazione* (primitive bloccanti, o esplicitamente tramite `pthread_testcancel()`).
[È fondamentale che nei loop continui di elaborazione si effettui una chiamata a `pthread_testcancel()` per verificare se un terzo ha richiesto la terminazione.]

È possibile disabilitare temporaneamente la cancellazione durante operazioni critiche per preservare la consistenza dei dati; al completamento dell'operazione, la cancellazione va riabilitata.

Prima della distruzione, il thread esegue i *cleanup handler* pre-registrati e i distruttori TLS.

5.11 Segnali

I segnali operano come interrupt software a livello utente. Possono essere:

- **Sincroni:** causati deterministicamente da operazioni in esecuzione (es. `SIGSEGV` per accesso a memoria non valida, `SIGILL` per istruzione illegale).

- **Asincroni:** scaturiti da eventi di I/O o dalla system call `kill()` di un altro processo.

Disposizioni possibili per ciascun segnale:

1. **Ignorare:** Il kernel scarta il segnale senza recapitarlo.
2. **Default:** Viene eseguita l'azione predefinita (terminazione, sospensione, continuazione, ecc.).
3. **Gestire (*Handle*):** Il processo esegue un gestore registrato; al termine, l'esecuzione ordinaria riprende.

Nei processi multithread, la **disposizione** è globale all'intero processo. Le **maschere dei segnali** (che determinano se un segnale deve rimanere pendente) sono invece configurabili per singolo thread.

[A scopo di messaggistica basilare, i processi possono sfruttare i segnali custom `SIGUSR1` e `SIGUSR2`.]

5.12 Comunicazione Inter-Processo (IPC)

5.12.1 Scambio di Messaggi

Ogni comunicazione transita attraverso il kernel (overhead per system call), ma fornisce sincronizzazione implicita ed è scalabile su architetture distribuite.

5.12.2 Memoria Condivisa

Il kernel mappa una regione di memoria fisica negli spazi di indirizzamento virtuale di due o più processi (potenzialmente a indirizzi logici diversi). L'accesso è diretto tramite puntatori, senza system call aggiuntive dopo l'inizializzazione. Richiede sincronizzazione esplicita.

Standard System V:

1. `shmget()`: alloca la memoria condivisa; richiede una chiave intera globale, la dimensione e i flag (`IPC_CREAT`, permessi ottali es. `0600`).
2. `shmat()` (*attach*): associa la regione allo spazio del processo (si passa `NULL` per lasciare al kernel la scelta dell'indirizzo).
3. `shmdt()` (*detach*): separa l'indirizzo logico dalla memoria fisica.
4. `shmctl(..., IPC_RMID, ...)`: rimuove la regione.

Standard POSIX:

1. `shm_open()`: crea o apre il segmento tramite una stringa identificativa (es. `/shm_my_app`), con flag `O_CREAT` e permessi ottali.
2. `ftruncate()`: espande la dimensione del segmento al valore desiderato.
3. `mmap()`: effettua il mapping del file descriptor nel processo (parametri: `PROT_READ` | `PROT_WRITE`, `MAP_SHARED`).
4. `munmap()` e `shm_unlink()`: rimuovono il mapping e il segmento.

5.13 Comunicazione Inter-Processo (IPC) tramite Socket

I socket seguono un modello **client-server**, è necessario l'intervento esplicito del kernel per il transito dei dati (system call)

Lato Server: `socket()` → `bind()` → `listen()` → `accept()` (si sblocca a ogni nuova connessione, generando un socket dedicato per quel client).

Lato Client: `socket()` → `connect()` (specifica l'indirizzo del server). Dopo la connessione, lo scambio dati avviene tramite `read/write` o `send/recv`.

[Essendo un argomento trattato approfonditamente nei corsi sulle reti, è sufficiente ricordare che, a connessione stabilita, la lettura e la scrittura avvengono assimilando il socket a un file descriptor.]

Famiglie di socket:

- **Dominio Unix (AF_UNIX / AF_LOCAL):** Per comunicazione sulla stessa macchina. L'indirizzo è un percorso nel file system (es. `/tmp/mysocket`). Più efficiente in quanto aggira lo stack di rete.
- **Socket IP (AF_INET / AF_INET6):** Per comunicazione in rete. L'indirizzo è dato dalla coppia (indirizzo IP, numero di porta).

6 – Virtualizzazione

6.1 Architetture di Virtualizzazione

La virtualizzazione introduce un livello di astrazione software che isola un ambiente operativo (*Virtual Machine*, VM) dall'hardware fisico sottostante. I vantaggi principali sono: migliore utilizzo delle risorse, isolamento tra istanze, possibilità di congelare e migrare lo stato di una VM.

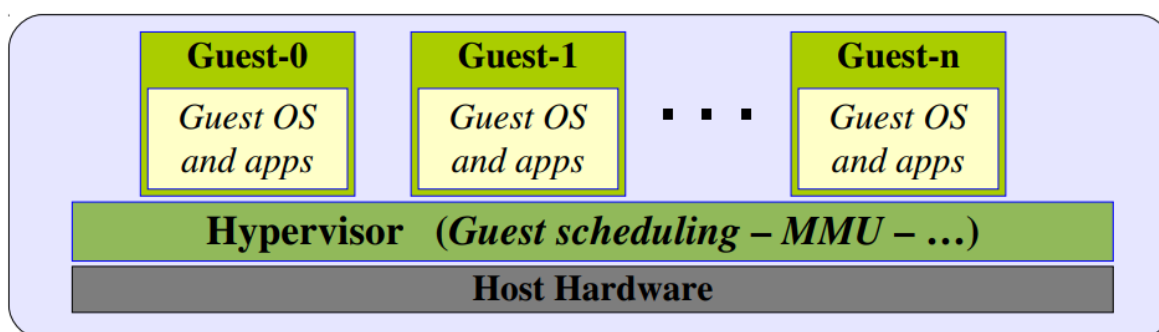
Il componente software che orchestra la virtualizzazione è denominato **Hypervisor** (o *Virtual Machine Monitor*).

6.1.1 Hypervisor di Tipo 1 (Bare-Metal)

L'Hypervisor viene eseguito **direttamente sull'hardware fisico**, al posto di un sistema operativo tradizionale. Il kernel Guest crede di interfacciarsi con periferiche reali; l'Hypervisor intercetta le operazioni privilegiate e le traduce in operazioni fisiche equivalenti (es. scrittura all'interno di un file immagine). Richiede che l'architettura Guest coincida con quella Host.

Il **Guest 0** è definito come amministratore del sistema virtualizzato, con privilegi analoghi a quelli di root in un sistema non virtualizzato.

[Esempi di Hypervisor di Tipo 1: Xen.]

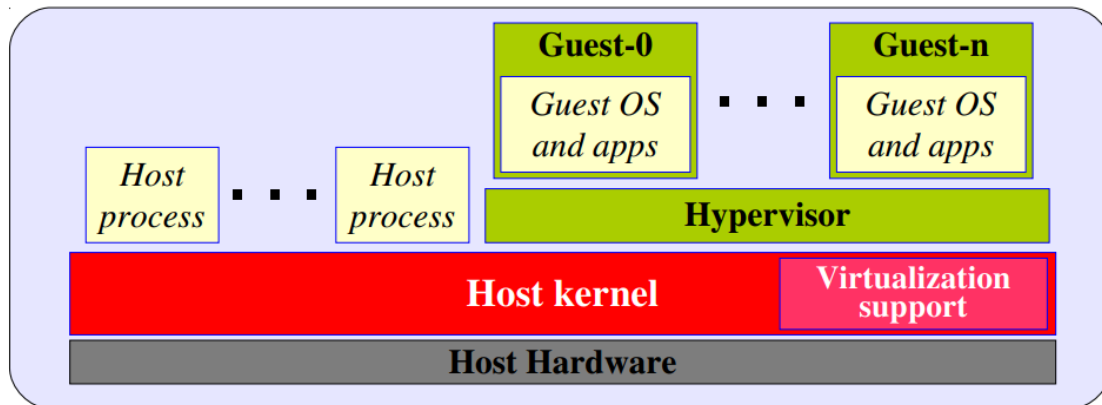


6.1.2 Hypervisor di Tipo 2 (Hosted)

L'Hypervisor opera come processo all'interno di un sistema operativo Host preesistente. Consente anche l'**emulazione**: le istruzioni di un'architettura Guest diversa da quella Host (es. ARM su x86) vengono tradotte dinamicamente in blocchi, con le traduzioni salvate in cache per evitare rielaborazioni.

Il sistema virtualizzato comunica con l'hardware fisico attraverso il kernel Host tramite system call, con un'operazione aggiuntiva rispetto a un sistema classico. È possibile inserire nello stack uno strato di *Virtualization Support* per eseguire certe istruzioni privilegiate più efficientemente, senza ricorrere alle system call ordinarie.

[Software come VirtualBox, VMware o QEMU rientrano in questa categoria, spesso accelerati su Linux tramite il modulo KVM (Kernel Virtual Machine).]



6.1.3 Supporto Hardware alla Virtualizzazione

All'interno di una VM, il kernel Guest è eseguito in modalità utente rispetto al processore fisico. I processori moderni integrano **estensioni hardware** per la virtualizzazione: il tentativo di eseguire istruzioni privilegiate dal kernel Guest genera un **trap**, catturato e gestito dall'Hypervisor.

Per poter implementare la virtualizzazione sono necessari almeno due livelli di privilegio hardware (kernel e user). Il kernel Guest, credendo di operare a massimo privilegio, tenta operazioni privilegiate che generano trap. Se tali trap non sono intercettati o gestiti correttamente dall'Hypervisor, la virtualizzazione non è realizzabile (*requisito di virtualizzabilità* di Popek e Goldberg, 1974).

*[Nei processori privi di queste estensioni si ricorre alla **Paravirtualizzazione**, che richiede la modifica del codice del kernel Guest per sostituire le istruzioni critiche con chiamate esplicite all'Hypervisor.]*

6.2 Ciclo di Vita del Processo: Esecuzione e Terminazione

Un programma in C inizia la sua esecuzione dalla funzione `main`, che riceve `argc` (conteggio degli argomenti) e `argv` (vettore di puntatori alle stringhe degli argomenti). Il `main` è il punto d'inizio *logico*, non quello d'inizio *effettivo*: la libreria standard esegue operazioni preparatorie prima di invocare `main`.

Terminazione pulita volontaria: `exit()` o `return` dal `main`. La `exit()` esegue un ciclo strutturato:

1. Invoca le funzioni registrate tramite `atexit()` in ordine inverso di registrazione.
2. Forza il flush (scrittura) dei buffer di I/O.
3. Elimina i file temporanei creati con `tmpfile()`.
4. Invoca la system call `_exit()`.

[`int atexit(void (*function)(void))` è la funzione per registrare un terminal handler. Le funzioni registrate non devono chiamare `exit()` per evitare loop ricorsivi senza uscita.]

La system call `_exit()/_Exit()`: Il kernel chiude i file descriptor aperti, scollega la memoria condivisa e gli IPC, invia `SIGCHLD` al processo padre per notificare la variazione di stato del figlio, memorizza il codice di uscita e adotta i processi figli orfani riassegnandoli a `init`.

Terminazione anomala: `abort()` (autoinvia `SIGABRT`) o ricezione di segnali fatali. Aggira le procedure di pulizia di `exit()`.

6.3 Ambiente di Esecuzione e Parametrizzazione

[L'analisi degli argomenti da linea di comando (es. flag come `-a` o opzioni con parametri) è semplificata dalla funzione POSIX `getopt()`, che esegue il parsing del vettore `argv` tramite variabili globali di stato come `optarg`, permettendo di definire i parametri accettati e le relative azioni tramite uno switch in un loop.]

Ogni processo eredita dal genitore (tipicamente la Shell) un **Ambiente di Esecuzione**: un vettore di stringhe formattate come `NOME=valore`. Le funzioni di accesso sono:

- `getenv("NOME")`: restituisce il puntatore al valore.
- `setenv("NOME", "valore", overwrite)`: inserisce o sovrascrive.
- `unsetenv("NOME")`: elimina la variabile.

[Con `setenv()` si inseriscono nuove coppie "nome=valore"; se il nome è già presente, viene sovrascritto il valore. Con `unsetenv()` si elimina una variabile d'ambiente.]

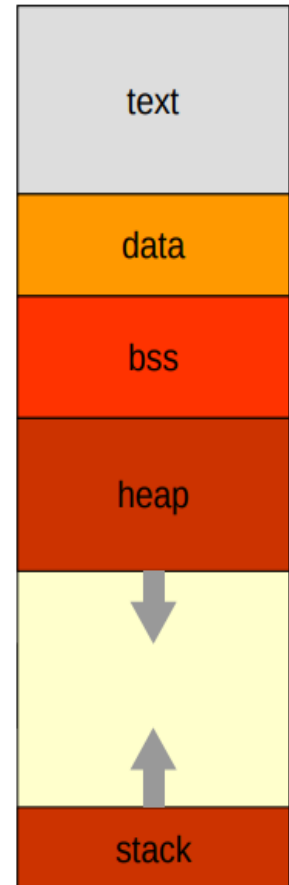
Variabili d'ambiente fondamentali:

- `HOME`: La directory predefinita dell'utente.
- `PWD`: La directory di lavoro corrente del processo.
- `PATH`: Una stringa che definisce una sequenza di percorsi separati da due punti (:). Quando il kernel deve lanciare un eseguibile specificato senza il suo percorso assoluto, interroga questa lista in ordine sequenziale per localizzare il binario (es. controllando in `/usr/local/bin`, poi `/usr/bin`, ecc.).

6.4 Layout della Memoria del Processo

L'immagine in memoria di un processo è divisa in regioni logiche (segmenti), organizzate per soddisfare i diversi requisiti dei dati e del codice:

1. **Segmento Text:** Contiene le istruzioni binarie (il codice macchina) ed è tipicamente a sola lettura. Dovrebbe generare errore il superamento dei limiti del segmento.
2. **Segmento Data:** Memorizza le variabili globali e statiche esplicitamente **inizializzate** a un valore nel codice sorgente.
3. **Segmento BSS (Block Started by Symbol):** Ospita le variabili globali e statiche **non inizializzate**. Per ottimizzare la dimensione del file eseguibile su disco, queste variabili non vengono salvate; il kernel alloca e azzerava dinamicamente questo segmento al caricamento del processo in memoria.
4. **Heap:** L'area dedicata all'allocazione dinamica gestita dal programmatore (tramite funzioni come `malloc`). L'espansione dell'Heap richiede l'intervento del kernel che altera il limite superiore dello spazio allocato (denominato *break* o `brk`). Inizialmente il break point è posizionato alla fine del **BSS** (heap vuoto).
5. **Stack:** Struttura LIFO utilizzata per gestire le chiamate a funzione (*stack frame*). Contiene gli indirizzi di ritorno e le variabili automatiche (locali) che esistono solo finché la funzione che le ha dichiarate è in esecuzione. [Esiste una funzione specifica, `alloca()`, per allocare memoria dinamicamente direttamente all'interno dello Stack Frame corrente; tale memoria viene distrutta automaticamente all'uscita dalla funzione, rimuovendo la necessità di chiamare esplicitamente la `free()`].



Heap e Stack si posizionano agli estremi della memoria disponibile per permettere la libera espansione senza collidere.

[Esiste la funzione `alloca()` per allocare memoria dinamicamente direttamente nello Stack Frame corrente; tale memoria viene deallocata automaticamente all'uscita dalla funzione, senza richiedere `free()`.]

6.5 Librerie Dinamiche

All'avvio del processo, il **Linker Dinamico** (*Loader*) identifica le dipendenze nell'intestazione dell'eseguibile e carica le librerie (`.so`) nello spazio di indirizzamento. I percorsi di ricerca sono influenzati dalla variabile `LD_LIBRARY_PATH`.

[L'amministratore può interrogare le dipendenze di un binario con l'utility `ldd`.]

Caricamento programmatico a runtime (API POSIX):

- `dlopen()`: carica una libreria nel processo. Modalità *lazy* (risolve i simboli solo al primo utilizzo) o *now* (risolve tutti i simboli prima del return della `dlopen()`).

- `dlsym(handle, "nome_simbolo")`: restituisce il puntatore a una funzione o variabile globale nella libreria. Handle speciali: `RTLD_DEFAULT` (prima occorrenza nell'ordine di default) e `RTLD_NEXT` (occorrenza successiva a quella corrente).

[Menzione collaterale merita la pratica dei salti non locali (`setjmp` e `longjmp`), che consentono di spostare istantaneamente il contesto di esecuzione su funzioni superiori nella catena di chiamate. Sono sconsigliati nei moderni paradigmi software a causa dei rischi di memory leak derivanti dalle deallocazioni non processate sullo stack intermedio.]

6.6 Limiti e Restrizioni dei Processi

Il kernel Unix applica restrizioni sull'uso delle risorse tramite le system call `getrlimit()` e `setrlimit()`. Ogni parametro ha due valori:

- **Soft Limit:** Limite attualmente in vigore; un processo non privilegiato può ridurlo ma non superare l'Hard Limit.
- **Hard Limit:** Tetto massimo assoluto; solo root può elevarlo.

Questi valori sono ereditati durante `fork()` e persistono attraverso `exec()`.

Limiti gestibili principali:

- `RLIMIT_AS`: dimensione massima dello spazio di indirizzamento.
- `RLIMIT_CORE`: dimensione massima del file core dump.
- `RLIMIT_CPU`: tempo CPU massimo consumabile.
- `RLIMIT_NOFILE`: numero massimo di file descriptor aperti.
- `RLIMIT_NPROC`: numero massimo di processi per utente.
- `RLIMIT_STACK`: dimensione massima dello stack.

[L'amministratore o l'utente possono ispezionare o modificare questi limiti per la shell corrente tramite `ulimit` (es. `ulimit -a` per elencarli tutti).]

6.7 Identificatori di Processo e Identità Utente

I PID possono essere riutilizzati dopo la terminazione di un processo. Alla terminazione, tutti i figli vengono ereditati da `init`.

Identità utente di un processo:

- **UID/GID Reale:** Identifica l'utente che ha materialmente lanciato il processo.
- **UID/GID Effettivo:** L'identità che il kernel valuta per concedere o negare l'accesso alle risorse (privilegi).

[Il kernel non considera l'ID reale per le decisioni di accesso, ma esclusivamente quello effettivo.]

Funzioni di lettura (non falliscono mai): `getpid()`, `getppid()`, `getuid()`, `geteuid()`, `getgid()`, `getegid()`.

6.8 Creazione dei Processi: `fork()` e Copy-On-Write

Copy-On-Write (COW): Al momento della `fork()`, le pagine di memoria **non** vengono copiate fisicamente. Vengono invece marcate in sola lettura e condivise tra padre e figlio tramite riferimenti incrociati. Solo quando uno dei due tenta una scrittura su una pagina, il kernel intercetta l'operazione e ne crea una copia indipendente per il processo scrivente. Questo evita sprechi di memoria, specialmente quando il figlio invoca immediatamente `exec()`.

Proprietà ereditate dal figlio: file descriptor aperti (con offset correnti), directory di lavoro, umask, identità utente, ambiente, limiti di risorse.

Proprietà non ereditate: allocazione CPU (azzerata), segnali pendenti, memorie bloccate dallo swap, operazioni I/O asincrone in corso, timer, lock su file.

6.9 Il Flag `close-on-exec`

Il flag `close-on-exec` su un file descriptor consente di mantenerlo aperto e accessibile dopo una `fork()`, ma di farlo chiudere automaticamente dal kernel al momento di una chiamata `exec()`. Evita la chiusura manuale di tutti i descrittori non desiderati nel nuovo programma.

6.10 Esecuzione di Programmi: La Famiglia `exec`

Le funzioni della famiglia `exec` sostituiscono l'immagine in memoria del processo corrente con quella di un nuovo programma. **Non ritornano mai** al chiamante in caso di successo. I suffissi determinano la sintassi dei parametri:

- **l (list) / v (vector):** Indicano se gli argomenti al programma sono passati come lista a lunghezza variabile (terminata da un puntatore `NULL`) oppure come vettore di puntatori a stringhe (array).
- **e (environment):** Permette di passare esplicitamente un vettore di stringhe contenente il nuovo ambiente (variabili d'ambiente).
- **p (path):** Consente di indicare soltanto il nome del file eseguibile. Il sistema si occuperà di cercarlo analizzando progressivamente i percorsi contenuti nella variabile d'ambiente `PATH`.

[Esiste inoltre il prefisso `f` per funzioni che accettano un file descriptor preaperto al posto di un percorso stringa.]

Durante `exec()`, il kernel verifica il formato dell'eseguibile (tipicamente ELF su Linux); se il file è testuale, prova a eseguirlo delegando l'interpretazione a `/bin/sh`.

Proprietà che persistono attraverso `exec()`: PID, PPID, directory correnti, file descriptor senza flag `close-on-exec`, Real UID/GID, Process Group ID, Session ID, terminale di controllo, timer interni, segnali pendenti, valore di *nice*, maschera dei segnali.

6.11 Alterazione dell'Identità e Sicurezza

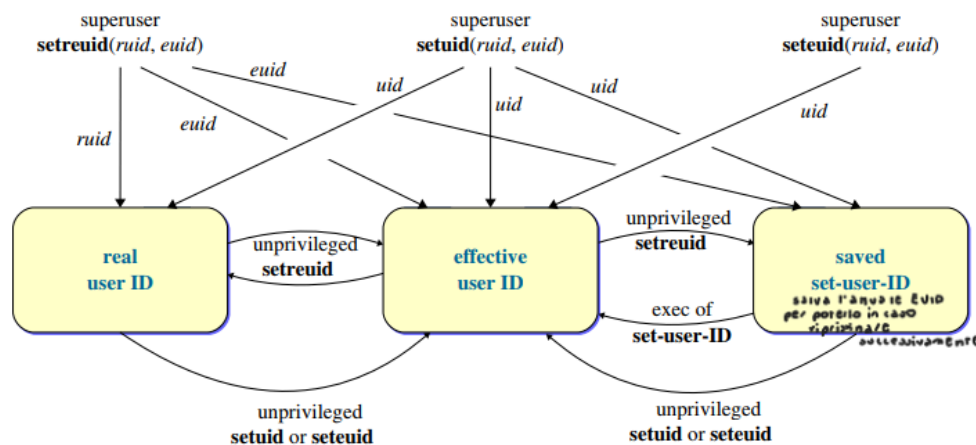
Bit SUID: Se il file eseguibile ha il bit Set-User-ID attivo, `exec()` imposta l'UID Effettivo del processo all'UID del proprietario del file, consentendo l'escalation temporanea dei privilegi.

Principio del minimo privilegio: È buona norma eseguire le operazioni con il numero minimo di privilegi necessario onde evitare potenziali errori o malfunzionamenti se i privilegi sono troppo alti. Si tiene traccia dell'identità precedente e si effettua un *roll-back* tramite il Saved Set-UID una volta finito il task con priorità più bassa.

Processi Privilegiati (UID Effettivo = 0): Possono assumere l'identità di qualunque utente, modificando UID Reale, Effettivo e Salvato.

Processi Non Privilegiati: Possono solo commutare l'UID Effettivo con il proprio UID Reale o recuperare il Saved UID. Ogni richiesta verso un UID diverso viene respinta con errore `EPERM`.

System call per l'identità: `setuid()`, `seteuid()`, `setreuid()`.



6.12 Attesa e Terminazione

- `wait()`: sospende il padre fino alla terminazione di un qualsiasi figlio.
- `waitpid()`: attende un figlio specifico (per PID) o un gruppo (PID negativo); supporta chiamate non bloccanti.

[Per completezza, esiste `waitid()`, che offre maggiore finezza sulle opzioni di blocco.]

6.13 La Funzione di Libreria `system()`

`system()` incapsula automaticamente `fork() + exec("/bin/sh -c ...") + wait()`.

Sebbene comoda per pipeline complesse, è:

- **Inefficiente:** overhead dovuto alla generazione di processi intermedi.
- **Pericolosa in contesti privilegiati:** le ampie manipolazioni ambientali della shell espongono a vulnerabilità gravi. **Non va mai usata in programmi con bit SUID attivo.**

[Durante l'utilizzo di primitive bloccanti come `wait()`, è comune gestire il codice d'errore `EINTR`, che non indica un fallimento irreversibile ma un'interruzione causata dalla ricezione di un segnale. L'implementazione conservativa standard ripete la chiamata.]

6.14 Terminali, Console e Procedure di Login

[Oggi l'interazione avviene prevalentemente tramite terminali virtuali o emulatori di terminale eseguiti all'interno di interfacce grafiche, noti come pseudo-tty.]

Catena di esecuzione del login:

1. `init` genera un processo `getty` per ogni linea di terminale.
2. `getty` inizializza la porta, richiede il nome utente e invoca `/bin/login`.
3. `login` verifica la password, configura UID/GID effettivi, inizializza le variabili d'ambiente (`HOME`, `SHELL`, `PATH`, `LOGNAME`) e lancia la shell predefinita.
4. Per accessi di rete: `sshd` (o `inetd`) biforca un figlio dedicato alla connessione, istanzia uno pseudo-terminale e invoca `login`.

6.15 Gruppi di Processi, Sessioni e Terminale di Controllo

6.15.1 La Gerarchia a Due Livelli

I processi sono organizzati in **Gruppi di Processi**, a loro volta racchiusi in **Sessioni**. Un gruppo non può estendersi su sessioni multiple; una sessione contiene più gruppi.

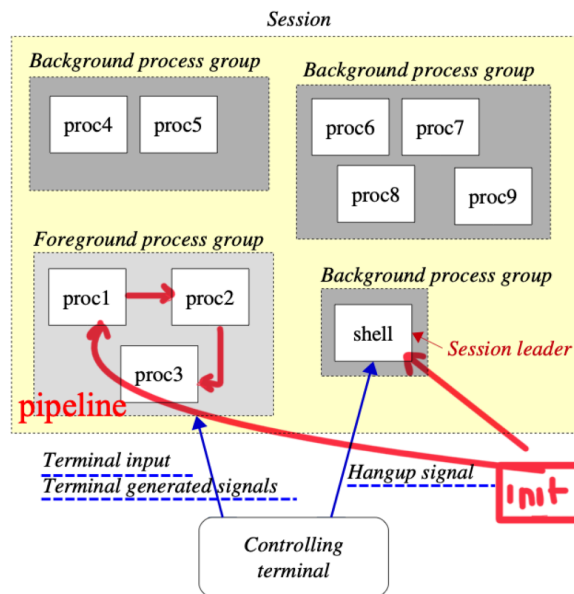
Leader del Gruppo: Identificato da un PGID; il leader è il processo il cui PID coincide con il PGID. I figli ereditano il PGID; un processo può lasciare il proprio gruppo tramite `setpgid(0, 0)`.

Leader della Sessione: Creato tramite `setsid()`. Un processo (non già leader di gruppo) diventa leader di una nuova sessione e di un nuovo gruppo, scollegandosi da eventuali terminali di controllo.

6.15.2 Il Terminale di Controllo

Una sessione può avere al più un **terminale di controllo** (accessibile come `/dev/tty`); un terminale può essere associato a una sola sessione alla volta. Il Session Leader lo acquisisce aprendo per primo il dispositivo terminale; può rinunciarvi con `ioctl(fd, TIOCNOTTY)`.

6.15.3 Foreground, Background e Gestione I/O



In ogni istante esiste un unico **Foreground Process Group**; tutti gli altri sono in background.

- **Input da tastiera:** riservato al gruppo in foreground. Un processo in background che tenta di leggere riceve `SIGTTIN` (default: sospensione). Se il segnale è bloccato/ignorato, la `read` fallisce con `EIO`.
- **Output:** consentito a tutti (default); se è attiva la flag `TOSTOP`, solo il foreground può scrivere; tentativi in background generano `SIGTTOU`.
- **Pipeline in foreground:** (comando1 | comando2) costituisce un gruppo che riceve input, output e segnali dal terminale. Shell dotate di *job control* permettono di avere multiple pipeline sospese o in esecuzione in background contemporaneamente, potendole scambiare. Quando un gruppo in foreground termina o viene sospeso, la shell (Session Leader) riceve un segnale (come `SIGCHLD`) e si riporta autonomamente in foreground.

6.15.4 Generazione e Inoltro dei Segnali

- **^C / ^Z:** generano rispettivamente `SIGINT` e `SIGTSTP`, inviati a tutti i membri del gruppo in foreground.
- **Disconnessione di rete/chiusura finestra:** genera `SIGHUP` inviato al Session Leader.

- **Terminazione del Session Leader con terminale:** il terminale viene scollegato dalla sessione (rendendolo disponibile per altre) e il gruppo in foreground riceve `SIGHUP`.

6.15.5 API per Gruppi e Terminali

Lo standard di sistema espone chiamate precise per manipolare queste gerarchie :

- `getpgrp()` / `getpgid(pid)`: Restituiscono il Process Group ID del processo corrente o di un PID specifico.
- `setpgid(pid, pgid)`: Assegna un processo a un gruppo. Sottostà a rigide regole di sicurezza: il figlio non deve aver già invocato una `exec()` e mittente/destinatario devono trovarsi nella medesima sessione.
- `tcgetpgrp(fd)`: Interroga il file descriptor del terminale per scoprire quale sia attualmente l'ID del gruppo in foreground.
- `tcsetpgrp(fd, pgrp_id)`: Impone al terminale di associare lo stato di foreground al gruppo specificato.

6.15.6 Gruppi di Processi Orfani

Un gruppo è **orfano** se il genitore di ogni membro si trova nello stesso gruppo o in una sessione diversa. Quando un gruppo diventa orfano con almeno un processo in stato *stopped*, il kernel invia a tutti i membri prima `SIGHUP` (terminazione default) e poi `SIGCONT` (per permettere la gestione di `SIGHUP`).

I gruppi orfani che tentano di accedere al terminale senza gestire `SIGTTIN/SIGTTOU` ricevono errore `EIO`. Il segnale `SIGTSTP` da terminale è consegnato ai gruppi orfani solo se viene gestito esplicitamente.

[In pratica: si consideri una shell (Gruppo A) che genera una pipeline $p1 \rightarrow p2 \rightarrow p3$ (Gruppo B). Se $p1$ muore inaspettatamente, $p2$ viene "adottato" dal processo radice (`init`). A questo punto il Gruppo B diventa orfano perché nessuno dei genitori dei processi rimasti si trova più nella stessa sessione ma in un gruppo diverso che possa fungere da controllore attivo.]

6.16 Processi Demone (Daemon)

I demoni sono processi eseguiti silenziosamente in background per l'intera durata del sistema, privi di terminale di controllo. Si distinguono dai thread kernel (es. `kworker`, `kswapd0`) in quanto girano a livello utente e lanciati all'avvio per la gestione di servizi continui (server web, accessi remoti).

Procedura canonica di daemonizzazione:

1. `umask(0)`: azzera la maschera di creazione file ereditata.
2. Prima `fork()` + terminazione del padre: il figlio diventa orfano adottato da `init`, perdendo lo status di leader di gruppo.

3. `setsid()`: crea una nuova sessione autonoma e sgancia il demone dal terminale originario.
4. Seconda `fork()` + terminazione del padre: garantisce che il demone non possa mai acquisire accidentalmente un nuovo terminale di controllo (non essendo più Session Leader).
5. `chdir("/")`: evita di mantenere bloccati file system montati.
6. Chiusura di tutti i file descriptor superflui ereditati.
7. Riapertura di `0, 1, 2` redirigendoli su `/dev/null`.

*[La logica amministrativa richiede spesso che i demoni garantiscano un'istanza singola tramite **lock file** in `/var/run` o `/var/lock/` (es. `nome.pid`). La configurazione in `/etc/` viene letta all'avvio; per ricaricarla a caldo si notifica il demone tramite `SIGHUP`.]*

6.17 Registrazione degli Errori e Sottosistema Syslog

*[Data la mancanza di un terminale per l'emissione dello standard error,] le applicazioni a livello utente e i demoni non possono stampare diagnostica su schermo. Si affidano pertanto al sottosistema centralizzato **syslog**, gestito da demoni specializzati (come **syslogd**).*

Il demone di logging preleva i messaggi generati dai processi, li colleziona e li categorizza, smistandoli in file specializzati sotto `/var/log/` [es. `auth.log` per l'autenticazione o la console globale per i messaggi imminenti di spegnimento di sistema] in base a regole configurabili dall'amministratore. **syslogd** raccoglie i dati da tre vettori principali: dispositivi kernel (es. `/dev/klog`), socket UNIX locali (es. `/dev/log`) e socket TCP/UDP per la rete (es. porta 514 [inoltrati via UDP per log remoti centralizzati]).

API POSIX:

- `openlog(ident, options, facility)`: inizializza il canale; la *facility* classifica il programma (es. `LOG_DAEMON`, `LOG_AUTH`, `LOG_CRON`).
- `syslog(priority, format, ...)`: invia il messaggio (`/dev/log`) con una priorità di gravità (da `LOG_EMERG` a `LOG_DEBUG`).
- `closelog()`: chiude il canale.
- `setlogmask(mask)`: filtra programmaticamente le priorità, scartando messaggi al di sotto della soglia impostata:
`(LOG_PRI(priorità1) | - | LOG_PRI(prioritàN))`.

7 – Input/Output di Base nei Sistemi Unix (I/O Non Bufferizzato)

7.1 File Descriptor e I/O di Basso Livello

L'interfaccia fondamentale per l'accesso ai file in POSIX si basa sulle system call dirette, dette **non bufferizzate**. Il termine non indica l'assenza di buffer nel kernel, bensì l'assenza di buffer aggiuntivi gestiti dalla libreria utente: ogni chiamata corrisponde esattamente a una system call verso il kernel.

I file attivi sono rappresentati univocamente da **File Descriptor**, valori interi non negativi il cui numero massimo per processo è controllato dal limite `OPEN_MAX`.

[Sebbene i numeri 0, 1 e 2 corrispondano da sempre ai canali in, out ed error, la buona prassi implementativa suggerisce di utilizzare le costanti `STDIN_FILENO`, `STDOUT_FILENO` e `STDERR_FILENO`.]

7.2 Apertura e Creazione di File (open e creat)

La system call `open()` restituisce il **numero di file descriptor libero più basso**. In caso di errore restituisce `-1` e imposta `errno`.

Flag obbligatori (mutuamente esclusivi):

- `O_RDONLY`: sola lettura.
- `O_WRONLY`: sola scrittura.
- `O_RDWR`: lettura e scrittura.

Flag aggiuntivi (combinabili con OR bit a bit):

- condizione costringe chi programma a inserire un terzo argomento opzionale alla funzione per definire analiticamente i permessi attribuiti in fase di genesi del file. [L'effetto combinato di apertura, creazione e svuotamento formava in passato un costruito a sé stante rappresentato dalla direttiva `creat()`].
- `O_EXCL`: Rende intrinsecamente esclusiva la generazione. In associazione a `O_CREAT`, blocca in errore irrevocabile il tentativo se il file indicato è preesistente.
- `O_TRUNC`: Determina lo svuotamento automatico di qualsiasi contenuto preesistente del file nel medesimo momento dell'apertura, riportando a zero i byte archiviati.
- `O_APPEND`: Interviene sulla coda dei dati bloccando il punto logico in modo che ogni singolo flusso di scrittura ricada tassativamente ed atomicamente alla fine dei dati preesistenti.
- `O_NONBLOCK`: Vincola I/O a non bloccare e disinnescare la sospensione in attesa del sistema operativo, utile se impiegato su pipeline per ricevere istantaneamente

notifica d'assenza d'informazioni dal canale comunicativo provocando l'insuccesso della chiamata a seguire e consentendo la prosecuzione nel calcolo.

- **Sincronizzazione Hardware (`O_SYNC`, `O_DSYNC`, `O_RSYNC`):** Impongono il raggiramento o l'attesa dei buffer ritardati in capo al kernel, pretendendo che le informazioni e i metadati transigano direttamente al disco d'archiviazione e ostacolando eventuali perdite per corruzione accidentale della RAM o blocchi elettrici.

[Il corredo comprende opzioni per non seguire i link simbolici (`O_NOFOLLOW`) e per impostare automaticamente il flag close-on-exec (`O_CLOEXEC`).]

[Se le configurazioni della distribuzione lo prevedono, l'accesso a nomi di file con lunghezza incompatibile con il file system genera un errore programmatico invece di essere silenziosamente troncato, come previsto da `POSIX_NO_TRUNC`.]

7.3 Relatività, Sicurezza e Protezione Concorrenziale (`openat`)

`openat(dirfd, path, flags, ...)` estende `open()` con un file descriptor di directory come primo argomento. Se `path` è relativo, la base non è la CWD del processo bensì la directory identificata da `dirfd`. Risolve due criticità:

Architetture Multithreaded: Il cambio della CWD (tramite `chdir`) è globale al processo e affligge tutti i thread. `openat()` permette a thread diversi di operare con basi diverse senza interferenze.

Race Condition TOCTOU (*Time-Of-Check to Time-Of-Use*): Un attaccante può sostituire una directory o un file nell'intervallo tra la verifica di sicurezza e l'effettiva operazione. Ancorando l'operazione a un file descriptor aperto e cristallizzato in precedenza, `openat()` segue gli spostamenti del file system in modo coerente, prevenendo questo vettore d'attacco.

7.4 Lettura e Scrittura (`read`, `write`, `close`)

Le signature principali:

1. `ssize_t read(int fd, void *buf, size_t count);`
2. `ssize_t write(int fd, const void *buf, size_t count);`
3. `int close(int fd);`

Entrambe restituiscono il numero di byte effettivamente trasferiti (tipo `ssize_t`) o `-1` in caso di errore.

[Ai fini ingegneristici è sconsigliato allocare buffer di grandi dimensioni nello stack (che ha spazio limitato e deallocazione automatica); si preferisce l'allocazione sull'Heap tramite

malloc. È inoltre buona norma limitare la visibilità delle variabili globali al solo modulo tramite il qualificatore `static` in C.]

Ritorni parziali (eventi fisiologici):

- In `read()`: il file può terminare (ritorno 0 = EOF) prima del raggiungimento del conteggio richiesto; i terminali trasmettono una riga alla volta; i canali IPC e le socket possono avere dati parzialmente disponibili; un segnale può interrompere la chiamata bloccante.
- In `write()`: riempimento del buffer di socket, superamento dei limiti di quota disco o interruzione da segnale possono causare scritture parziali.

Il codice produttivo deve sempre gestire i ritorni parziali tramite un loop che itera `read()/write()` fino al completamento dell'intera operazione o fino all'incontro di una condizione di errore non recuperabile. Ignorare i ritorni parziali è una delle sorgenti più frequenti di bug sottili nei programmi di sistema.

7.5 Modifica del Punto d'Accesso e Sparse File (lseek)

Ogni file descriptor mantiene un **offset corrente** che avanza automaticamente a ogni operazione. La system call `lseek()` lo modifica esplicitamente:

```
4. off_t lseek(int fd, off_t offset, int whence);
```

Valori di `whence`:

- **SEEK_SET**: Posizione iniziale del file.
- **SEEK_CUR**: Posizione corrente
- **SEEK_END**: Posizione finale del file

Su file descriptor non posizionabili (pipe, socket, terminali), `lseek()` fallisce con errore `ESPIPE`.

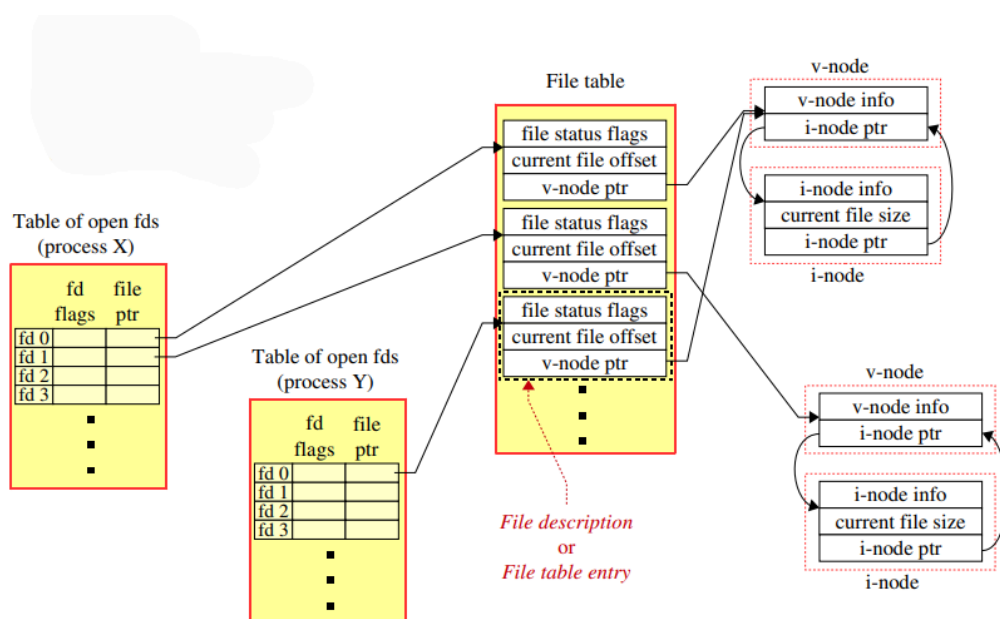
[Nelle distribuzioni a 32 bit, gli offset sono limitati a 4 GB. Il problema si risolve definendo `_FILE_OFFSET_BITS=64` nel sorgente (o in fase di compilazione) senza richiedere modifiche architetturali al codice, purché i tipi `off_t` siano usati coerentemente.]

8 – Architettura Interna dell'I/O, Concorrenza e Libreria Standard

8.1 Strutture Dati del Kernel per la Gestione dei File

Per permettere a processi multipli, anche inconsapevoli l'uno dell'altro, di condividere l'accesso ai medesimi file, il kernel di un sistema operativo Unix-like implementa un'architettura logica basata su tre strutture dati principali:

- **Tabella dei Processi e Tabella dei File Descriptor (Locale):** Il kernel mantiene una traccia globale di tutti i processi. All'interno della struttura associata a ciascun processo, esiste una "Tabella dei File Descriptor" locale. Ogni *entry* in questa tabella rappresenta un file descriptor in uso o disponibile e contiene i flag specifici di quel descrittore [come il flag `close-on-exec`] e un puntatore a una struttura dati di livello inferiore.
- **Tabella dei File (Globale):** Questa struttura è centralizzata e condivisa tra tutti i processi del sistema. Contiene le "File Description" (da non confondere con i descrittori). Per ogni file aperto, una *entry* in questa tabella memorizza lo stato di accesso (lettura, scrittura), i flag applicati al momento dell'apertura (es. sincronizzazione), l'offset corrente (la posizione attuale di lettura/scrittura) e un puntatore alla struttura v-node/i-node.
- **Tabella degli i-node (Globale):** Rappresenta il file fisico residente sullo storage. Contiene metadati cruciali centralizzati, come le dimensioni del file, i permessi, i *timestamp* e i puntatori ai blocchi fisici del disco. Un singolo file fisico possiede un unico i-node, indipendentemente da quanti processi lo stiano leggendo.



8.2 Condivisione dei File e Viste Indipendenti

A seconda di come un file viene aperto, l'interazione tra i processi può variare drasticamente:

- **Viste Indipendenti:** Se due processi (o lo stesso processo in momenti distinti) eseguono una system call `open()` sul medesimo file, il kernel genera due file descriptor distinti che puntano a *due entry separate* nella Tabella dei File Globale, le quali punteranno a loro volta al medesimo i-node. Avendo entry diverse, i due accessi mantengono offset e flag completamente indipendenti: i processi possono scorrere il file in posizioni diverse contemporaneamente senza interferire. Eventuali modifiche ai dati o ai metadati (es. dimensione) risulteranno tuttavia visibili a entrambi.
- **Viste Condivise:** Se due file descriptor (nello stesso processo o in processi distinti) puntano alla *stessa entry* nella Tabella dei File Globale, essi condividono lo stesso offset e gli stessi flag di stato. Una lettura che fa avanzare l'offset tramite un descrittore influenzerà la posizione di partenza di una successiva lettura effettuata tramite l'altro.

8.3 Operazioni Atomiche e Prevenzione delle Race Condition

Quando più processi agiscono in parallelo, sequenze di operazioni non atomiche possono generare corruzione dei dati a causa di *race condition* (condizioni di competizione).

- **Atomicità in Scrittura (Append):** Se due processi tentano di accodare dati allo stesso file eseguendo in sequenza una `lseek()` fino alla fine del file (EOF) seguita da una `write()`, lo scheduler potrebbe sospendere il primo processo subito dopo la sua `lseek()`. Se il secondo processo esegue `lseek()` e `write()`, altera la fine del file; quando il primo processo riprende, sovrascriverà i dati appena inseriti dal secondo. [Come accennato nel capitolo precedente, l'uso del flag `O_APPEND` all'apertura del file risolve il problema imponendo al kernel di posizionarsi atomicamente alla fine del file ad *ogni* singola operazione di scrittura.]
- **Accesso Posizionale Atomico (`pread` / `pwrite`):** In scenari in cui occorre leggere o scrivere dati a offset specifici e casuali, senza alterare l'offset globale del file (che potrebbe essere in uso da altri thread), il sistema POSIX fornisce le funzioni `pread()` e `pwrite()`. Queste ricevono l'offset come argomento addizionale, effettuano lo spostamento e l'I/O in maniera del tutto atomica e lasciano invariato l'offset corrente della *file description*. Tali primitive falliscono logicamente se il file non supporta il posizionamento (es. pipe o socket).
- **Creazione Esclusiva:** [Il flag `O_EXCL` in combinazione con `O_CREAT` è essenziale per generare file privati e sicuri.] Senza di esso, un tentativo di verifica pre-esistente che fallisce, seguito da una creazione, potrebbe essere interrotto da un processo malevolo che crea il file nel mezzo, alterandone i permessi o i riferimenti logici. Con

`O_EXCL`, l'assenza e la successiva creazione del file avvengono atomicamente dal lato del kernel.

8.4 Duplicazione dei Descrittori e Redirezione dell'I/O

Per manipolare intenzionalmente le tabelle dei file in modo da far convergere più descrittori verso la stessa risorsa logica, si ricorre alle system call di duplicazione (vogliamo due file description che puntano alla stessa file description):

- **`dup()`**: Riceve in input un file descriptor valido e ne restituisce uno nuovo (scegliendo il numero intero più basso correntemente disponibile). Entrambi punteranno alla medesima entry nella Tabella dei File. [Il nuovo descrittore nasce con il flag *close-on-exec* disabilitato.]
- **`dup2()`**: Permette di specificare quale numero intero assegnare al nuovo file descriptor. Qualora l'identificativo richiesto sia già in uso e mappato a un file, **`dup2`** si occuperà di chiuderlo atomicamente prima di procedere alla sovrascrittura logica. Se i due identificatori passati coincidono, la funzione non compie alcuna operazione.

Il meccanismo della redirezione (Shell): Queste funzioni sono la spina dorsale della redirezione dell'I/O eseguita dalla shell (es. `< file.txt`). La procedura tipica implementata dalla shell prevede di:

1. Chiudere il file descriptor standard (es. `close(0)` per lo standard input).
2. Eseguire una `open()` sul file desiderato. Poiché il sistema alloca il file descriptor più basso libero, esso assegnerà proprio lo `0`, mappandolo al file di testo.
3. Richiamare `exec()` per lanciare il programma richiesto dall'utente.

[Poiché la `exec()` preserva i file descriptor sprovvisti di flag di chiusura, il nuovo programma continuerà a leggere ignaro dal proprio descrittore `0` credendo di dialogare con un terminale interattivo, mentre i dati proverranno dal file re-indirizzato dalla shell.]

8.5 Sincronizzazione dei Buffer del Kernel (Flushing)

[L'I/O "non bufferizzato" significa unicamente assenza di buffer a livello libreria utente, ma implica comunque il transito nei buffer gestiti internamente al kernel.] Per garantire l'affidabilità e l'effettivo deposito delle informazioni sulle memorie di massa persistenti, esistono funzioni di svuotamento (*flushing*):

- **`fsync(fd)`**: Forza lo svuotamento dei buffer associati al file descriptor indicato e blocca l'esecuzione del processo finché i dati e i relativi metadati (come dimensioni e timestamp aggiornati) non sono materialmente scritti sul disco.

- **`fdatasync(fd)`**: Variante più rapida [affine alle funzionalità del flag `O_DSYNC`] che assicura il trasferimento fisico dei soli dati, procrastinando se possibile l'aggiornamento dei metadati non strettamente critici.
- **`sync()`**: A differenza delle precedenti, questa funzione non accetta argomenti; essa comunica al kernel la direttiva di accodare massivamente la sincronizzazione di *tutti* i buffer ritardati presenti nel sistema. Sebbene restituisca immediatamente il controllo, non garantisce che i dischi abbiano concluso l'operazione. Risulta fondamentale prima di procedure di ibernazione, snapshot o arresto del sistema.

8.6 Manipolazione dei Descrittori a Runtime (**`fcntl`**)

Una volta che un file è stato aperto, alterarne i flag intrinseci (come l'attivazione a posteriori del `close-on-exec`) non può essere fatto invocando nuovamente `open()`. Si ricorre alla funzione **`fcntl()`** (File Control). Questa system call riceve il descrittore, un comando operativo specifico e un argomento variabile. Oltre a ispezionare e sovrascrivere i flag del descrittore o del file, `fcntl()` implementa comandi come `F_DUPFD`, il quale costituisce un metodo analogo (e storicamente fondativo) per attuare la medesima duplicazione operativa gestita oggi da `dup` e `dup2`.

8.7 Il Flag **`O_NONBLOCK`** e l'Errore **`EAGAIN`**

[Le operazioni di lettura e scrittura su file regolari sono intrinsecamente "non bloccanti" dal punto di vista logico: il dato è fisicamente presente sul disco e, al netto dei fisiologici tempi di latenza dell'hardware, il kernel è sempre in grado di soddisfare la richiesta o di segnalare la fine del file.]

La situazione cambia radicalmente quando si opera su canali di comunicazione inter-processo (IPC) come Pipe, FIFO o Socket. In questi scenari, se un processo tenta di leggere da un canale vuoto (o di scrivere su un canale pieno), il comportamento di default del kernel è quello di bloccare il processo chiamante, sospendendolo finché la risorsa non si rende disponibile.

In architetture ad alte prestazioni, per evitare che un processo si sospenda in attesa perdendo cicli di elaborazione, è possibile attivare il flag `O_NONBLOCK` [(impostabile in fase di apertura, o successivamente tramite la funzione `fcntl()`)]. Se l'operazione di I/O non può essere completata immediatamente, la system call fallisce in modo istantaneo e restituisce un errore specifico identificato dalla costante macro **`EAGAIN`** (o `EWOULDBLOCK`). Questo codice diagnostico indica esplicitamente che la risorsa non è pronta, permettendo al processo di dedicare la CPU ad altre computazioni e di ritentare l'operazione in un secondo momento.

8.8 Lock sui File (File Locking) e Sincronizzazione

Attraverso la system call `fcntl()`, il sistema Unix offre un meccanismo di sincronizzazione nativo denominato **File Locking** (o *Record Locking*).

Invece di utilizzare primitive di sincronizzazione esterne, un processo può richiedere al kernel di applicare un "lock" logico sull'accesso a un file. La particolarità e la potenza di questo strumento risiedono nella sua estrema granularità: non è obbligatorio bloccare l'accesso all'intero file, ma è possibile specificare un esatto *range di byte* su cui si intende operare. Questo paradigma consente a molteplici processi di scrivere simultaneamente su porzioni differenti del medesimo file senza causare corruzione dei dati.

[Come precedentemente analizzato, qualora due file descriptor puntino alla medesima *File Description* nella tabella globale del kernel (ad esempio a seguito di una `dup()` o di una `fork()`), un'alterazione dei flag di stato tramite `fcntl()` su uno dei descrittori modificherà istantaneamente il comportamento anche dell'altro descrittore, poiché la struttura dati sottostante che governa lo stato è condivisa tra i due.]

8.9 Posizionamento nel File: Astrazioni della Libreria `stdio.h`

[Fino ad ora, il posizionamento dell'offset è stato gestito utilizzando la system call pura `lseek()`, che accetta e restituisce un tipo di dato `off_t`.] Lavorando con le interfacce bufferizzate della Libreria Standard del C (`stdio.h`), le funzioni a disposizione per navigare all'interno del file (rappresentato dal puntatore opaco `FILE *`) si diversificano in base allo standard di riferimento:

- **`ftell()` e `fseek()`**: Interfacce storiche del linguaggio C, basate sull'architettura dei numeri interi lunghi (`long int`). [Il loro limite strutturale primario emerge sui sistemi operativi a 32-bit: un `long int` con segno non può rappresentare posizioni superiori a 2 Gigabyte, rendendo queste funzioni inefficaci per la gestione di file di grandi dimensioni.]
- **`ftello()` e `fseeko()`**: Introdotte dallo standard POSIX per superare i limiti storici dell'architettura a 32-bit. Sono semanticamente identiche alle precedenti, ma sostituiscono il tipo `long int` con il tipo `off_t` [(lo stesso utilizzato nativamente da `lseek()`)]. Questo garantisce che l'offset sia sempre rappresentato da un tipo di dato sufficientemente ampio per l'architettura su cui il codice viene eseguito, prevenendo fenomeni di *integer overflow*.
- **`fgetpos()` e `fsetpos()`**: Appartenenti allo standard C ISO (e pertanto garantite su qualsiasi piattaforma, non solo Unix), astraggono completamente il concetto di posizione utilizzando un tipo di dato dedicato e opaco denominato `fpos_t` (spesso implementato internamente dal compilatore come una `struct` complessa).
 - `fgetpos()` acquisisce la posizione logica attuale e la salva nella variabile `fpos_t` fornita tramite puntatore.

- `fsetpos()` utilizza quella stessa variabile per ripristinare il cursore logico all'esatta posizione memorizzata.
- L'utilizzo di questo tipo astratto assicura la portabilità assoluta del codice cross-platform, pur nascondendo programmaticamente al programmatore il valore numerico effettivo in byte dell'offset.

8.9.1 Interazione Diretta con il Dispositivo (`ioctl`)

Quando si opera su dispositivi hardware astratti come file (es. terminali o porte seriali), le funzioni standard di lettura e scrittura non sono sufficienti per inviare comandi di controllo specifici del dispositivo (es. variare il *baud rate* di una seriale o disabilitare l'eco dei caratteri su un terminale).

In questi casi, la libreria standard e le system call standard delegano l'operazione alla funzione `ioctl()` (Input/Output Control). Questa system call riceve il file descriptor del dispositivo, un codice di richiesta specifico (dipendente dal dispositivo stesso) e, opzionalmente, un terzo parametro (solitamente un puntatore) per inviare o ricevere configurazioni. Poiché i comandi di `ioctl` non sono standardizzati ma variano per ogni famiglia di dispositivi, l'uso corretto richiede la consultazione della documentazione specifica del driver associato.

[In alcuni sistemi (non rigorosamente standard POSIX ma supportati dalla maggior parte delle distribuzioni Linux), è possibile accedere a una directory virtuale `/dev/fd/` che mappa i file descriptor aperti dal processo corrente. Aprire `/dev/fd/0` equivale logicamente a duplicare lo standard input, un meccanismo a volte utilizzato in script di shell avanzati ma sconsigliato in codice C portabile].

8.9.2 L'I/O Bufferizzato: Gli Stream della Libreria Standard

Per isolare il programmatore dalla complessità e dall'overhead delle system call dirette, la libreria C (libc) implementa un'interfaccia di I/O ad alto livello basata sul concetto di **Stream**, rappresentati dal tipo opaco `FILE *`. Ogni stream incapsula internamente:

- Il file descriptor associato (`int`).
- I flag di accesso e di stato degli errori.
- Un **Buffer di memoria in User Space** gestito direttamente dalla libreria.
- I cursori che tracciano l'avanzamento dei dati letti/scritti nel buffer.

Politiche di Bufferizzazione

La gestione di questo buffer locale può seguire tre paradigmi distinti, configurabili tramite le funzioni `setvbuf()` o `setbuf()` prima di effettuare qualsiasi operazione di I/O sullo stream:

1. **Fully Buffered (Bufferizzazione Totale):** I dati vengono scritti nel buffer della libreria. La system call `write()` viene invocata automaticamente dalla libreria *solo* quando il buffer è completamente pieno (o viene chiamato esplicitamente

`fflush()`). In lettura, la libreria preleva un grosso blocco di dati con una singola `read()` e lo serve al programma a porzioni, riducendo drasticamente le invocazioni al kernel. [Questa è la politica di default per l'accesso a file regolari su disco].

2. **Line Buffered (Bufferizzazione di Riga):** Ottimizzata per l'interazione umana testuale. La libreria attende di rilevare il carattere di fine riga (newline, `\n`) prima di trasferire l'intero blocco di testo tramite system call. [Questa è la politica tipica di default quando lo stream è collegato a un terminale (es. lo *standard output*, `stdout`), permettendo di stampare righe intere in modo coerente].
3. **Unbuffered (Non Bufferizzato):** La libreria non utilizza alcun buffer. Ogni invocazione di una funzione di libreria (es. `fputc`) si traduce in un'immediata system call corrispondente, senza accumulare dati. [Questa è l'impostazione obbligata e garantita dallo standard C per lo *standard error* (`stderr`), affinché i messaggi di crash o di diagnostica raggiungano istantaneamente l'utente, prevenendo perdite se il processo terminasse rovinosamente distruggendo un eventuale buffer in memoria].

Apertura Avanzata degli Stream (`freopen` e `fdopen`)

Oltre alla classica `fopen()`, la libreria fornisce metodi per gestire l'apertura e l'associazione degli stream in scenari più complessi:

- **`freopen()`:** Chiude lo stream specificato (se aperto) e lo riapre associandolo a un nuovo file. È utilizzata tipicamente per redirigere i canali standard (`stdin`, `stdout`, `stderr`) verso dei file specifici all'interno del programma. [Questa chiamata provvede anche a pulire e reimpostare l'orientamento precedente dello stream (byte-oriented o wide-oriented)].
- **`fdopen()`:** Permette di associare uno stream della libreria C (un oggetto `FILE *`) a un file descriptor di basso livello (`int`) preesistente. [Risulta fondamentale quando si ottiene un descrittore da system call POSIX (come `pipe`, `socket` o `open` con flag specifici non supportati da `fopen`) ma si desidera poi manipolarlo con le più comode funzioni bufferizzate].

Interazione con i File Descriptor (`fileno`)

`fileno()`: Esegue l'operazione opposta rispetto a `fdopen()`. Dato un puntatore a uno stream (`FILE *`), restituisce il file descriptor (`int`) associato internamente. [Questo è indispensabile quando si deve invocare una system call che non accetta oggetti stream, come `fcntl()`, `fsync()` o `dup2()`].

Funzioni di I/O Orientate agli Stream

La libreria fornisce interfacce per accessi specializzati:

- **Per Carattere:** `fgetc()`, `fputc()` operano su un singolo byte alla volta. [Esistono le macro `getc()/putc()` ottimizzate e le varianti specifiche per l'alfabeto esteso e Unicode (`getwc`, `putwc`), utilizzabili previa configurazione dell'orientamento dello stream tramite `fwide()`]. [Una funzionalità unica di questo livello è `ungetc()`, che

permette al parser di "rimettere" logicamente un carattere nel buffer dello stream (spostando semplicemente il cursore a ritroso), utile durante la scrittura di analizzatori lessicali].

- **Per Riga:** `fgets()`, `fputs()` elaborano stringhe fino al terminatore di fine linea. [La funzione `gets()` storica è totalmente deprecata per questioni di sicurezza, poiché, non permettendo di specificare la dimensione del buffer target, espone il software a letali attacchi di Buffer Overflow].
- **Per Blocchi (Dati Binari):** `fread()`, `fwrite()` operano su vettori di strutture dati o byte raw, indicando la dimensione del singolo oggetto e il numero di oggetti da elaborare. Restituiscono il numero di *oggetti* processati correttamente (non il numero di byte).

I/O Formattato

Le funzioni di I/O formattato consentono di convertire stringhe di testo in tipi di dato primitivi (e viceversa) attraverso stringhe di formato:

- **Output Formattato:** * `printf()`: Stampa sullo standard output.
 - `fprintf()`: Stampa su uno stream `FILE *` specificato.
 - `sprintf()` e `snprintf()`: Stampano all'interno di un buffer di memoria (array di caratteri). [`snprintf()` è la versione sicura che previene il buffer overflow, richiedendo la specifica della dimensione massima del buffer target].
 - `dprintf()`: Stampa direttamente su un file descriptor di basso livello (POSIX).
- **Input Formattato:** * `scanf()`: Legge dallo standard input.
 - `fscanf()`: Legge da uno stream `FILE *`.
 - `sscanf()`: Legge analizzando una stringa in memoria.

Posizionamento negli Stream

Per spostare il cursore logico all'interno dello stream, la libreria fornisce diverse funzioni basate su standard differenti:

- **Standard C Storico:** `ftell()` (restituisce la posizione attuale) e `fseek()` (sposta il cursore). [Fanno uso del tipo `long int`, risultando problematiche su sistemi a 32-bit per file di dimensioni superiori a 2 GB].
- **Estensione POSIX:** `ftello()` e `fseeko()`. Operano in maniera identica ma impiegano il tipo nativo `off_t`, superando i limiti dimensionali del `long int`.
- **Standard C Moderno/ISO:** `fgetpos()` e `fsetpos()`. Utilizzano un tipo di dato astratto e opaco, `fpos_t`, per garantire la massima portabilità nella registrazione e nel ripristino di una specifica posizione.
- **`rewind()`:** Riporta istantaneamente il cursore all'inizio dello stream [equivalente a `fseek(fp, 0, SEEK_SET)`].

File Temporanei

Per l'archiviazione volatile di dati in esecuzione, sono fornite apposite funzioni di creazione:

- **Standard ISO C:** `tmpfile()` crea automaticamente un file temporaneo univoco aperto in modalità binaria di aggiornamento (`wb+`). La peculiarità di questo approccio è che il file viene scollegato logicamente (*unlinked*) dal file system subito dopo la creazione: scomparirà fisicamente alla chiusura dello stream (`fclose`) o al termine del programma. [La funzione `tmpnam()`, che generava solo il nome univoco del file senza crearlo, è oggi sconsigliata e deprecata per vulnerabilità legate alle *race conditions*].
- **Standard POSIX:** `mkstemp()` richiede una stringa template (che deve terminare obbligatoriamente con 6 caratteri `XXXXXX`), modifica il template con un nome univoco, crea il file in modo sicuro restituendone direttamente il *file descriptor*. Parallelamente, `mkdtemp()` crea in modo univoco e sicuro un'intera directory temporanea.

Memory Streams (Stream in Memoria)

Introdotti nelle specifiche POSIX.1-2008, i Memory Stream permettono di utilizzare le potenti funzioni di I/O della libreria standard (come formattazione e buffering) direttamente su blocchi di memoria RAM, senza coinvolgere il file system fisico. L'output diventa visibile solo dopo una `fflush()` o una `close()`.

- **`fmemopen()`:** Apre uno stream che legge e scrive su un buffer di dimensione fissa fornito dal programmatore. [Se al posto del buffer viene passato `NULL`, la libreria alloca la memoria dinamicamente e la rilascerà in automatico alla chiusura dello stream]. [Se vi è spazio residuo, le operazioni di scrittura appongono automaticamente il terminatore di stringa `\0`].
- **`open_memstream()`:** Apre uno stream orientato ai byte in *sola scrittura*. Il buffer viene allocato dinamicamente e cresce in modo automatico se si scrivono dati oltre la capacità iniziale. [A differenza di `fmemopen`, la deallocazione tramite `free()` del buffer finale è una responsabilità esplicita del chiamante dopo aver chiuso lo stream]. [Esiste inoltre la variante `open_wmemstream()`, progettata per operare su stringhe di *wide characters* (`wchar_t`) e che richiede l'inclusione di `<wchar.h>`].

9 - Consistenza della Memoria e Riordinamento delle Istruzioni

L'architettura classica dell'elaboratore e le ottimizzazioni dei linguaggi partono da un assunto fondamentale: il flusso di esecuzione è sequenziale e isolato. In scenari che coinvolgono sistemi multiprocessore, programmazione concorrente (thread) o interazione diretta con l'hardware (I/O memory-mapped), questa illusione si spezza. L'ordine in cui il codice sorgente è scritto non corrisponde necessariamente all'ordine in cui gli accessi alla memoria avvengono fisicamente o sono visibili al resto del sistema.

Queste discrepanze, fonte di insidiosi bug non deterministici, scaturiscono da due livelli di ottimizzazione.

9.1 Il Livello Software: Il Compilatore e **volatile**

Il compilatore (es. GCC) analizza il codice e, per massimizzare le performance, è autorizzato a stravolgere la sequenza di accesso alla RAM: può scambiare due letture indipendenti, può dedurre il risultato di un ciclo e sostituirlo con una costante, o può eliminare del tutto letture multiple di una stessa variabile caricandone il valore in un registro.

Se il nostro codice sta dialogando in memoria condivisa con un altro thread, o sta leggendo da un indirizzo hardware di una periferica (dove ogni accesso in lettura causa un "effetto collaterale", come lo svuotamento di un buffer di ricezione sulla seriale), l'ottimizzazione distruggerà la logica operativa. L'istruzione standard in C per limitare questa azione è il qualificatore **volatile**. Dichiarare una variabile (o un puntatore) **volatile** notifica al compilatore che il valore può cambiare esternamente, e che quindi **ogni** accesso in lettura o scrittura deve tradursi in un'effettiva transazione fisica sulla memoria, esattamente nel punto e per il numero di iterazioni prescritte dal sorgente, senza caching in registro locale.

[Tuttavia, **volatile** garantisce che il compilatore non rimuova gli accessi, ma *non vieta* che questi vengano riordinati tra loro rispetto ad altre variabili non volatili. Per forzare il compilatore a non alterare la sequenza logica del codice attorno a un punto critico, si ricorre alle **Barriere del Compilatore** (Compiler Barriers), solitamente implementate tramite costrutti inline-assembly opachi che indicano artificialmente che la memoria globale potrebbe subire mutazioni (es. `asm volatile("" : : : "memory")`)).

9.2 Il Livello Hardware: Store Buffer, Cache e Invalidazione

Persino risolvendo le ottimizzazioni del compilatore, è l'hardware stesso a riordinare le istruzioni internamente. I processori moderni superano l'estrema lentezza della RAM inserendo moduli logici ad altissime prestazioni: le *Cache* (L1, L2) e gli *Store Buffer*.

Quando un Core (P1) esegue un'istruzione di scrittura su una variabile *A*, attendere che l'operazione si propaghi ai vari livelli di cache comporterebbe un inutile rallentamento. Pertanto, la scrittura viene "parcheggiata" in un micro-registro ad alta velocità denominato **Store Buffer**. P1 considera l'operazione conclusa e procede, delegando allo Store Buffer l'incombenza di notificare il resto del sistema asincronamente. Contestualmente, per poter scrivere, P1 deve richiedere in rete (bus) il controllo esclusivo del blocco di memoria contenente *A*, inoltrando ai restanti processori un comando di Invalidazione per distruggere le copie locali.

La rottura della consistenza: Se P1 scrive su *A* (salvato nello store buffer) e subito dopo scrive su un flag di sblocco *B* (che P2 sta attendendo), e l'aggiornamento di *B* si propaga in rete prima dell'invalidazione di *A*, il processore P2 rileverà il flag *B* scattato, leggerà il contenuto di *A* dalla *sua* cache non ancora invalidata, ed eseguirà codice errato con un dato obsoleto. L'ordine spaziale garantito dal codice è stato invertito temporalmente dall'asincronia dell'hardware.

La risoluzione di questi complessi disallineamenti tra memorie cache distinte (che mina alla base gli algoritmi di sincronizzazione lock-free a basso livello) necessita l'introduzione dei *Modelli di Consistenza della Memoria* e di istruzioni hardware dedicate denominate *Memory Barrier* (Barriere di Memoria), tematiche avanzate cruciali nello sviluppo di sistemi multiprocessore concorrenti.

9.2.1 Concorrenza Hardware

La coerenza dei dati in cache può richiedere l'implementazione hw di linee di invalidazione (si invalida l'intera linea dov'è presente il dato e non la singola cella) che vengono attivate o meno se il dato è ancora valido (aggiornato con quello in memoria) o se è stato modificato da un altro processore e diventa "datato". Le invalidazioni da fare vengono inserite in una coda che viene svuotata ogni volta che il processore è libero. Si può avere che una linea venga segnalata come inconsistente, accodata e letta prima che l'inconsistenza si traduca nell'attivazione dell'apposita linea. In quel caso il processore è convinto di aver letto un dato ancora valido

Stati MESI Sono necessari 4 stati per indicare le varie possibilità di una cella condivisa e non solo più due per la validità.

- **M:** modificato, usato solo dal processore che esegue la modifica, gli altri mettono I
- **E:** esclusivo, il dato è presente solo nella cache corrente (memoria non condivisa), è necessario per evitare propagazione di messaggi se il dato viene modificato
- **S:** condiviso e il dato è in questo momento uguale a quello in memoria
- **I:** invalidato

Esecuzione Speculativa e Prefetching: I processori pre-caricano i dati prima che servano (*prefetching*) e, in presenza di diramazioni condizionali (es. un `if`), iniziano a eseguire contemporaneamente entrambi i rami in maniera speculativa (*speculative execution*). Gli accessi alla memoria generati dal ramo scartato avvengono fisicamente nell'hardware (modificando lo stato della cache), alterando l'ordine logico degli eventi e aprendo le porte a vulnerabilità di tipo *side-channel* (come Spectre o Meltdown).

9.3 I Modelli di Consistenza della Memoria

Per poter ragionare sulla correttezza dei programmi paralleli, si definiscono tre ordini logici:

- **Program Order:** L'ordine delle istruzioni così come scritte nel codice sorgente.
- **Execution Order:** L'ordine con cui il processore esegue fisicamente gli accessi in memoria.
- **Perceived Order:** L'ordine con cui *gli altri* processori osservano i cambiamenti effettuati.

I **Modelli di Consistenza della Memoria** sono insiemi di regole teoriche che restringono l'universo di tutti i possibili riordinamenti percepiti. Un modello definisce quali sequenze di eventi sono da considerarsi "valide" per l'hardware in uso.

Un modello **A** si definisce **più forte (più restrittivo)** di un modello **B** se tutte le esecuzioni valide per **A** lo sono anche per **B**, ma non viceversa (ossia **A** limita maggiormente i riordinamenti). Se i due modelli ammettono set di esecuzioni disgiunti, si dicono *non confrontabili*.

Una **scrittura** si definisce come eseguita dal processore *i* rispetto al processore *k* quando esso tenta di leggere la stessa locazione e viene restituito il valore scritto da *i*.

Una **lettura** del processore *i* si definisce come eseguita rispetto al processore *k* quando la scrittura del processo *k* non influenza il valore letto da *i*.

Si definiscono **lettura** e **scrittura** eseguite globalmente se le precedenti definizioni sono eseguite rispetto a tutti gli altri processori e non sul singolo *k*.

9.3.1 Consistenza Locale (Local Consistency)

È il modello più debole (ovvero il più permissivo) implementato nell'hardware reale.

Regola: Ogni singolo processore vede gli accessi che lui stesso compie nello stesso ordine specificato dal *Program Order*.

Non impone alcun vincolo su come i processori percepiscono l'ordine degli accessi effettuati da *altri* processori. In questo modello, algoritmi di sincronizzazione basilari (come l'uso di un flag per segnalare il completamento di un task) falliscono in modo catastrofico, perché un processore in attesa potrebbe "vedere" l'innescò del flag *prima* della scrittura dei dati correlati. Non si può usare per ambienti paralleli.

9.3.2 Consistenza Sequenziale (Sequential Consistency - SC)

È il modello più forte, restrittivo e intuitivo per i programmatori.

Esiste un **unico ordine globale** degli accessi alla memoria, su cui tutti i processori concordano. Inoltre, le operazioni di ogni singolo processore appaiono in questo ordine globale in accordo con il loro *Program Order*.

Vieta implicitamente l'esecuzione fuori ordine (*out-of-order execution*) e impone che ogni scrittura attenda acknowledgment globale prima di procedere, annullando di fatto i benefici prestazionali di Store Buffer e Cache.

9.3.3 Consistenza Causale (Causal Consistency)

Introduce il concetto di causalità (dipendenza) tra gli accessi. Due eventi sono causalmente correlati se il risultato dell'uno dipende inequivocabilmente dall'altro (es. P2 legge un valore scritto in precedenza da P1 su una variabile).

Regola: Tutti i processori del sistema devono concordare sull'ordine di quegli accessi che sono legati da una **relazione causale**.

Gli accessi che non sono causalmente correlati (concorrenti o su variabili indipendenti) possono essere visti in ordini differenti dai vari processori.

[La Consistenza Causale è strettamente più debole della Sequenziale, ma più forte della Locale].

9.3.4 Consistenza P-RAM (Pipelined RAM)

Regola: Le **scritture** eseguite da un singolo processore sono viste da *tutti* gli altri processori nell'ordine in cui sono state emesse.

Non vi è alcun vincolo sull'ordinamento relativo delle scritture effettuate da processori diversi. P2 potrebbe vedere prima una scrittura di P1 e poi una di P3, mentre P4 potrebbe percepire la scrittura di P3 prima di quella di P1.

9.3.5 Consistenza di Cache (Cache Consistency)

Regola: *Tutti* i processori concordano sull'ordine delle scritture effettuate su una **singola, identica variabile**.

Operazioni su variabili diverse possono essere osservate in ordini del tutto casuali.

[Questa consistenza è il risultato diretto dei protocolli hardware di Cache Coherence (come MESI), in cui l'invalidazione di una linea di cache forza una visione ordinata per quella singola cella di memoria. La Consistenza di Cache non è direttamente confrontabile con la Consistenza Causale].

9.3.6 Consistenza di Processore (Processor Consistency)

Regola: Il sistema deve rispettare contemporaneamente le regole della consistenza **P-RAM** e le regole della consistenza di **Cache**.

Le esecuzioni valide in questo modello appartengono all'intersezione matematica delle esecuzioni valide nei due modelli generatori. L'adozione di questo paradigma è determinante nell'ingegneria dei sistemi poiché abilita lo sviluppo di CPU ad altissime prestazioni. Se la

Consistenza Sequenziale pura imporrebbe rallentamenti insostenibili, la Consistenza di Processore permette all'hardware di sfruttare le ottimizzazioni architetturali mantenendo al contempo un modello logico comprensibile per chi sviluppa software di sistema.

Sotto questo modello, qualora due distinti processori operino su **variabili differenti** (es. P1 esegue **A=1** e P2 esegue **B=1**), è ammesso che terzi processori osservino tali mutazioni in sequenze diverse (P3 potrebbe rilevare **A** seguita da **B**, mentre P4 rileva **B** prima di **A**). Le regole di ordinamento rigido sono circoscritte esclusivamente alle dipendenze interne del core o agli accessi conflittuali sullo stesso indirizzo RAM.

9.3.7 Slow Consistency (SC)

È un modello non molto utilizzato che prevede che tutti i processori vedano nello stesso ordine le scritture su ciascuna locazione eseguita dallo stesso processore. Le scritture vengono "accodate" in un buffer e non possono essere riordinate con scritture successive.

9.4 Modelli ibridi

I modelli ibridi nascono dalla necessità pratica di bilanciare l'alta velocità di calcolo e il rigore della coerenza dei dati. I progettisti si sono resi conto che non tutte le operazioni in memoria hanno bisogno dello stesso livello di severità:

- **Dati ordinari:** Quando un processore sta macinando calcoli su variabili locali o array, non è necessario che il resto del sistema veda immediatamente e in perfetto ordine ogni singolo bit modificato.
- **Operazioni critiche:** Quando i processori si scambiano segnali (come semafori, lock o flag di completamento), l'ordine deve essere rigoroso e inequivocabile per evitare crash o corruzione dei dati.

I modelli ibridi mescolano quindi le regole: permettono comportamenti "rilassati" e fuori ordine per i dati normali, ma impongono colli di bottiglia stringenti (spesso basati sulla consistenza sequenziale) solo quando si interagisce con speciali **variabili di sincronizzazione**.

9.4.1 Il Modello di Consistenza "Weak" (Debole)

Ottimizza le prestazioni dividendo le variabili in due categorie: *variabili di dati* e *variabili di sincronizzazione*.

Le tre regole sono:

1. **Sincronizzazione forte:** Gli accessi alle variabili di sincronizzazione sono sequenzialmente coerenti. Tutti i processori del sistema le vedono mutare esattamente nello stesso ordine.
2. **Flushing in uscita:** Prima che un processore possa accedere a una variabile di sincronizzazione, tutte le sue precedenti operazioni sui dati devono essere completate e rese visibili globalmente (la cache viene riversata nella memoria principale).

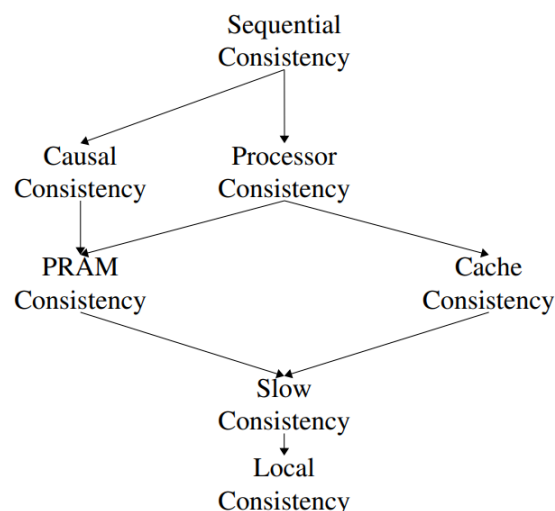
3. **Flushing in entrata:** Prima che un processore possa accedere a un qualsiasi dato ordinario (lettura/scrittura), tutte le precedenti operazioni di sincronizzazione devono essere state completate.

9.4.2 Release consistency

Si introducono due tipi di accesso di sincronizzazione chiamati **acquire** e **release**. La prima è una barriera semipermanente che impedisce alle istruzioni successive di essere riordinate con quelle precedenti alla acquire. La release ha la funzione inversa, impedisce che le istruzioni precedenti vengano riordinate con le successive.

9.4.3 Entry consistency

Molto simile al modello release ma ogni variabile condivisa è associata a una variabile di sincronizzazione quindi ciascun accesso fatto a tale variabile diventa analogo ad una acquire o a una release.



9.5 Barriere del Compilatore

Poiché l'hardware reale implementa modelli deboli per favorire le prestazioni, il programmatore deve inserire manualmente dei punti di sincronizzazione per impedire i riordinamenti critici.

La **Barriera del Compilatore** è una direttiva (spesso un costrutto in assembly *inline* opaco) che segnala al compilatore che l'intera memoria potrebbe subire modifiche asincrone.

Il compilatore viene costretto a terminare e a memorizzare i risultati di tutte le istruzioni precedenti alla barriera prima di procedere con l'emissione del codice macchina per le istruzioni successive. Questo garantisce il rispetto del *Program Order* a livello software (escludendo eventuali futuri riordinamenti operati a livello hardware dalla CPU).

9.5.1 Sincronizzazione Software: Le Barriere di Memoria (Memory Fences)

[Poiché la rimozione di queste strutture penalizzerebbe le performance, l'hardware affida al programmatore l'onere di inserire punti di sincronizzazione espliciti laddove la coerenza dei dati sia vitale per la logica algoritmica].

Le **Barriere di Memoria** agiscono a livello dei circuiti integrati del processore. Nei kernel (come Linux), queste istruzioni sono tipicamente astratte tramite macro C dedicate:

Barriera di Scrittura (`smp_wmb` - Write Memory Barrier) Questa istruzione agisce sullo **Store Buffer** per coordinare le operazioni in uscita.

Incontrando questa barriera, il processore riceve l'ordine di non rendere pubbliche le scritture successive finché non ha garantito che ogni operazione antecedente (nello Store Buffer) sia stata propagata o ordinata in modo definitivo.

Risulta indispensabile quando una serie di scritture (es. il setup di un oggetto seguito dall'impostazione di `ready = 1`) debba essere percepita dal sistema con una sequenzialità temporale rigorosa.

Barriera di Lettura (`smp_rmb` - Read Memory Barrier) Mira a stabilizzare l'**Invalidate Queue** per garantire letture coerenti.

Al sopraggiungere della barriera, la CPU congela le letture a seguire finché non ha processato e applicato *ogni* notifica di invalidazione ancora pendente nella coda.

Si inserisce tipicamente dopo la verifica di un flag (es. se `ready == 1`). Assicura che le letture dei dati correlati avvengano prelevando i valori aggiornati dalla memoria globale, neutralizzando il rischio di accesso a dati vecchi presenti in cache locale.

Barriera Completa (`smp_mb` - Full Memory Barrier)

[Si configura come lo strumento di sincronizzazione più pesante e costoso]. Integra le funzioni delle barriere di lettura e scrittura: impone lo svuotamento integrale dello Store Buffer e lo smaltimento totale dell'**Invalidate Queue**.

Questa direttiva inibisce ogni forma di *out-of-order execution* nel punto di chiamata. Nessuna operazione successiva può essere iniziata finché ogni istruzione precedente non è stata definitivamente consolidata e resa visibile all'intero sistema.

9.5.2 Esempi di architetture

Architettura Alpha

L'architettura DEC Alpha è storicamente nota per possedere uno dei modelli di consistenza della memoria più deboli e permissivi mai realizzati. Di conseguenza, richiede un uso estensivo ed esplicito di barriere per garantire la coerenza.

- **Barriera Completa (`smp_mb`) e in Lettura (`smp_rmb`):** Entrambe vengono implementate con la rigorosa istruzione `mb` (Memory Barrier), che blocca ogni riordinamento sia in lettura che in scrittura.
- **Barriera in Scrittura (`smp_wmb`):** Utilizza l'istruzione `wmb` (Write Memory Barrier), che agisce in maniera ottimizzata esclusivamente sull'ordine delle scritture (gestendo lo Store Buffer).
- **Barriera di Dipendenza Dati (`smp_read_barrier_depends`):** [L'Alpha è l'unica architettura mainstream che non rispetta intrinsecamente le dipendenze di dato nei sistemi multi-processore (se si legge un puntatore e poi si dereferenzia per leggere il valore puntato, l'hardware Alpha potrebbe tentare di riordinare le due letture)]. Per questo motivo, solo su Alpha questa macro viene tradotta in un'istruzione hardware reale (tramite `mb` o varianti dedicate), mentre sulle altre architetture è considerata superflua.

Architettura ARMv7

Nei dispositivi mobili ed embedded, l'architettura ARM adotta un modello *Weak Memory Ordering*, privilegiando consumi ridotti e parallelismo spinto. Le istruzioni di barriera (raffinate specificamente dalla versione v7) sono essenziali per la sincronizzazione hardware:

- **Barriera Completa e in Lettura (`smp_mb`, `smp_rmb`):** Mappate tramite l'istruzione `dmb` (Data Memory Barrier, spesso con dominio `sh` - shareable) o `dsb` (Data Synchronization Barrier). Impediscono fisicamente l'esecuzione di accessi successivi fino al completamento globale di quelli precedenti.
- **Barriera in Scrittura (`smp_wmb`):** Viene mappata sull'istruzione `dmb st` (Store) o `dsb st`, che garantisce unicamente l'allineamento e il completamento delle operazioni di scrittura.
- **Dipendenze di Dato:** L'hardware ARMv7 rispetta le dipendenze di dato in modo nativo; pertanto, la macro `smp_read_barrier_depends` si risolve in un'operazione nulla (*nothing*).

Architettura MIPS32

I processori MIPS, pionieri dell'approccio classico RISC, presentano un modello di gestione delle barriere molto più semplificato ma estremamente conservativo rispetto ad ARM e Alpha.

- **Barriere Universali (`smp_mb`, `smp_rmb`, `smp_wmb`):** Indipendentemente dal fatto che la richiesta del kernel sia una barriera completa, di sola lettura o di sola scrittura, l'architettura MIPS32 risolve l'esigenza con un'unica istruzione assembly: `sync` (talvolta trascritta `synch`). Questa istruzione forza il processore ad attendere il completamento di tutte le scritture in sospeso e svuota le code di memoria, comportandosi di fatto sempre come una costosa barriera completa.
- **Dipendenze di Dato:** Analogamente all'ARM, il MIPS rispetta nativamente le dipendenze, rendendo la macro di dipendenza un'operazione vuota (*nothing*).

Architettura IA-32 (Intel x86)

L'architettura IA-32 (e la sua evoluzione x86-64) implementa un modello di consistenza noto

come *Total Store Order* (TSO), ovvero una consistenza di processore fortemente ordinata. In questo modello le letture non vengono mai riordinate rispetto ad altre letture, e le scritture mai rispetto ad altre scritture. [Il solo riordinamento critico permesso nativamente dall'hardware x86 è una lettura successiva che "sorpassa" una scrittura precedente non ancora consolidata]. Data la severità nativa dell'hardware, le barriere esplicite sono meno frequenti:

- **Barriera Completa (`smp_mb`):** Utilizza l'istruzione hardware dedicata `mfence` (Memory Fence), che serializza completamente il flusso bidirezionale verso la memoria.
- **Barriera in Lettura (`smp_rmb`):** Mappata sull'istruzione `lfence` (Load Fence). Tuttavia, poiché le letture sono già fortemente ordinate dal TSO, in molti contesti il kernel la degrada a una semplice barriera software per inibire il solo compilatore (`barrier()`).
- **Barriera in Scrittura (`smp_wmb`):** Mappata sull'istruzione `sfence` (Store Fence). Anche in questo caso, potendo contare sull'ordinamento nativo dell'x86, questa istruzione è spesso rimpiazzata da un generico `barrier()`.
- **Dipendenze di Dato:** Come per ARM e MIPS, l'architettura x86 garantisce la coerenza sulle dipendenze, risolvendo la macro in *nothing*.

10 - Sincronizzazione

10.1 Problemi Fondamentali della Concorrenza

Quando più thread o processi vengono eseguiti in modo concorrente o parallelo dividendo lo stesso spazio di indirizzamento o le stesse risorse, l'assenza di coordinamento genera tre criticità sistemiche insidiose:

- **Race Condition (Condizione di Competizione):** Si verifica quando il risultato finale dell'esecuzione dipende strettamente dall'ordine temporale non deterministico con cui i singoli flussi eseguono le proprie istruzioni in memoria.
- **Starvation (Inedia):** Condizione in cui un task pronto per l'esecuzione viene indefinitamente penalizzato o ignorato dalle strategie di scheduling, rimanendo in attesa perpetua di accedere a una risorsa. Uno dei casi
- **Deadlock (Stallo):** Una situazione di blocco circolare in cui due o più task rimangono sospesi poiché ciascuno è in attesa di una risorsa esclusiva attualmente detenuta dall'altro.

10.2 Il Problema della Sezione Critica

Si definisce **Sezione Critica** quella specifica porzione di codice all'interno di un programma in cui si effettua l'accesso, la modifica o la scrittura su una risorsa condivisa (variabili globali, tabelle, file descriptor). Per garantire la consistenza dei dati, qualsiasi soluzione software o hardware al problema della sezione critica deve soddisfare contemporaneamente tre requisiti formali:

1. **Mutua Esclusione (Mutual Exclusion):** Se un task è in esecuzione all'interno della propria sezione critica, a nessun altro task deve essere consentito l'accesso alla medesima sezione critica o a risorse correlate.
2. **Progresso (Progress):** Se nessun task si trova nella sezione critica e vi sono flussi che richiedono di entrarvi, la scelta di quale task possa accedervi non può essere rimandata indefinitamente. Solo i task che stanno attivamente richiedendo l'accesso possono partecipare alla selezione.
3. **Attesa Limitata (Bounded Waiting):** Deve esistere un limite strutturale al numero di volte in cui a un task viene negato l'accesso alla sezione critica a favore di altri richiedenti, prevenendo così scenari di starvation.

[L'architettura logica di un programma concorrente standard prevede che la sezione critica sia preceduta da una sezione di ingresso (Entry Section) volta ad acquisire i diritti di accesso, e seguita da una sezione di uscita (Exit Section) deputata al rilascio della risorsa, prima di ritornare alla sezione rimanente (Remainder Section)].

10.3 Supporto Hardware di Basso Livello

I processori moderni mettono a disposizione istruzioni atomiche indivisibili basate sul ciclo di lettura-modifica-scrittura blindato sul bus di sistema. Esistono le varianti di lettura e scrittura atomica, ma richiedono numerosi accessi e risulta inefficiente.

Le due varianti fondamentali sono:

- **Test-and-Set:** Un'istruzione che legge un flag di stato in memoria, ne restituisce il valore attuale e contemporaneamente lo imposta a `true` in un singolo ciclo hardware.
- **Compare-and-Swap (CAS):** confronta il contenuto di una cella di memoria con un valore atteso e, solo in caso di coincidenza, vi scrive un nuovo valore.

Queste primitive hardware fungono da mattoni fondamentali per l'edificazione dei lock software esclusivi.

[Esiste la variante di **Load-link** e **Store-Block** che vengono usati in ingresso e in uscita della sezione critica. Sono "associati" e nel momento in cui una lettura/scrittura viene eseguita in contemporanea a un altro processo, il primo dei due che esegue la **Store-Block** invalida l'altra istruzione che deve essere rieseguita. È un meccanismo del tipo "qualcuno si è messo in mezzo? Se sì, si deve ripetere"]

10.4 Primitive di Sincronizzazione di Basso Livello

Sfruttando le istruzioni atomiche, il software implementa lock esclusivi per isolare le sezioni critiche. A basso livello si distinguono due filosofie di gestione dell'attesa qualora il lock sia occupato:

10.4.1 Lo Spinlock (Lock ad Attesa Attiva)

Lo spinlock obbliga il task escluso a eseguire un ciclo continuo e infinito di verifica (*polling* o *busy-waiting*) controllando atomicamente lo stato della variabile di lock finché questa non viene rilasciata dal detentore.

- **Vantaggi:** Elimina totalmente l'overhead computazionale legato alla sospensione del thread e al successivo switch di contesto.
- **Svantaggi:** Consuma attivamente il 100% dei cicli della CPU durante l'attesa, surriscaldando il core e sottraendo potenza di calcolo utile ad altri processi.
- **Ambito d'uso:** È sostenibile esclusivamente in sistemi multiprocessore per proteggere sezioni critiche estremamente brevi (poche istruzioni) o all'interno del Kernel del sistema operativo dove la sospensione del codice (sleep) è vietata [ad esempio nei gestori di interrupt o nelle routine dello scheduler].

Spinlock con ticket Si basa sul concetto dell'algoritmo del panettiere in cui ogni persona (task) che arriva e vuole usufruire del servizio (risorsa condivisa) prende il numero/ticket e si mette in coda.

Il task che arriva prende automaticamente un ticket che viene aggiornato globalmente (l'incremento è una sezione critica), il numero corrente è l'indicatore generale di quale task

viene eseguito e qual è il successivo. Quando un task ha finito incrementa (sezione critica) il contatore indicante chi ha diritto alla risorsa.

Spinlock con array Ha le stesse basi dell'algoritmo con i ticket, ma al posto di avere una variabile globale su cui fare polling, si usa un vettore di dimensione N (numero massimo di task in coda) e ognuno ha una cella assegnata in cui fare polling. Volendo una struttura riusabile, una volta che un task ha finito l'esecuzione della sezione critica mette come bloccato il proprio posto e sblocca il successivo. Facendo così il task che sta eseguendo polling sulla cella $m+1$ è libero di procedere e chiunque si trovi con la cella m assegnata viene sospeso in quanto il lock è bloccato.

10.4.2 Lock Lettori-Scrittori (Reader-Writer Locks)

I lock esclusivi, pur sicuri, limitano la concorrenza anche quando non strettamente necessario. In molti scenari applicativi, la maggior parte dei task esegue operazioni di sola lettura, mentre le modifiche (scritture) sono sporadiche. Poiché la lettura concorrente di una risorsa non altera la consistenza dei dati, l'uso di un lock esclusivo penalizzerebbe ingiustificatamente le prestazioni.

I **Lock Lettori-Scrittori (RW Locks)** risolvono questa inefficienza specializzando i task in due categorie:

- **Lettori (Readers):** Molteplici thread possono acquisire simultaneamente il lock in modalità condivisa, a patto che non vi siano scrittori attivi.
- **Scrittori (Writers):** Un thread che necessita di modificare i dati deve acquisire l'accesso in modalità esclusiva. Nessun altro lettore o scrittore può accedere alla risorsa durante questa fase.

[L'implementazione dei lock lettori-scrittori deve tuttavia gestire il rischio di starvation (inedia): se un flusso continuo di lettori mantiene il lock occupato, uno scrittore in attesa potrebbe rimanere bloccato indefinitamente, degradando la reattività del sistema].

10.4.3 Lock Sequenziali (Sequential Locks / Seqlocks)

Per superare il problema della starvation degli scrittori nei lock lettori-scrittori tradizionali, i kernel moderni (come Linux) introducono i **Lock Sequenziali (Seqlocks)**. Il principio cardine di questa primitiva è dare massima priorità agli scrittori, impedendo che i lettori possano bloccarli.

Il funzionamento si basa su un contatore sequenziale intero gestito dal kernel:

1. **Lo Scrittore:** Incrementa il contatore all'inizio della scrittura (rendendolo dispari), modifica i dati e incrementa nuovamente il contatore al termine (rendendolo pari). Lo scrittore non attende mai i lettori.
2. **Il Lettore:** Non acquisisce alcun lock reale e non modifica lo stato del sistema. Eseguce invece una **lettura speculativa**: legge il contatore prima di accedere ai dati e lo rilegge subito dopo. Se il contatore è pari e non è cambiato tra le due letture, la lettura è valida. Se il contatore risulta dispari o modificato, significa che uno scrittore

ha interrotto il processo: il lettore scarta i dati obsoleti e reitera il ciclo finché non ottiene una lettura consistente. Se il lettore cerca di accedere a una sezione critica con uno scrittore attualmente all'interno, la funzione di lettura diventa bloccante. Si deve attendere che il writer esca dalla sezione per poter proseguire.

10.5 Strutture Dati Lock-Free (Cenni)

Un approccio radicalmente alternativo alla sincronizzazione basata su lock (bloccanti o speculativi) è la progettazione di **Strutture Dati Lock-Free**. In questo paradigma, l'accesso concorrente viene gestito interamente tramite combinazioni avanzate di istruzioni hardware [come Compare-And-Swap], eliminando del tutto il concetto di "blocco".

[Sebbene queste strutture offrano un parallelismo teoricamente perfetto e immunità ai deadlock, la loro progettazione è caratterizzata da una complessità matematica estrema. Piccoli errori logici nella gestione della memoria concorrente possono introdurre vulnerabilità distruttive. Di conseguenza, nella prassi ingegneristica, le strutture lock-free vengono implementate solo per componenti software rigidamente definiti e ed esistenti, sconsigliandone lo sviluppo ad-hoc].

10.6 Granularità dei Lock nei Sistemi Database

Il livello di astrazione e la complessità delle primitive di sincronizzazione scalano proporzionalmente alla dimensione dei dati gestiti. Mentre a livello di sistema operativo sono sufficienti lock a due o tre stati, l'architettura dei sistemi di database (in particolare i database distribuiti e ad alta concorrenza che servono un numero enorme di utenti simultanei) richiede una granularità molto più spinta.

Un database gestisce una gerarchia di risorse: l'intero sistema, le tabelle, i singoli blocchi, i record e finanche i singoli campi di un record. Se un processo deve aggiornare un singolo campo, bloccare l'intera tabella congelerebbe inutilmente migliaia di utenti paralleli.

Per massimizzare il parallelismo e distinguere accuratamente chi vuole scrivere e dove vuole scrivere, i database implementano lock ad **altissima granularità** dotati di complessi motori a stati (che arrivano a gestire anche trenta stati differenti per singolo lock). Questi stati permettono a task differenti di toccare record separati (o persino campi diversi dello stesso record, come il campo A e il campo C) contemporaneamente, coordinandoli in maniera del tutto coperta e non distruttiva.

10.7 Strategie di Lock

Esistono diversi approcci per gestire il locking:

- **lock gigante:** L'intero codice, come una libreria, è protetto da un singolo lock. Sicuramente è l'approccio più semplice e permette di portare codice non parallelo in ambienti che lo sono; il difetto principale è la perdita dell'eventuale parallelismo presente.
- **lock a grana grossa:** Il codice è diviso in sottosistemi indipendenti come i vari moduli del sistema operativo (fs, gestione della memoria, scheduler etc) e ognuno è

protetto da un lock dedicato. Questo aumenta di molto il parallelismo, ma la chiamata tra sottosistemi richiede comunque un lock globale

- **lock a grana fine:** Si va a lockare ogni struttura dati andando ad aumentare esponenzialmente il parallelismo come la complessità di gestione di tutti i lock. Si introducono delle regole apposite come la gerarchia di lock. Questa regola prevede di acquisire i lock in un determinato ordine in base al loro livello della gerarchia.

10.8 Problemi di locking

Sebbene i lock siano indispensabili per garantire la mutua esclusione, il loro utilizzo espone il sistema a criticità architetturali spesso non deterministiche e complesse da diagnosticare (bug di sincronizzazione).

10.8.1 Il Deadlock

Il Deadlock (Stallo) si verifica tipicamente in presenza di una **dipendenza circolare** tra task nell'acquisizione di risorse (lock). *Esempio classico:* Un task **T1** acquisisce il lock **L1**. Un task **T2** acquisisce il lock **L2**. Successivamente, **T1** tenta di acquisire **L2** e si blocca (essendo detenuto da **T2**). Parallelamente, **T2** tenta di acquisire **L1** e si blocca (essendo detenuto da **T1**). Nessuno dei due task può rilasciare il lock posseduto finché non ottiene il secondo, generando un blocco perenne.

La soluzione standard consiste nell'imporre una **gerarchia stretta di acquisizione**. Se l'architettura stabilisce che i lock devono essere sempre acquisiti in un ordine prefissato (es. prima **L1**, poi **L2**), la dipendenza circolare diviene impossibile. Un task che detiene **L2** e necessita di **L1** dovrà forzatamente rilasciare **L2**, acquisire **L1** e tentare di riacquisire **L2**.

10.8.2 L'Inversione di Priorità (Priority Inversion)

Nei sistemi real-time o che adottano scheduling a priorità rigida, l'uso incauto dei lock può portare al fenomeno dell'Inversione di Priorità, in cui un task ad alta priorità viene di fatto "rallentato" a favore di task a priorità inferiore. Si verifica tipicamente in presenza di tre task: **TH** (Priorità Alta), **TM** (Media) e **TL** (Bassa).

1. Il task **TL** viene mandato in esecuzione e acquisisce un lock **L**.
2. Il task **TH** diventa pronto e viene eseguito subentrando a **TL**. **TH** tenta di acquisire il lock **L**, ma lo trova occupato e si blocca in attesa di **TL**.
3. Un task **TM** diventa pronto. Poiché **TM** ha priorità superiore rispetto a **TL**, lo scheduler lo manda in esecuzione.
4. Il task **TL** (che detiene il lock di cui ha bisogno **TH**) non riceve tempo CPU perché soppiantato da **TM**. Di conseguenza, **TM** esegue ignorando **TH**, causando l'inversione logica delle priorità. [Questo bug celebre bloccò una delle prime sonde spaziali marziane (Pathfinder), richiedendo una riprogrammazione in volo].

Soluzione:

- **Disabilitare la Preemption:** Il task che detiene un lock non può essere sospeso dal processore, assicurando che lo rilasci rapidamente.
- **Priority Ceiling (Tetto di Priorità):** Un task che acquisisce un lock eleva immediatamente e artificialmente la propria priorità al massimo livello possibile (o al livello del task a priorità più alta del sistema), garantendosi esecuzione ininterrotta contro i task intermedi.
- **Priority Inheritance (Ereditarietà della Priorità):** Il task a bassa priorità eredita dinamicamente e temporaneamente la priorità del task bloccato in attesa sul suo lock. Nell'esempio, **TL** erediterebbe la priorità di **TH**, impedendo a **TM** di interromperlo.

10.8.3 Lock ed Esecuzione in Spazio Interruzione (Interrupt Handlers)

[Come noto, gli handler associati agli Interrupt Hardware o ai Segnali (in User Space) interrompono asincronamente il flusso di esecuzione normale. L'utilizzo di un lock condiviso tra il codice principale e la routine dell'handler è estremamente rischioso.] Se il codice principale acquisisce il lock e, subito dopo, un interrupt scatena l'esecuzione dell'handler, quest'ultimo tenterà di acquisire il medesimo lock. Trovandolo occupato, l'handler attenderà all'infinito. Poiché il flusso principale non riprenderà esecuzione finché l'handler non terminerà, il lock non sarà mai rilasciato, causando il blocco irreversibile dell'intero processo.

Quando una variabile è condivisa tra codice asincrono (handler) e codice ordinario, quest'ultimo, prima di acquisire il lock, deve esplicitamente disabilitare temporaneamente la ricezione di interrupt o segnali, riabilitandola solo dopo il rilascio.

10.8.4 Prestazioni e Contesa

[Si ribadisce che i lock basati su primitive atomiche comportano una spesa architetturale elevata]. L'overhead cresce esponenzialmente al crescere del numero di task in **contesa** (che competono per la medesima risorsa). Al momento dello sblocco, tutti i task in attesa subiscono cache-miss simultanei a causa dei messaggi hardware di invalidazione, saturando il bus di memoria.

Al fine di mitigare questo degrado (soprattutto su server altamente paralleli), si ricorre al **Lock a Grana Fine** (invece di un unico lock globale per un intero sottosistema, si associa un lock dedicato a ogni singola struttura dati) o a tecniche come il **Tournament Lock** (i task competono su lock intermedi ad albero per diluire il carico sul lock critico finale).

10.9 Primitive di Sincronizzazione di Alto Livello

Le primitive di basso livello (Spinlock), non sono sufficienti né ottimali quando i flussi di esecuzione necessitano di complesse dinamiche di coordinamento, come il blocco volontario in attesa di eventi prolungati. Tali esigenze impongono l'utilizzo di primitive che si interfacciano in modo nativo con lo **Scheduler** del Sistema Operativo, sollevando il programmatore dalla gestione manuale delle code e riducendo il consumo di CPU.

10.9.1 I Semafori

Concepiti da E. Dijkstra, i Semafori generalizzano il concetto di lock. Un semaforo è strutturalmente un numero intero **non-negativo** a cui è associata una coda di processi in attesa.

Il semaforo ammette due sole operazioni, rigorosamente atomiche (e tipicamente supportate internamente dallo scheduler tramite lock a basso livello o primitive dedicate):

1. **sem_wait (o P)**: Tenta di decrementare il valore del semaforo di un'unità. Se il valore preesistente era maggiore di zero, l'operazione ha successo e il task procede. Se il valore era zero o negativo, lo scheduler blocca il task e lo accoda alla struttura del semaforo (Stato *Blocked*).
2. **sem_signal (o V)**: Incrementa di un'unità il valore del semaforo. Se vi sono processi in attesa bloccati nella coda del semaforo, uno di essi viene risvegliato (riportato allo stato *Ready*) e gli viene "assegnato" il passaggio.

Nota: Un **Semaforo Strong (Forte)** garantisce il rispetto dell'ordine di arrivo FIFO (First-In First-Out) durante i risvegli, escludendo matematicamente la starvation. Un **Semaforo Weak (Debole)**, più semplice da implementare a livello di kernel, attinge casualmente dalla coda dei processi dormienti.

Il semaforo si definisce **Binario** qualora venga vincolato ad assumere unicamente i valori 0 o 1 (assumendo così un comportamento speculare al Mutex). A differenza del Mutex, tuttavia, l'operazione di **sem_signal** non deve essere obbligatoriamente invocata dal task che in precedenza ha eseguito la **sem_wait**.

10.9.2 Mutex

Il mutex modifica radicalmente l'approccio: se un thread tenta di acquisire il lock e lo trova occupato, il kernel interviene sospendendo il thread (stato *Blocked* o *Sleeping*), inserendolo in una coda di attesa legata a quel preciso mutex e riallocando immediatamente la CPU a un altro task pronto.

A differenza di un semaforo binario, solo chi ha acquisito il lock del mutex può eseguire una unlock. Esiste una variante definita **mutex ricorsivo** (o **rientrante**) nella quale un task può acquisire lo stesso mutex più volte prima di rilasciarlo e ovviamente il rilascio deve essere eseguito lo stesso numero di volte degli acquisti.

10.9.3 Le Variabili di Condizione (Condition Variables)

Le variabili di condizione permettono a un thread di arrestarsi volontariamente e cedere la CPU (dormire) in attesa che il programma raggiunga un certo "stato" o che una determinata espressione logica (*condizione*) assuma valore vero.

Ogni Variabile di Condizione (CV) opera strettamente accoppiata a un **Lock Esclusivo (Mutex)** (il quale protegge la lettura e modifica della condizione logica) e si controlla tramite due interfacce:

1. **cond_wait**: Invocata dal thread in attesa. La sua esecuzione effettua due azioni atomiche: rilascia il Mutex di protezione e fa dormire il thread, iscrivendolo nella coda interna della CV. [Quando il thread viene in seguito risvegliato, la funzione ritrova e acquisisce automaticamente il Mutex prima di restituire il controllo al codice].
2. **cond_notify / cond_signal**: Invocata dal thread che ha alterato lo stato o i dati condivisi. Si occupa di risvegliare un singolo task dormiente nella coda della CV. [Esiste inoltre la variante **cond_notify_all** (Broadcast), che risveglia simultaneamente l'intera platea di processi in attesa].

La Semantica Mesa e il Rischio di Risveglio Spurio

Nell'implementazione universale (*Semantica Mesa*), il risveglio di un task bloccato non garantisce a priori che egli ritrovi la condizione logica nel medesimo stato in cui il thread notificatore l'aveva predisposta. Nello scarto temporale tra il risveglio e l'effettiva riesecuzione sulla CPU, un terzo processo concorrente (a parità di priorità) potrebbe acquisire il lock e stravolgere nuovamente i dati. Ne consegue che la chiamata a **cond_wait** non va mai inserita all'interno di un'istruzione di controllo logico semplice (come un **if**), bensì all'interno di un ciclo iterativo (**while**). Questo obbliga il task risvegliato a rivalutare esplicitamente i dati prima di proseguire.

10.10 Pattern Architetture di Sincronizzazione

Le primitive analizzate permettono di risolvere le criticità implementando specifici schemi comportamentali (Pattern):

- **Mutua Esclusione (con Semafori)**: Qualora i Mutex non fossero disponibili, si impiega un semaforo preinizializzato al valore massimo di 1. La sezione critica viene inclusa a "sandwich" tra una **sem_wait()** in ingresso e una **sem_signal()** in uscita in tutti i processi con la sezione critica concorrente, questo fa sì che il primo task che arriva entri nella sezione critica.
- **Multiplexing (Limitatore di Risorse)**: Estensione della Mutua Esclusione impiegata per vincolare l'ingresso a non più di **K** task simultaneamente (es. connessioni massime al database). In tal caso, il semaforo viene inizializzato a valore K.
- **Segnalazione / Handshake (con Semafori)**: Un task B è costretto ad attendere l'elaborazione di A. Si impiega un semaforo inizializzato a 0. Il task B invoca la **wait()** (bloccandosi all'istante) , mentre il task A, ultimate le operazioni, invoca la **signal()** destando l'attendente.
- **Il Rendez-vous (Appuntamenti Singoli)**: Implementazione bilaterale in cui il Task A attende B, e contemporaneamente B attende A. Si utilizzano due semafori (diversi) preinizializzati a 0. Entrambi i task incrociano le istruzioni: inviano prima una **signal()** sul proprio semaforo e immediatamente dopo un blocco in **wait()** sul semaforo avversario.
- **La Barriera (Sincronizzatore Multiplo)**: È l'astrazione finale che unifica N esecuzioni in parallelo: una "linea tracciata sulla sabbia" in cui tutti i processi devono confluire (e bloccarsi) prima di potervi ripartire allineati. [Esistono implementazioni puramente hardware e barriere software supportate dal POSIX (**pthread_barrier_wait**)]. [Si costruiscono tramite contatori incrementati in modo atomico (via Mutex); quando il limite è raggiunto, il task terminale invoca istruzioni di sblocco a cascata o segnali di notifica *broadcast* per tutti i restanti]. La barriera deve

essere implementata in due fasi per permettere un riutilizzo di quest'ultima. Si procede con due semafori multiplex distinti che aspettano l'arrivo di tutti i task nella prima fase e solo l'ultimo sblocca il secondo (`sem_Wait(b.phase2)`) precedentemente inizializzato a 1. L'ultimo task inizia a risvegliare uno degli altri della fase uno che si risvegliano a ruota. L'ultimo task che arriva alla fase 2 porta a 0 `phase1` e mette una `sem_signal(b.phase2)` per riportarlo al valore 1

10.11 Condizioni di deadlock

1. Mutua esclusione: è impossibile da evitare in quanto necessaria per la sincronizzazione
2. No preemption: Difficile da evitare se non con l'implementazione di un rollback che risulta essere molto costoso. Si deve "fotografare" il task prima del deadlock e ristabilire la sua condizione a quando non aveva acquisito una risorsa condivisa.
3. Hold and Wait: Nel caso di più oggetti di sincronizzazione, quando viene acquisito il primo e il secondo risulta essere bloccato, ci si mette in attesa avendo comunque il lock del primo
4. Attesa circolare: un numero di task richiede in maniera circolare delle risorse e nel momento in cui tutti aspettano il successivo si incorre in deadlock.

Il deadlock risulta possibile se sono verificate le prime 3 condizioni, ci sono tre metodi di gestione del deadlock:

1. Prevenzione: si vuole prevenire qualsiasi possibilità di deadlock, la mutua esclusione non è evitabile in quanto elemento fondante della sincronizzazione.
 - Disabilitare Hold and Wait: tutte le risorse devono essere acquisite simultaneamente, pure se necessarie per un breve lasso temporale. questo risulta estremamente **inefficiente**.
 - Abilitare la preemption: necessaria l'implementazione di rollback molto costosi e non sempre implementabili in quanto alcune risorse potrebbero non essere ripristinabili. Si vuole che quando una richiesta viene rifiutata, il processo liberi tutte le risorse acquisite,
 - Disabilitare le attese circolari: Si deve definire una gerarchia delle risorse e si possono acquisire solo quelle di ordine inferiore. Se si necessita la risorsa 1 e 2 con relativo grado A e B ($A > B$), bisogna necessariamente acquisire prima 1 e poi 2 pure se 1 verrà usata solo in seguito.
2. Evitare deadlock: Si garantisce una risorsa solo quando la risorsa richiesta non porta a possibili deadlock. Il sistema operativo necessita di conoscere tutte le future richieste in termini di risorse nel caso peggiore.
3. Trovare i deadlock: quando un deadlock viene trovato si può procedere in modi diversi:
 - terminare tutti i processi con deadlock (approccio comune)
 - terminare un processo alla volta bloccato finché non si risolve il deadlock: l'ordine è determinante e il risultato non è detto che sia valido anche se si elimina il deadlock
 - rollback di tutti i processi con deadlock: è estremamente costoso in quanto si deve implementare un meccanismo di backup e di restore

Per il metodo 2 è stato predisposto l'algoritmo dei banchieri dove il sistema operativo tiene traccia di tutti i processi con le loro risorse usate e quelle ancora necessarie per eseguire il loro lavoro. Una richiesta si dice sicura solo quando esiste **almeno** un task che necessita meno risorse delle disponibili per terminare. (es. sono disponibili 7 risorse e il task ne ha bisogno ancora di 3 vuol dire che ci si trova in uno stato sicuro).

$\exists i \mid \forall j \text{ Needed}(i, j) \leq \text{Available}(j)$ con i processo e j risorsa.

11 – I/O Avanzato

11.1 Locking sui File (File Locking)

Un processo può richiedere al sistema operativo di applicare un "lock" su una specifica porzione di un file aperto (indicando offset di inizio e fine). Questo meccanismo, gestibile primariamente tramite la system call `fcntl` (o le varianti di libreria `lockf` e `flock`), si divide in due tipologie fondamentali:

1. **Lock Consigliato (Advisory Lock):** È lo standard POSIX ed è universalmente supportato. Il lock non blocca fisicamente l'accesso al file: esso si basa su una cooperazione volontaria. Un processo *ben scritto* proverà ad acquisire il lock prima di eseguire una `read` o `write`; se lo trova occupato, si fermerà. Tuttavia, se un processo malevolo o mal programmato esegue direttamente una `write` ignorando il lock, il sistema operativo glielo lascerà fare senza restrizioni.
2. **Lock Obbligatorio (Mandatory Lock):** Il blocco è imposto a basso livello dal kernel. Nessuna system call di I/O (da parte di alcun processo) andrà a buon fine su quella porzione di file finché il lock non viene rilasciato dal legittimo proprietario. [Sebbene semanticamente più sicuro, non fa parte dello standard Unix puro (è presente in Linux ma la sua affidabilità in ambienti complessi è spesso criticata), ed è pertanto sconsigliato in software altamente portabile].

11.2 I/O Bloccante e Non Bloccante

[Ogni interazione con il kernel tramite system call può astrattamente definirsi "bloccante" se priva di una garanzia sul tempo massimo di completamento].

- **I/O su File Regolari:** System call come `read` o `write` su file archiviati in storage di massa sono considerate **non bloccanti**. Anche se il disco è lento, il kernel garantisce che i dati verranno trasferiti in un tempo hardware deterministico.
- **I/O Bloccante per Design:** La lettura o scrittura su Terminali (in attesa di digitazione utente), Socket di Rete (in attesa di pacchetti dal mondo esterno) o Pipe (in attesa di dati da altri processi) è intrinsecamente **bloccante**. Se non vi sono dati, il processo viene sospeso dallo scheduler a tempo indefinito. Lo stesso accade per una system call `open` su una Named Pipe in cui non vi è alcun lettore dall'altro capo.

Forzare l'I/O Non Bloccante: In applicazioni ad alte prestazioni (o in interfacce grafiche che non possono "congelarsi"), si può richiedere al kernel di convertire queste chiamate in operazioni **non bloccanti**.

- Si può effettuare in fase di apertura, inserendo il flag `O_NONBLOCK` nella system call `open()`.
- Si può impostare dinamicamente a runtime (es. per lo standard input) utilizzando la funzione `fcntl()`, leggendo i flag attuali (`F_GETFL`) e riscrivendoli sommati a `O_NONBLOCK` (`F_SETFL`).

Il Comportamento: Se una `read` o una `write` non bloccante non trova dati (o non trova spazio nel buffer), la system call fallisce istantaneamente. Restituisce `-1` e imposta la variabile di errore `errno` sulla costante macro `EAGAIN` ("riprova più tardi"), permettendo al programma di gestire l'attesa senza sospendere l'intero processo.

11.3 Multiplexing dell'I/O (`select` e `poll`)

Quando un singolo processo (es. un server di rete o un terminale SSH) deve attendere e smistare dati provenienti da file descriptor contemporanei e indipendenti (es. tastiera locale e socket remota), l'utilizzo di chiamate bloccanti fermerebbe il processo sul primo canale in ascolto, ignorando completamente i dati in arrivo sul secondo. Ricorrere all'I/O non bloccante in un ciclo continuo per interrogare i canali (*polling*) causerebbe un consumo parassita estremo del processore (busy-waiting).

La soluzione architetturale è il **Multiplexing dell'I/O**: il processo si sospende su una *singola* system call centralizzata, delegando al kernel l'onere di risvegliarlo non appena *almeno uno* dei canali sorvegliati diventa pronto per l'accesso (ovvero quando una `read` o `write` su di esso non risulterebbe bloccante).

11.3.1 La funzione `select`

Storicamente è la prima funzione introdotta a questo scopo. Prende in input degli "insiemi" (strutture dati di tipo `fd_set` gestite come array di bit) contenenti i descrittori da monitorare in Lettura, Scrittura e per le Eccezioni.

- Accetta un parametro per impostare un **Timeout massimo** di attesa (fino al microsecondo tramite `struct timeval`).
- Se l'attesa scade, restituisce 0. Se fallisce (es. interrotta da un segnale) restituisce `-1` e imposta `errno` a `EINTR`. Se ha successo, restituisce il numero di file descriptor pronti.

La `select` modifica internamente e distruttivamente gli insiemi `fd_set` passati, obbligando il programmatore a doverli reinizializzare manualmente (tramite le macro `FD_ZERO`, `FD_SET`, ecc.) a ogni singolo ciclo di `while`. Inoltre, richiede come parametro il numero identificativo del file descriptor più alto da testare `+ 1`.

11.3.2 La funzione `poll` (e varianti)

Introdotta successivamente per colmare le limitazioni di `select`. Piuttosto che insiemi di bit, accetta un array di strutture specializzate (`struct pollfd`), permettendo di raggruppare i file descriptor che distano molto numericamente senza costringere il kernel a testare il vuoto intermedio.

- Ogni `struct pollfd` contiene il file descriptor (`fd`), gli eventi da testare preparati dal programmatore (campo `events`, es. `POLLIN` per lettura, `POLLOUT` per scrittura), e un campo `revents` (return events) che verrà popolato unicamente dal kernel al risveglio.
- Poiché il kernel altera solo il campo delle risposte senza distruggere i campi di configurazione, le strutture non devono essere ricreate ad ogni loop, efficientando il codice.
- Anche `poll` restituisce eventi d'eccezione utilissimi, come `POLLHUP` (quando il terminale o la connessione di rete dall'altra parte viene fisicamente interrotta).
- *[Esistono varianti come `pselect` e `ppoll` che permettono di passare una maschera di segnali da ignorare temporaneamente mentre il processo è in fase di sospensione].*

11.4 I/O Vettoriale (Scatter-Gather I/O)

Spesso, a livello logico di applicazione, i dati da inviare a un file sono allocati in variabili o array di memoria completamente scorporati e non contigui (es. l'header di un protocollo, il body e la firma crittografica). Analogamente in lettura.

Inizialmente si implementava tramite decine di system call write consecutive (con overhead gravoso) o istanziando un macro-buffer ausiliario per concatenare preventivamente tutti i dati (overhead di copia memoria e spreco di RAM). Come soluzione è nato l'**I/O vettoriale**.

Tramite le funzioni `readv` e `writev`, si passa al kernel un array di strutture `struct iovec`. Ogni struttura funge da puntatore a un frammento di RAM (indicando l'indirizzo base e la lunghezza).

- **Gather Write (`writev`):** Il kernel "raccolge" i frammenti dalle disparate locazioni di memoria in User Space e li "spara" nel canale sottostante in un singolo, fluido blocco logico, eseguendo un unico context switch (una sola system call).
- **Scatter Read (`readv`):** Al contrario, il kernel assorbe un blocco massivo dal disco e lo "sparpaglia" riversandolo in compartimenti stagni pre-determinati dal vettore di strutture. *[Per entrambe esiste la variante `preadv` e `pwritev`, che esegue l'operazione in maniera atomica partendo da uno specifico offset del file, preservando il file cursor globale (il cursore di lettura) in ambienti multi-thread].*

11.5 Memory Mapping (La Funzione `mmap`)

[Si tratta dell'astrazione più performante per manipolare file enormi]. Tramite la system call `mmap`, chiediamo al kernel di creare una corrispondenza diretta tra lo spazio di indirizzamento in memoria virtuale del nostro processo (User Space) e i blocchi del file sottostante. L'effetto pratico è che il file appare come un gigantesco array di byte allocato in RAM: non si invocano più `read` o `write`, si usano semplici puntatori C per accedere, leggere e alterare istantaneamente il contenuto.

- **Modalità Operative:**

- **MAP_SHARED:** Le scritture eseguite col puntatore in memoria RAM vengono automaticamente recepite e riversate nel file su disco dal kernel. Il file muta, e se altri processi stanno mappando il medesimo file, vedranno in tempo reale le alterazioni.
- **MAP_PRIVATE:** Se il processo tenta di alterare i dati, il kernel innesca una logica di *Copy-On-Write*. Altera una copia privata del dato senza minimamente inficiare il file originale sottostante.
- **MAP_ANONYMOUS:** [Fondamentale]. Viene richiesto un mapping slegato da qualsiasi file descriptor fisico. Restituisce nient'altro che una porzione di RAM azzerata. Questo è l'esatto meccanismo intrinseco con cui i sistemi operativi moderni implementano nativamente la funzione `malloc()` quando un programma esige tonnellate di heap memory dinamica.

Funzionalità di controllo connesse al Mapping:

- **munmap:** Destituisce il mapping e invalida i puntatori.
- **msync:** In un mapping di tipo **SHARED**, ordina al kernel di interrompere le ottimizzazioni di delay e riversare immediatamente e fisicamente le modifiche pendenti della RAM sulle trame del disco fisso (forcing flush).
- **mprotect:** Permette, in via esecutiva, di alterare dinamicamente i diritti di accesso (Lettura/Scrittura/Esecuzione) di frammenti del mapping (con una granularità che deve rispettare i confini imposti dalle *Pagine di Memoria* fisiche dell'hardware, tipicamente blocchi da 4 KB). Può finanche revocare totalmente l'accessibilità (**PROT_NONE**), causando la segnalazione di una gravissima violazione d'accesso (Segfault) al minimo tentativo di intrusione.

12 – Segnali

I segnali costituiscono il meccanismo fondamentale di notifica asincrona in spazio utente gestito dal kernel UNIX. Un segnale può essere concepito come un interrupt software: esso interrompe l'ordinario flusso di esecuzione di un processo per costringerlo a gestire un evento imprevisto.

L'asincronia intrinseca dei segnali implica che essi possano manifestarsi in punti precisi ma del tutto imprevedibili del codice sorgente durante le successive esecuzioni del medesimo programma. Le sorgenti generatrici si suddividono in tre macro-categorie:

- **Interazione Hardware:** Eventi scaturiti direttamente dalle periferiche o dalla CPU, come la pressione di combinazioni di tasti sul terminale di controllo [ad esempio la combinazione **Ctrl+C** volta a generare un segnale di **SIGINT**] o eccezioni hardware catastrofiche inviate come risposta a violazioni della protezione di memoria.
- **Notifiche del Kernel:** Messaggi inviati dal nucleo per segnalare il mutamento di stato di una risorsa, come l'arrivo di dati in un'operazione di input/output asincrono o la chiusura improvvisa di un canale di comunicazione.
- **Comunicazione Inter-Processo:** Segnali inviati esplicitamente da un processo a un altro (o a sé stesso) sfruttando apposite chiamate di sistema.

12.1 Ciclo di Vita

Il ciclo di vita di un segnale si articola in tre fasi logiche distinte gestite dalle strutture dati interne del sistema operativo:

1. **Generazione (Generation):** L'evento si verifica e il kernel aggiorna il vettore dei segnali nel Process Control Block (PCB) del processo destinatario, contrassegnando il bit corrispondente.
2. **Pendenza (Pending):** Intervallo temporale in cui il segnale è stato generato ma non ancora consegnato. Un segnale rimane in stato pendente se il processo ha momentaneamente mascherato o bloccato quel determinato segnale tramite la sua maschera dei segnali locale.
3. **Consegna (Delivery):** Il kernel forza il processo ad agire sul segnale non appena questo viene smascherato o il processo ritorna in modalità User Space dopo una transizione al kernel.

[Come già esaminato nel capitolo 5, ogni segnale possiede un'azione di default predefinita dal sistema (come la terminazione immediata, la generazione di un file di core dump per il debugging, la sospensione o l'ignoramento silente). Segnali critici come **SIGKILL** e **SIGSTOP** rimangono non intercettabili e non ignorabili a tutela della stabilità del sistema operativo].

13 – Gestione della Memoria Fisica

13.1 Allocazione a Partizioni e Frammentazione Esterna

La gestione della memoria fisica (RAM) richiede che il kernel organizzi lo spazio disponibile per ospitare i segmenti dei processi attivi. Nei modelli di allocazione contigua, ogni processo deve occupare un blocco di indirizzi fisici linearmente consecutivi.

Quando il sistema opera ad alta concorrenza, i processi vengono continuamente creati, eseguiti e terminati. La terminazione di cicli di task distribuiti in memoria [ad esempio la chiusura sequenziale del processo 2 e del processo 4] rilascia le rispettive aree, generando dei "buchi" vuoti di RAM intervallati da processi ancora in esecuzione [come i processi 3 e 5].

Questo fenomeno strutturale prende il nome di **Frammentazione Esterna (External Fragmentation)**: la memoria RAM totale libera complessiva è quantitativamente sufficiente per soddisfare una nuova richiesta di allocazione [ad esempio un nuovo processo che esige 20 MB contigui], ma tale spazio si presenta spezzettato in frammenti isolati e non consecutivi. Di conseguenza, il kernel si vede costretto a rifiutare il caricamento del nuovo processo, sprecando risorsa fisica utile.

Esistono due tipi di partizionamento: **statico** dove tutte le partizioni hanno dimensione fissata, questo ha come svantaggio che processi con dimensione minore di quella fissa sprecheranno molto spazio occupandone di non utilizzato. Questa è definita **frammentazione interna**. Il secondo tipo è il **dinamico** dove si istanzia una partizione grande quanto il necessario andando a eliminare la frammentazione interna, rimane il problema della frammentazione esterna quando un processo viene terminato e lascia un buco intermedio.

13.2 Compattazione e Rilocazione

Per ovviare alla frammentazione esterna senza ricorrere a complessi schemi di paginazione, l'architettura software del sistema operativo può attuare la **Compattazione della Memoria**.

La compactazione consiste nel traslare fisicamente i processi ancora attivi all'interno della RAM [spostando e rilocando, ad esempio, i processi 3 e 5] per farli convergere verso un unico estremo dello spazio indirizzi. Questa operazione unifica tutti i buchi sparsi in un'unica, grande area di memoria contigua, rendendola finalmente idonea per ospitare nuove partizioni.

La compactazione introduce un *overhead* computazionale straordinariamente pesante, poiché richiede il trasferimento fisico di megabyte o gigabyte di dati da una cella di RAM all'altra, tenendo la CPU impegnata in cicli di copia asincroni. Inoltre, questa tecnica è realizzabile solo se il sistema supporta la **Rilocazione Dinamica degli Indirizzi** a livello hardware (tramite registri base e limite gestiti dalla MMU), consentendo al programma di continuare l'esecuzione anche se i suoi indirizzi fisici cambiano a runtime.

13.3 Introduzione al Buddy System

Per gestire l'allocazione e la deallocazione di blocchi contigui di memoria fisica minimizzando la frammentazione esterna con un overhead ridotto, i kernel moderni implementano algoritmi specializzati, tra cui spicca il **Buddy System (Sistema dei Gemelli)**.

Il Buddy System organizza la memoria fisica strutturandola esclusivamente in blocchi la cui dimensione è una **potenza di 2** (2^n). Quando un processo richiede una partizione di dimensione X :

Il sistema cerca un blocco libero di dimensione minima sufficiente ($2^n < X$).

Se non esiste, prende un blocco più grande (2^{n+1}) e lo divide esattamente a metà, generando due blocchi "gemelli" (*buddies*) di dimensione 2^n . Uno viene allocato, l'altro rimane libero.

Quando un processo termina e libera il proprio blocco, il kernel verifica istantaneamente se il blocco gemello adiacente è anch'esso libero. In caso positivo, i due gemelli vengono fusi (*coalescing*) in modo automatico e immediato per ricostituire il blocco originario di potenza superiore, stroncando sul nascere la frammentazione esterna attraverso una logica matematica semplice e veloce.

13.3.1 Implementazione tramite Albero Binario

Questo algoritmo viene implementato in modo efficiente attraverso una **struttura dati ad albero binario**:

- **Radice dell'albero:** Rappresenta la partizione completa originale (l'intera memoria fisica a disposizione).
- **Nodi di primo livello:** Rappresentano le due partizioni identiche ottenute dal primo dimezzamento della radice.
- **Nodi di terzo livello:** Rappresentano sotto-partizioni aventi una dimensione pari a un quarto della memoria iniziale, e così via procedendo verso il basso.
- **Nodi foglia:** Rappresentano i blocchi fisici effettivamente utilizzabili e allocabili in quel determinato istante di tempo. Un nodo smette di essere una foglia quando viene diviso nei suoi discendenti.

Quando un processo invoca il rilascio di una risorsa (la quale corrisponde necessariamente a un nodo foglia contrassegnato come allocato), il kernel interroga l'albero per verificare se il nodo adiacente (ovvero l'altro figlio che condivide lo stesso identico nodo genitore) si trova nello stato libero. Se il gemello è libero, l'algoritmo distrugge i due nodi foglia e contrassegna il loro genitore comune come nuovo nodo foglia libero, risalendo la gerarchia.

13.4 Il Vincolo di Contiguità Fisica per le Periferiche

Questa esigenza, lungi dallo svanire con l'avvento della memoria virtuale, rimane un vincolo stringente nei sistemi hardware moderni.

Il motivo risiede nel fatto che non tutti gli attori coinvolti nella gestione e nel trasferimento dei dati sono provvisti di moduli hardware in grado di implementare le tecniche di virtualizzazione. L'esempio canonico è costituito dalle **periferiche hardware**:

Durante uno scambio di dati tramite buffer, le periferiche si aspettano di ricevere dal sistema operativo esclusivamente un indirizzo fisico di partenza e la lunghezza lineare del buffer

stesso.

Poiché le periferiche non integrano al proprio interno un'unità di gestione della memoria virtuale (MMU), esse operano e comunicano unicamente tramite indirizzi di memoria fisica.

Al contrario, per i processi utente questo vincolo decade: un processo può rilevare un intervallo di indirizzi come logicamente contiguo anche quando, nella realtà della memoria fisica, le relative pagine si trovano memorizzate in blocchi sparsi e non consecutivi.

A causa di questo vincolo architetturale, il kernel deve mantenere la capacità di partizionare la memoria fisica in blocchi contigui di dimensioni adeguate, liberandoli e riavvolgendoli all'occorrenza. Questo reintroduce il problema della frammentazione esterna, l'insorgenza di frammenti inutilizzati troppo piccoli tra due partizioni attive riduce l'efficienza del sistema. In questo specifico contesto, la compattazione della memoria tramite rilocalizzazione dinamica dei buffer può risultare impraticabile: se una periferica ha un'operazione di I/O avviata su un determinato buffer fisico, spostare la posizione di quel buffer a runtime causerebbe il fallimento distruttivo del trasferimento hardware.

14 – Gestione della Memoria Logica

14.1 Il Concetto di Segmento come Unità Logica

Per superare la rigidità di dover mantenere l'intera immagine di un processo costantemente e interamente caricata in RAM, si può suddividere l'immagine interna del processo in porzioni distinte, gestibili separatamente dal sistema operativo.

L'immagine di un programma è strutturalmente composta da sezioni funzionali (codice, dati inizializzati, heap, stack). La tecnica della **Segmentazione** trasforma queste sezioni in unità logiche chiamate **Segmenti**, rendendole esplicitamente visibili al programmatore o al compilatore, che ne diventano consapevoli.

I segmenti presentano per definizione **dimensioni diverse e variabili** tra loro (es. il segmento del codice avrà una dimensione diversa da quello dello stack). Attraverso questa astrazione, l'indirizzo generato da un processo non corrisponde più a un indirizzo fisico lineare, poiché i singoli segmenti devono poter essere spostati, allocati o rilocati in posizioni diverse della memoria fisica nel corso del tempo o tra differenti esecuzioni. Questo permette una gestione flessibile: il kernel può mantenere in RAM solo i segmenti di cui il processo necessita in un determinato istante, rimuovendo gli altri e ripristinandoli dinamicamente non appena si verifica una richiesta di accesso.

14.2 Meccanismo di Traduzione e Tabella dei Segmenti

Poiché un segmento può essere mappato su indirizzi fisici arbitrari e non contigui rispetto agli altri segmenti del medesimo processo, si rende necessario un meccanismo hardware di traduzione degli indirizzi.

L'indirizzo logico generato dal processo è un indirizzo strutturato composto da una coppia di valori:

Indirizzo Logico = Numero Segmento, Offset

Il numero del segmento identifica l'indice logico della sezione, mentre l'offset specifica la distanza in byte a partire dall'inizio di quel determinato segmento.

Per tradurre questa coppia in un indirizzo fisico lineare (utilizzabile dalla RAM), il sistema operativo si appoggia a una struttura dati denominata **Tabella dei Segmenti (Segment Table)**. Ogni riga (*entry*) della tabella corrisponde a un indice di segmento e memorizza tre informazioni fondamentali:

1. **Indirizzo di Base (Base Address):** L'indirizzo di memoria fisica reale da cui il segmento ha inizio. All'offset zero del segmento corrisponderà esattamente l'indirizzo di base; all'offset uno corrisponderà **base + 1**, e così via.
2. **Dimensione Limite (Limit/Size):** Il massimo offset valido consentito all'interno di quel segmento.
3. **Permessi di Accesso:** Flag di protezione che definiscono se quel blocco può essere letto, scritto o eseguito (notazione **rwX**).

L'algoritmo di traduzione eseguito dall'hardware esegue due controlli contestuali ad ogni accesso:

- Verifica che l'offset richiesto dal processo sia strettamente inferiore alla dimensione limite del segmento ($\text{Offset} < \text{Limite}$). Se l'offset supera il limite, l'hardware genera un'eccezione (*trap*), che costringe il kernel a inviare un segnale di errore fatale (tipicamente **SIGSEGV**) al processo.
- Verifica che la tipologia di operazione (es. un tentativo di scrittura) sia coerente con i permessi associati alla entry della tabella.

14.3 Protezione, Isolamento e Condivisione

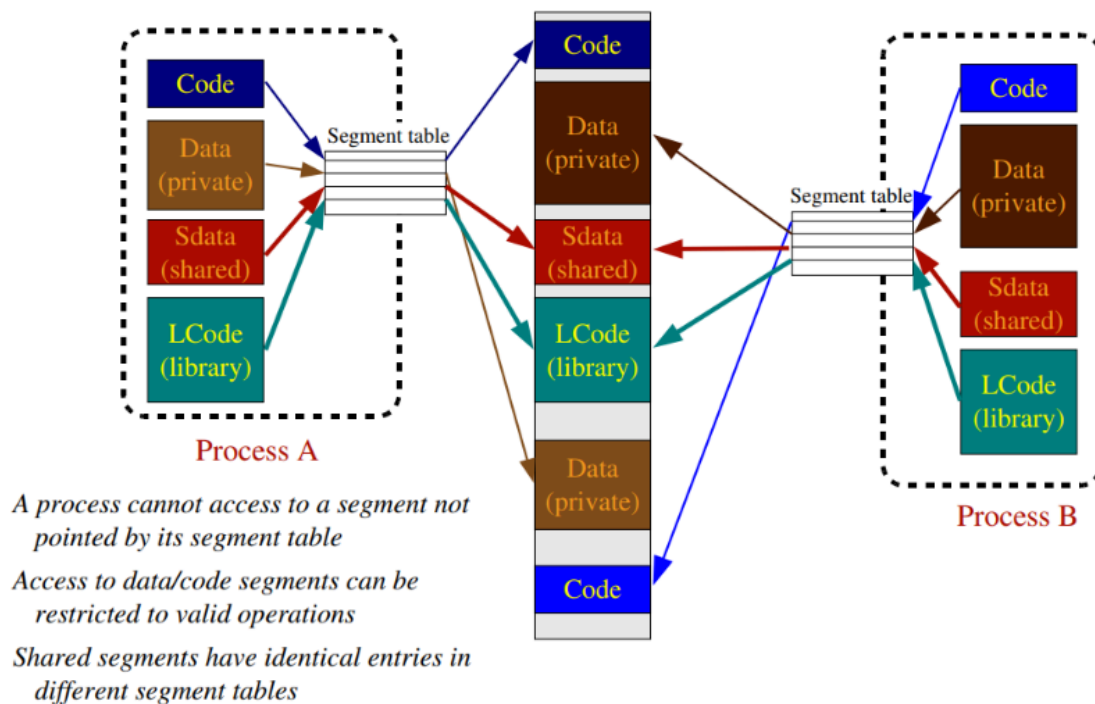
L'architettura basata su tabella dei segmenti permette di realizzare i requisiti di protezione e isolamento tra processi in modo nativo. Ogni processo possiede una propria Tabella dei Segmenti indipendente:

- Se il **Processo A** e il **Processo B** sono contemporaneamente in memoria, le entry della tabella del **Processo A** punteranno alle partizioni fisiche contenenti il proprio codice e i propri dati privati.
- La tabella del **Processo B** punterà a partizioni fisiche completamente distinte. Poiché nella tabella del **Processo B** non è fisicamente presente alcun puntatore o riferimento alle partizioni del **Processo A**, per il **Processo B** è matematicamente impossibile accedere o corrompere la memoria del **Processo A**, garantendo l'isolamento totale.

Allo stesso modo, la segmentazione permette la **Condivisione Controllata** delle risorse tra processi autonomi:

Condivisione di Dati: Se il kernel deve istanziare un'area di memoria condivisa (*shared memory*) tra il **Processo A** e il **Processo B**, è sufficiente configurare la entry del **segmento 1** del **Processo A** e la entry del **segmento 2** del **Processo B** in modo che entrambe contengano il medesimo identico *Indirizzo di Base* fisico. I due processi utilizzeranno indici di segmento potenzialmente diversi, ma ogni operazione di lettura o scrittura interagirà sulla stessa identica cella di memoria fisica.

Condivisione di Codice: Per ottimizzare la RAM in presenza di librerie condivise o programmi identici in esecuzione multipla, il kernel mantiene in memoria fisica un'unica partizione contenente il codice binario (configurato in sola lettura). Le tabelle dei segmenti di tutti i processi coinvolti conterranno entry che puntano alla medesima partizione di codice, evitando inutili duplicazioni di spazio.



14.4 Frammentazione Esterna

Sebbene la segmentazione elimini il problema della frammentazione interna (poiché ogni segmento viene dimensionato esattamente alla grandezza logica richiesta, evitando porzioni inutilizzate all'interno dell'unità), essa soffre strutturalmente del problema della **Frammentazione Esterna**.

Poiché i segmenti hanno dimensioni variabili, il loro continuo caricamento e scaricamento dalla RAM genera nel tempo dei buchi di spazio libero intervallati da segmenti attivi. Sebbene la somma totale dei buchi sia quantitativamente sufficiente per ospitare un nuovo segmento, l'assenza di una singola partizione fisica contigua costringe il sistema al blocco, richiedendo l'intervento di pesanti operazioni di compattazione della memoria.

15 – Paginazione

15.1 Pagine e Frame

Per superare i limiti imposti dalla frammentazione esterna delle partizioni a dimensione variabile, l'architettura dei sistemi operativi moderni adotta la strategia della **Paginazione**. A differenza della segmentazione, la paginazione spezza completamente il legame con la semantica logica del programma, operando in modo del tutto trasparente rispetto al programmatore, il quale non è consapevole della suddivisione in corso.

La paginazione si basa sul partizionamento rigido e geometrico dell'intero spazio indirizzi in blocchi aventi tutti la **stessa identica dimensione fissa** (scelta architetturale standard, ad esempio 4096 byte / 4 KB):

- **Frame (Page Frame):** La memoria fisica (RAM) viene suddivisa in blocchi fissi sequenziali chiamati Frame.
- **Pagine (Pages):** Lo spazio indirizzi logico visto dal processo viene suddiviso in blocchi della medesima dimensione chiamati Pagine.

Il principio fondamentale di questo modello è che **qualunque pagina logica può essere mappata su un qualunque frame di memoria fisica**, anche se non consecutivi tra loro. Il kernel può allocare due pagine logicamente consecutive del processo su due frame fisici situati agli estremi opposti della RAM, poiché l'interfaccia hardware si occuperà di ricostruire la linearità degli indirizzi.

Questo elimina totalmente la frammentazione esterna: se vi è un frame libero nel sistema, esso possiede per definizione la dimensione esatta per ospitare una qualsiasi pagina in attesa. L'unico spreco residuo è dato dalla **Frammentazione Interna**, che si verifica esclusivamente sull'ultima pagina di un processo qualora la dimensione totale dell'immagine non sia un multiplo perfetto di 4 KB. In media, lo spreco si attesta a mezza pagina per processo, una quantità considerata trascurabile a fronte dei benefici prestazionali.

15.2 Struttura dell'Indirizzo e Page Table

In un sistema paginato, l'indirizzo logico lineare non viene più interpretato come un valore unico, ma viene equamente suddiviso dall'hardware in due porzioni distinte:

1. **Numero di Pagina (Page Number):** I bit più significativi dell'indirizzo (la parte alta) identificano l'indice della pagina logica corrente. In un'architettura a 32 bit con pagine da 4 KB, i 12 bit meno significativi servono per indirizzare i byte interni alla pagina, lasciando i restanti 20 bit per definire il numero di pagina.
2. **Offset:** I bit meno significativi rappresentano lo spostamento lineare all'interno della pagina stessa.

La traduzione viene operata attraverso la **Tabella delle Pagine (Page Table)**. A differenza della segmentazione, l'offset non deve essere sommato all'indirizzo di base, ma viene semplicemente concatenato (attaccato) in coda all'indirizzo del frame ottenuto dalla tabella, poiché tutti i frame sono rigidamente allineati ai confini della dimensione di pagina.

Questo elimina la necessità di eseguire somme aritmetiche a livello di circuiti, velocizzando l'accesso.

Ogni riga (*entry*) della Tabella delle Pagine memorizza:

- L'indirizzo del Frame fisico corrispondente.
- I bit di protezione per regolare i permessi di accesso (**Read/Write/Execute**).
- Il **Bit di Presenza/Validità (Present/Valid Bit)**: Un flag cruciale che notifica se quella determinata pagina logica è attualmente mappata su un frame fisico in RAM oppure no. Se un processo tenta di accedere a una pagina il cui bit di presenza è impostato a zero, l'hardware rileva un errore e solleva un'eccezione strutturale al kernel.

15.3 Il Ruolo della MMU e il TLB

Il meccanismo di traduzione viene interamente eseguito in hardware dall'unità funzionale interna al processore denominata **MMU (Memory Management Unit)**. La Tabella delle Pagine di un processo, data la sua dimensione, risiede fisicamente all'interno della memoria RAM. Il processore tiene traccia della sua posizione attraverso speciali registri di controllo che puntano all'indirizzo di partenza della tabella corrente.

Il Problema del Doppio Accesso: Poiché la tabella risiede in RAM, ogni singolo accesso alla memoria richiesto dal programma (es. la lettura di una variabile) si tradurrebbe in due accessi fisici reali: il primo per consultare la Tabella delle Pagine e ottenere il frame, il secondo per leggere il dato vero e proprio. Questo dimezzerebbe le prestazioni del calcolatore.

Il TLB: Per abbattere questo overhead, la MMU integra al proprio interno una piccola memoria cache hardware ad altissima velocità denominata **TLB (Translation Lookaside Buffer)**. Il TLB conserva una copia locale delle entry relative alle ultime pagine visitate dal processo, sfruttando il **Principio di Località** (temporale e spaziale), secondo il quale un programma che accede a una pagina tenderà a operare su indirizzi vicini e per un intervallo di tempo prolungato.

15.3.1 Gestione di TLB Hit e TLB Miss

Ad ogni richiesta di I/O in memoria, la MMU interroga preventivamente il TLB:

TLB Hit: Le informazioni sulla pagina sono presenti nella cache interna. La MMU genera istantaneamente l'indirizzo fisico controllando i permessi, eseguendo l'operazione in un solo ciclo di clock senza consultare la tabella in RAM.

TLB Miss: Le informazioni non sono presenti nella cache del TLB. In questo scenario si aprono due strade implementative a seconda dell'architettura hardware:

- *Gestione Hardware:* Il processore interrompe temporaneamente il task, accede autonomamente alla Tabella delle Pagine in RAM, preleva i dati, aggiorna il TLB e completa l'accesso.

- **Gestione Software:** Il processore solleva una *trap* di eccezione trasferendo il controllo al kernel. Il codice del sistema operativo interroga la tabella in memoria, scrive le informazioni nel TLB tramite istruzioni dedicate e fa ripartire l'istruzione del processo, che questa volta troverà i dati nel TLB (*hit*) procedendo regolarmente.

15.3.2 Overhead nel Cambio di Contesto e ASID

Poiché la Tabella delle Pagine è strettamente privata per ogni singolo processo, quando lo scheduler esegue un cambio di contesto (*context switch*) deve aggiornare il registro speciale del processore facendolo puntare alla tabella del nuovo processo entrante.

Tuttavia, le entry memorizzate all'interno del TLB appartengono al processo uscente e non sono più valide per il nuovo contesto. L'approccio standard prevede lo **svuotamento totale (flush) del TLB** ad ogni cambio di processo. Questo introduce un pesante overhead prestazionale: il nuovo processo, al riavvio, subirà una cascata di TLB Miss consecutivi finché la cache non si sarà nuovamente riempita, rallentando l'esecuzione.

Per mitigare questo difetto, alcune architetture hardware integrano il meccanismo dell'**ASID (Address Space Identifier)** : ogni entry del TLB viene taggata con un codice numerico che identifica il PID del processo proprietario. Durante il cambio di contesto, il kernel non svuota il TLB; la MMU si limiterà a filtrare e utilizzare solo le entry il cui ASID coincide con l'identificativo del processo correntemente in esecuzione, permettendo la coesistenza di dati di processi diversi nella cache ed eliminando l'overhead di ricarica.

15.4 Tabella delle Pagine Multi-livello

Nelle architetture a **32 bit** con pagine standard da **4 KB**, lo spazio indirizzi si articola su **20 bit** per il numero di pagina, imponendo una tabella composta da **2²⁰** righe (oltre un milione di entry). Poiché la tabella deve contenere anche le entry per le pagine non mappate (per poter segnalare gli accessi illegali), ogni processo deve allocare in RAM una tabella mastodontica, anche se utilizza solo pochi kilobyte di memoria reale per il proprio codice. Il problema diventa insolubile su architetture a **64 bit**, dove il numero di pagine potenziali renderebbe la tabella troppo grande per risiedere fisicamente in memoria.

La soluzione universale è la **Tabella delle Pagine Multi-livello**. Questo modello suddivide il numero di pagina in più porzioni gerarchiche, indicizzando la tabella stessa come una struttura ad albero.

Prendendo come riferimento il modello classico a due livelli per architetture a 32 bit:

L'indirizzo logico viene frammentato in tre campi: **Directory di Pagina (10 bit)**, **Tabella delle Pagine (10 bit)** e **Offset (12 bit)**.

I 10 bit più significativi indicizzano la Tabella delle Pagine di Primo Livello (*Page Directory*). Ogni entry di questa directory non contiene l'indirizzo del frame fisico finale, bensì il puntatore alla base di una Tabella delle Pagine di Secondo Livello.

I secondi 10 bit selezionano l'entry specifica all'interno della tabella di secondo livello, la quale memorizza l'indirizzo del frame fisico reale.

Se un processo utilizza solo una porzione ridotta del proprio spazio di indirizzamento (ad esempio, solo gli indirizzi mappati sotto l'indice 5 della directory principale), il kernel allocherà esclusivamente la Page Directory di primo livello e l'unica tabella di secondo livello associata all'indice 5. Tutte le restanti tabelle di secondo livello non vengono allocate in RAM, risparmiando un'immensa quantità di spazio di archiviazione per gli spazi di indirizzamento sparsi (*sparse address spaces*). Nelle architetture a 64 bit, la gerarchia viene estesa fino a quattro o cinque livelli di profondità per gestire l'immensità degli indirizzi.

15.5 Tabella delle Pagine Invertita (Inverted Page Table)

Un approccio alternativo, concepito per svincolare la dimensione delle tabelle dalla grandezza dello spazio virtuale dei processi, è la **Tabella delle Pagine Invertita**.

Mentre una tabella standard mappa le pagine logiche verso i frame fisici (crescendo in base al numero di pagine dei processi), la tabella invertita opera al contrario: essa contiene **un'unica entry per ogni singolo frame reale presente nella memoria fisica**. La dimensione della tabella diventa quindi strettamente proporzionale alla quantità di RAM fisica installata sulla macchina, indipendentemente dal numero di processi attivi nel sistema. Ogni entry memorizza la coppia (PID del processo, Numero di pagina logica) attualmente ospitata in quel determinato frame.

Quando un processo genera un indirizzo logico, l'hardware deve individuare quale frame ospiti quella determinata pagina. Eseguire una ricerca lineare sull'intera tabella invertita ad ogni accesso alla memoria comporterebbe un rallentamento inaccettabile.

Funzione di Hash: Per ottenere un accesso immediato, l'architettura si appoggia a una **funzione di Hash**. Il numero di pagina logica e il PID vengono elaborati matematicamente dalla funzione di hash per calcolare istantaneamente l'indice esatto della tabella in cui effettuare il controllo. In caso di conflitti (più pagine che producono lo stesso indice di hash), il sistema gestisce l'ambiguità attraverso brevi liste concatenate, mantenendo il tempo di ricerca estremamente ridotto. [Nonostante l'efficienza teorica nello spazio, questo modello rende complessa l'implementazione della memoria condivisa, motivo per cui i sistemi moderni preferiscono adottare le tabelle multilivello].

15.6 Architetture Ibride: Segmentazione Paginata

Le strategie di segmentazione e paginazione non sono mutuamente esclusive, ma possono essere combinate all'interno del medesimo processore per capitalizzare i vantaggi di entrambe (la semantica logica e i permessi della segmentazione uniti alla flessibilità anti-frammentazione della paginazione). L'esempio storico e pervasivo è l'architettura **Intel x86 a 32 bit**.

In questa architettura, la traduzione dell'indirizzo avviene in due stadi hardware successivi:

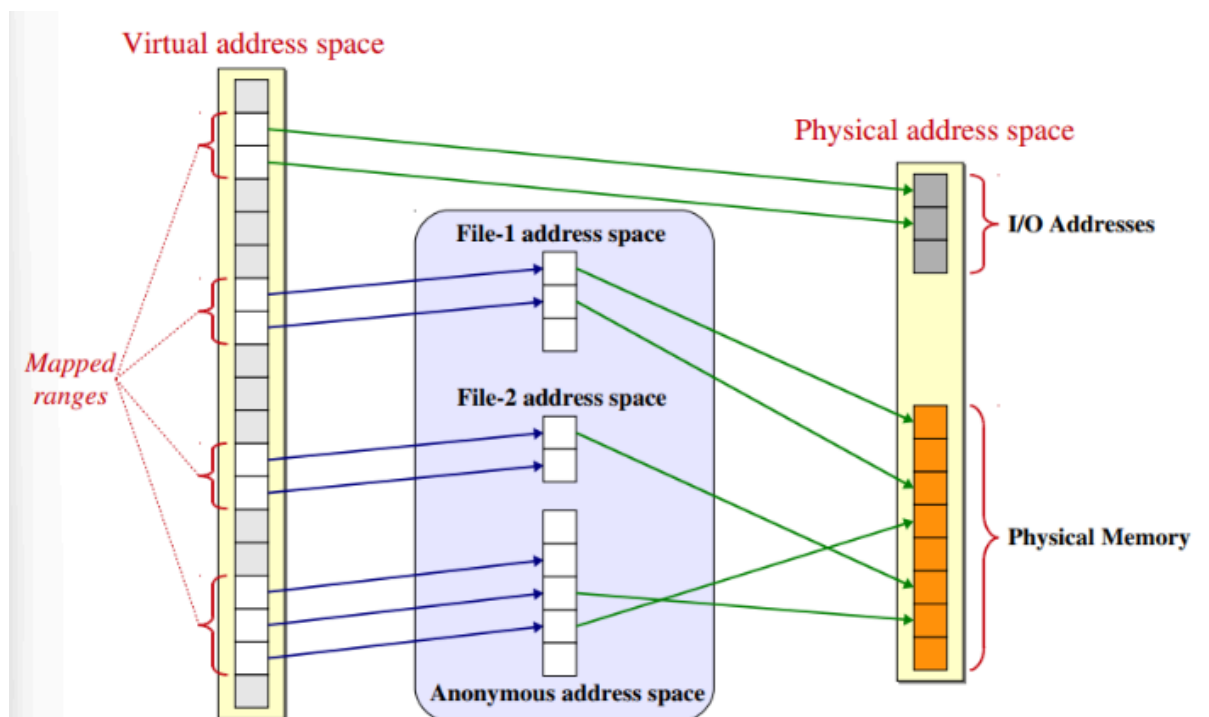
1. **Stadio della Segmentazione:** Il processo genera un indirizzo logico composto da selettore di segmento e offset. La MMU consulta la Tabella dei Segmenti del processo e genera un indirizzo intermedio a 32 bit denominato **Indirizzo Lineare**.
2. **Stadio della Paginazione:** L'Indirizzo Lineare ottenuto viene intercettato dal motore di paginazione, che lo scompone secondo lo schema multi-livello a due stadi (10 bit

directory, 10 bit tabella, 12 bit offset), consultando la Tabella delle Pagine per ricavare infine l'indirizzo fisico reale destinato alla RAM.

16 – Memoria Virtuale, Gestione dei Page Fault e Algoritmi di Sostituzione

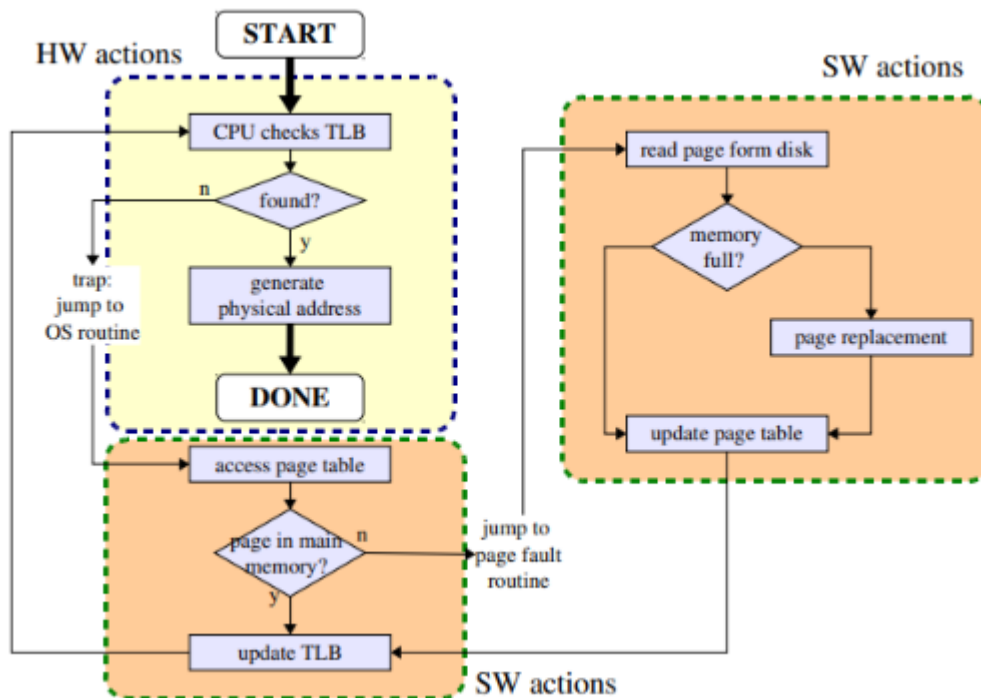
16.1 Fondamenti della Memoria Virtuale e Page Fault

La combinazione dei meccanismi di traduzione hardware e l'integrazione dei dispositivi di memoria di massa (*storage* come HDD o SSD) abilita il concetto di **Memoria Virtuale**. In questo modello, lo spazio di indirizzamento a disposizione di un processo non è più limitato e vincolato dalla quantità di RAM fisica installata sulla macchina, ma esteso fino ai limiti fisici dei bit di indirizzamento della CPU (es. 4 GB completi per sistemi a 32 bit), allocando le pagine in modo trasparente tra la RAM e lo storage.



Quando un processo tenta di accedere a una pagina logica, la MMU interroga la Tabella delle Pagine e verifica lo stato del *Present/Valid Bit*:

- Se il bit è attivo, la pagina risiede in un frame di RAM e l'accesso avviene istantaneamente.
- Se il bit è impostato a zero, ma l'indirizzo fa parte dello spazio legale del processo [mappato ad esempio tramite una precedente chiamata `mmap` o allocato nello spazio anonimo dell'heap], significa che la pagina è stata temporaneamente scaricata o risiede ancora sullo storage. Questa condizione scatena un'eccezione hardware denominata **Page Fault**.



La gestione del Page Fault viene interamente orchestrata dal software del kernel attraverso una precisa sequenza di passaggi:

1. Il processore interrompe l'esecuzione del processo e passa in modalità Kernel Mode, invocando il gestore dei page fault.
2. Il kernel verifica la validità dell'indirizzo. Se l'indirizzo è del tutto invalido (fuori dallo spazio indirizzamento autorizzato), invia un segnale di terminazione (**SIGSEGV**).
3. Se l'accesso è valido, il kernel individua la posizione della pagina all'interno dello storage.
4. Viene selezionato un frame libero all'interno della RAM fisica per ospitare i dati in arrivo.
5. Il kernel avvia un'operazione di lettura dallo storage per copiare i dati all'interno del frame selezionato (operazione di *I/O di blocco*, straordinariamente lenta rispetto ai tempi della CPU).
6. Al termine del trasferimento, il kernel aggiorna la Tabella delle Pagine del processo: associa il numero di pagina al frame fisico e imposta il *Present Bit* a 1.
7. Il kernel comanda il riavvio esatto dell'istruzione del processo che aveva causato il fault: questa volta la MMU rileverà la pagina presente, completando l'operazione in modo del tutto trasparente per il programma.

16.2 Il Working Set e Tempo Effettivo di Accesso (EAT)

L'efficienza di un sistema in memoria virtuale è strettamente legata al rispetto del principio di località. In un dato intervallo di tempo, un processo non interagisce con l'intera mole dei suoi dati, ma si focalizza su un sottoinsieme limitato di pagine denominato **Working Set**. La dimensione e i componenti del Working Set cambiano dinamicamente a seconda della fase di elaborazione del programma. Il kernel deve garantire che le pagine facenti parte del Working Set corrente rimangano costantemente mappate in RAM fisica per evitare un collasso delle prestazioni.

L'impatto dei page fault sulle performance globali viene formalizzato attraverso la formula del **Tempo Effettivo di Accesso (Effective Access Time - EAT)**:

$$EAT = (1 - P) \cdot T_{mem} + P \cdot T_{fault}$$

Dove:

- P rappresenta la probabilità di riscontrare un page fault durante un accesso alla memoria ($0 < P < 1$).
- T_{mem} è il tempo medio di accesso alla RAM fisica (ordine di grandezza dei nanosecondi).
- T_{fault} è il tempo necessario per gestire interamente l'eccezione di page fault, comprensivo dei tempi di lettura hardware dallo storage di massa (ordine di grandezza dei millisecondi).

Poiché l'ordine di grandezza di T_{fault} supera di sei ordini di scala quello di T_{mem} (millisecondi vs nanosecondi), il termine di destra domina completamente l'equazione se la probabilità P non viene mantenuta prossima allo zero. Di conseguenza, l'unico obiettivo critico delle politiche di gestione della memoria virtuale è **minimizzare al massimo il page fault rate**.

16.3 Page Replacement

Quando si verifica un page fault e la memoria fisica (RAM) risulta completamente piena, il kernel non dispone di frame liberi per ospitare la nuova pagina. Deve pertanto attuare una strategia di **Sostituzione delle Pagine (Page Replacement)**: selezionare un frame occupato, scaricarlo se necessario il contenuto sullo storage, contrassegnare la relativa entry come non presente ed assegnare il frame svuotato alla nuova pagina in ingresso.

L'algoritmo ottimale teorico richiederebbe la conoscenza del futuro (eliminare la pagina che non verrà utilizzata per il maggior intervallo di tempo a venire), una condizione impossibile nella pratica. Vengono quindi implementati algoritmi euristici di approssimazione:

16.3.1 Algoritmo FIFO (First-In, First-Out)

Sceglie come vittima da eliminare il frame che è stato mappato in memoria fisica da più tempo, gestendo i frame come una coda sequenziale rigida.

- **Svantaggi:** È un algoritmo cieco rispetto al principio di località. Una pagina mappata da molto tempo potrebbe essere il fulcro di un loop intensivo corrente; eliminarla causerà un immediato page fault successivo.
- **L'Anomalia di Belady:** Soffre di un comportamento controintuitivo: in determinate sequenze di accesso, **aumentando il numero di frame fisici a disposizione del processo, il numero di page fault aumenta** invece di diminuire, evidenziando una cattiva gestione della risorsa hardware.

16.3.2 Algoritmo LRU (Least Recently Used)

Applica direttamente il principio di località: seleziona come vittima la pagina che non viene accesa o utilizzata da più tempo, assumendo che sia la meno probabile per i calcoli futuri.

- **Svantaggi:** Sebbene prestazionalmente eccellente e immune all'anomalia di Belady, richiede un overhead hardware insostenibile: ogni entry della pagina dovrebbe memorizzare un marcatore temporale (timestamp) o un contatore aggiornato ad *ogni singolo accesso* alla memoria, appesantendo i circuiti del processore.

16.3.3 Algoritmo NRU, Second Chance e Clock (Approssimazioni di LRU)

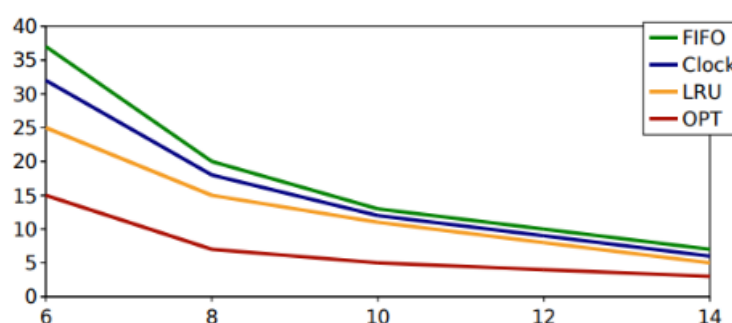
Per capitalizzare i vantaggi di LRU con un costo hardware ridotto, i sistemi reali implementano algoritmi di approssimazione basati sull'aggiunta di due soli bit di stato all'interno della entry della Tabella delle Pagine:

- **Bit di Utilizzo (User/Referenced Bit):** Impostato automaticamente a 1 dall'hardware non appena la pagina viene letta o scritta. Viene periodicamente azzerato a intervalli regolari dal kernel (es. allo scadere del timer di sistema).
- **Bit di Modifica (Dirty Bit):** Impostato automaticamente a 1 dall'hardware solo quando sulla pagina viene eseguita un'operazione di scrittura.

L'algoritmo **Second Chance** (o **Algoritmo dell'Orologio/Clock**) organizza logicamente le entry delle pagine come una lista circolare presidiata da un puntatore (*lancetta*):

1. Quando serve un frame, la lancetta esamina la pagina corrente e controlla il *Bit di Utilizzo*.
2. Se il bit è pari a 1, la pagina riceve una "seconda opportunità": il kernel azzerava il bit di utilizzo e sposta la lancetta sulla pagina successiva.
3. Se il bit è pari a 0, la pagina viene selezionata come vittima.

L'ottimizzazione tramite Dirty Bit: Prima di sovrascrivere la pagina vittima, il kernel controlla il *Dirty Bit*. Se il bit è 0 (pagina pulita), i dati in RAM sono identici a quelli sullo storage; il kernel può sovrascrivere direttamente il frame con una singola operazione di lettura della nuova pagina. Se il bit è 1 (pagina sporca), il processo ha modificato i dati in RAM; il kernel deve **eseguire prima una scrittura sullo storage** per salvare le modifiche e solo dopo procedere alla lettura della nuova pagina, raddoppiando l'overhead di I/O ($2 \times T_{\text{fault}}$). Pertanto, l'algoritmo esegue passaggi di scansione differenziati (*round robin*) per localizzare e preferire l'eliminazione di pagine che siano contemporaneamente non usate e pulite (*User=0, Dirty=0*), preservando le prestazioni del sistema.



16.4 Allocazione dei Frame e Thrashing

Quando si esegue un programma, il kernel deve determinare quanti frame fisici assegnare a quel determinato processo. Le politiche si dividono in:

- **Allocazione Fissa:** Ogni processo riceve un numero rigido di frame stabilito a priori (es. equamente diviso o proporzionale alla dimensione del file eseguibile su disco). La sostituzione delle pagine in caso di fault è strettamente *locale* (il processo deve sacrificare uno dei suoi frame).
- **Allocazione Dinamica (Inseguimento del Working Set):** Il kernel monitora costantemente il *page fault rate* del singolo processo. Se il processo sperimenta un numero di fault superiore a una soglia critica, significa che i frame assegnati sono inferiori al suo Working Set attuale: il kernel incrementa dinamicamente il numero di frame fisici a lui dedicati. Se i fault scendono quasi a zero, il kernel riduce i frame per ridistribuirli.

16.4.1 Il Problema del Thrashing (Iperpaginazione)

Il fenomeno del **Thrashing (Iperpaginazione)** costituisce il collasso prestazionale più grave di un sistema in memoria virtuale, scaturito da un ciclo di feedback positivo distruttivo tra lo scheduler e il motore di paginazione:

1. All'aumentare del numero di processi in esecuzione (grado di multiprogrammazione), l'utilizzo della CPU aumenta, ottimizzando il sistema.
2. Superata una soglia critica, la memoria fisica globale si esaurisce: i processi iniziano a subire page fault intensivi perché i frame totali disponibili scendono al di sotto della somma dei singoli Working Set.
3. Il gestore dei page fault sospende i processi in attesa del lento trasferimento dati dallo storage, svuotando la coda dei processi pronti.
4. Lo scheduler rileva che la CPU è completamente inutilizzata (ferma allo 0% in attesa dei dischi). Credendo che il sistema sia scarico, lo scheduler **introduce nuovi processi in memoria** per tentare di alzare l'utilizzo del processore.
5. I nuovi processi esigono frame, scatenando un'ulteriore cascata esponenziale di page fault.
6. Il sistema collassa in uno stato di stallo totale in cui l'intero calcolatore spende il 100% delle proprie risorse hardware e del tempo macchina unicamente nello spostare pagine avanti e indietro tra la RAM e lo storage (*thrashing*), arrestando ogni calcolo utile.

16.5 Copy-On-Write e Page Pinning

In conclusione, la flessibilità della paginazione abilita ottimizzazioni sistemiche fondamentali per la sicurezza e la reattività:

- **Copy-On-Write (COW) applicato alla `fork()`:** la system call `fork()` genera un processo figlio indipendente copia del padre. Invece di duplicare fisicamente i frame in RAM (operazione lenta), il kernel duplica unicamente la Tabella delle Pagine del padre nel figlio: entrambi i processi punteranno inizialmente ai medesimi frame fisici, configurati tuttavia temporaneamente in *sola lettura*. Se uno dei due processi esegue una scrittura, la MMU intercetta il trap di violazione, riconosce lo stato di COW, alloca

un nuovo frame isolato in RAM, vi copia il contenuto originale e ne sblocca i permessi in lettura/scrittura in modo esclusivo per il processo scrivente, posticipando la spesa hardware solo dove strettamente necessario.

- **Page Pinning (Blocco dei Frame):** Esistono scenari e system call dedicate per forzare il kernel a "bloccare" (*pinning*) determinate pagine all'interno della RAM fisica, impedendo tassativamente che l'algoritmo di sostituzione possa scaricarle sullo storage. Questo risponde a due esigenze:
 1. *Sicurezza:* Evitare che dati estremamente sensibili (come password o chiavi crittografiche memorizzate in RAM) vengano scritti in chiaro sulle trame dello storage di massa permanente, dove un utente malintenzionato potrebbe estrarli ed esaminarli offline. [Questa funzionalità è tipicamente riservata a processi utente con privilegi elevati].
 2. *Stabilità del Kernel:* Le porzioni fondamentali del codice del kernel (incluso lo scheduler stesso e i gestori delle interruzioni) devono risiedere stabilmente in frame bloccati in RAM fisica, per garantire esecuzione immediata ed evitare il blocco ricorsivo del sistema in caso di page fault sulle routine di gestione della memoria stessa.

17 – Pipe Anonime

17.1 Definizione e Proprietà Fondamentali

La forma più comune e nativa di comunicazione inter-processo (IPC) in ambiente UNIX è costituita dalle **Pipe Anonime**. Una pipe agisce come un canale di comunicazione puramente monodirezionale, in cui i dati transitano seguendo una rigida logica FIFO (First-In, First-Out).

[Alcuni sistemi operativi specifici implementano varianti di pipe bidirezionali; tuttavia, poiché queste estensioni non garantiscono la portabilità dello standard, la prassi impone l'impiego dei Socket qualora l'architettura esiga un flusso di comunicazione bidirezionale distribuito]. [I dettagli implementativi dei socket non verranno espansi in questa sede, essendo demandati ai corsi specialistici di reti di calcolatori].

Le pipe anonime vengono allocate internamente all'interno dello spazio di memoria gestito dal kernel e sono prive di una presenza o di un nome all'interno del file system globale. Questo vincolo strutturale implica che la pipe sia invisibile dall'esterno. Di conseguenza, due o più processi possono comunicare tramite una pipe anonima solo ed esclusivamente se condividono un **progenitore comune**. I processi figli, ereditando l'intero contesto e le tabelle delle system call dal processo padre al momento della duplicazione, ottengono l'accesso ai medesimi file descriptor che puntano al canale di comunicazione istanziato nel kernel.

17.2 Meccanica dei Buffer e Atomicità delle Scritture

Lo spazio di memorizzazione interno di una pipe non è illimitato; il canale si appoggia a un buffer di capacità definita, il cui dimensionamento varia in base alla configurazione e alla versione del sistema operativo in uso.

- **Dimensioni del Buffer:** Sotto il sistema operativo Linux, è storicamente garantito che il buffer interno possa ospitare almeno una quantità di dati pari alla dimensione di una singola pagina di memoria virtuale (tipicamente 4 KB). Nelle release moderne del kernel, la capacità standard di default è stata elevata a 64 KB. Tale parametro può essere modificato programmaticamente a runtime mediante l'utilizzo della system call `fcntl()` combinata con appositi comandi di controllo dedicati alla manipolazione del descrittore.
- **Atomicità e la macro `PIPE_BUF`:** Lo standard POSIX definisce la costante macro `PIPE_BUF`, che stabilisce il limite quantitativo entro il quale un'operazione di scrittura è garantita essere **atomica**. Il valore minimo garantito da qualunque sistema conforme è di 512 byte, ma su piattaforme Linux esso coincide stabilmente con 4 KB.
 - Se un processo esegue una `write()` di dimensione inferiore o uguale a `PIPE_BUF`, il kernel garantisce che l'intero blocco di dati venga iniettato nel buffer in modo indivisibile. Nessun altro processo concorrente potrà parzializzare o inserirsi nel flusso di scrittura.
 - Se la dimensione della scrittura supera il tetto di `PIPE_BUF`, l'operazione perde il requisito di atomicità. Il kernel è autorizzato a frammentare i dati in più tranches non predicibili, permettendo ad altri processi scrittori paralleli di

intrecciare i propri dati all'interno della medesima pipe, corrompendo la coerenza del flusso logico.

17.3 Creazione e Invocazione della System Call **pipe**

L'istanziamento di una pipe avviene in spazio utente richiamando l'apposita system call **pipe()**. La funzione non accetta percorsi stringa (mancando una fase di **open** esplicita), ma richiede come argomento un array di due elementi interi:

```
int pipe(int pipefd[2]);
```

In caso di successo, la funzione restituisce il valore **0** e popola l'array con due nuovi file descriptor validi, che rappresentano le due estremità del canale:

- **pipefd[0] (Estremità di Lettura):** Identifica il canale da cui estrarre i dati. [Per memorizzarne la funzione, si pensi all'analogia con l'indice dello standard input, che corrisponde storicamente al valore **0**].
- **pipefd[1] (Estremità di Scrittura):** Identifica il canale in cui inserire i dati. [Per analogia, l'indice richiama il valore dello standard output, storicamente mappato sul valore **1**].

Qualora il sistema non disponga di risorse sufficienti, la chiamata fallisce restituendo il valore **-1**. Poiché la pipe è un flusso puramente sequenziale, qualsiasi tentativo di riposizionare il cursore logico tramite la system call **lseek()** fallisce sistematicamente generando un errore di tipo **ESPIPE**.

17.4 Comportamento delle Primitive **read** e **write**

L'interazione con i descriptor di una pipe ricalca l'interfaccia standard dei file non bufferizzati, ma introduce vincoli di sincronizzazione stretti a seconda dello stato delle estremità.

17.4.1 Dinamiche della Chiamata **read**

Quando un processo interroga l'estremità di lettura (**pipefd[0]**) tramite la system call **read()**, il comportamento del kernel varia in base alla presenza di dati e allo stato dello scrittore:

- **End-of-File (EOF):** Se il processo (o tutti i processi) sul lato di scrittura ha chiuso definitivamente il file descriptor **pipefd[1]**, una volta che il lettore ha esaurito gli eventuali dati residui accumulati nel buffer, la **read()** restituisce immediatamente **0**. Questo segnala formalmente la fine del file, notificando che non potranno mai giungere nuovi dati dal canale.
- **Blocco e Sospensione:** Se la pipe è vuota, ma l'estremità di scrittura è ancora aperta presso qualche processo, la **read()** ordinaria **blocca il processo chiamante**, sospendendolo in attesa che la controparte inietti nuovi dati tramite una **write()**.
- **Gestione Non Bloccante (O_NONBLOCK):** Se il descrittore della pipe è stato preventivamente configurato con flag non bloccante tramite la system call **fcntl()**, il tentativo di lettura su una pipe vuota non sospende il task. La **read()** fallisce

immediatamente restituendo `-1` e impostando la variabile globale `errno` al valore della macro `EAGAIN`.

- [Tentativi di lettura multipli e contemporanei eseguiti da processi diversi sulla stessa identica estremità di lettura generano un comportamento non specificato dallo standard, richiedendo una sincronizzazione esplicita a livello software per evitare corruzioni].

17.4.2 Dinamiche della Chiamata `write`

L'operazione di scrittura sull'estremità `pipefd[1]` è strettamente vincolata all'esistenza di un potenziale lettore dall'altro capo del canale:

- **Canale Interrotto (Errore `EPIPE`):** Se l'estremità di lettura della pipe è stata completamente chiusa da tutti i processi, qualsiasi tentativo di eseguire una `write()` fallisce istantaneamente. La system call restituisce il valore `-1` e scrive nella variabile `errno` il codice d'errore `EPIPE`.
- **Generazione del Segnale `SIGPIPE`:** Contestualmente al fallimento della system call, il kernel invia imperativamente al processo scrittore il segnale asincrono `SIGPIPE`. Poiché l'azione di default associata a `SIGPIPE` è la terminazione forzata del programma, il codice deve esplicitamente intercettare o ignorare tale segnale per poter gestire l'errore `EPIPE` a livello logico senza subire un crash immediato.
- **Saturazione del Buffer (Pipe Piena):** Se il buffer interno della pipe è completamente pieno, una `write()` in modalità bloccante sospende l'esecuzione del processo. Il task rimane congelato finché il lettore non estrae dati dal canale, liberando lo spazio necessario. In modalità non bloccante (`O_NONBLOCK`), la scrittura su una pipe satura restituisce `-1` con errore `EAGAIN`.

17.5 Pattern d'Uso Canonico tramite `fork`

Dato che i descrittori hanno significato solo all'interno del processo che li ha istanziati, il pattern d'uso standard prevede una rigorosa sequenza di sdoppiamento e successiva chiusura selettiva delle estremità non utilizzate per stabilire un flusso coerente:

1. **Fase 1 (Inizializzazione):** Il processo padre invoca `pipe()` ottenendo i due descrittori operativi.
2. **Fase 2 (Duplicazione):** Il padre esegue la system call `fork()`. Il processo figlio nasce ereditando fedelmente la tabella dei file descriptor, ottenendo l'accesso alla medesima struttura nel kernel.
3. **Fase 3 (Chiusura Complementare):** Poiché il canale è monodirezionale, ciascun processo deve dismettere l'estremità che non compete alla propria logica di flusso:
 - Se l'architettura prevede che il figlio scriva e il padre legga, il processo figlio deve invocare immediatamente `close(pipefd[0])` per sigillare il lato lettura. Successivamente utilizzerà `write(pipefd[1], ...)`.
 - Dualmente, il processo padre deve invocare `close(pipefd[1])` per chiudere il lato scrittura. Successivamente interrogherà il canale tramite un ciclo di `read(pipefd[0], ...)` finché quest'ultima non restituirà zero (EOF), indicando che il figlio ha concluso le trasmissioni e chiuso il canale.

18 – Pipe con Nome (FIFO)

18.1 Le Pipe con Nome (FIFO) come File Speciali

Per superare il limite strutturale delle pipe anonime (che costringono i processi a condividere un legame di parentela), i sistemi UNIX introducono le **Pipe con Nome**, comunemente definite **FIFO**.

Una FIFO è un file speciale memorizzato all'interno della gerarchia del file system, identificato dal flag di tipo **p**. A differenza delle pipe anonime, le FIFO possiedono un percorso stringa persistente (es. `/tmp/my_fifo`), permettendo a due processi completamente indipendenti e privi di legami di parentela di localizzare il canale e stabilire una comunicazione.

- **Transitorietà dei dati:** Nonostante la persistenza del nome sul disco, la FIFO agisce puramente come un'interfaccia logica. I dati scambiati non vengono mai scritti stabilmente sullo storage di massa; transitano interamente all'interno di buffer allocati in RAM gestiti dal kernel. Quando tutti i processi coinvolti chiudono i rispettivi descrittori, il buffer viene azzerato, e la riapertura successiva del file darà origine a una pipe logicamente vuota.

18.2 Creazione, Apertura e Sincronizzazione delle FIFO

A livello di codice C, una pipe con nome viene generata richiamando la system call `mkfifo()` (o la variante relativa `mkfifoat()`):

```
int mkfifo(const char *pathname, mode_t mode);
```

La funzione accetta il percorso del file e la maschera ottale dei permessi di accesso (soggetta alla umask del processo). In caso di successo restituisce `0`, mentre restituisce `-1` qualora il file sia già esistente o si verifichino violazioni nei permessi di scrittura sulla directory target. In spazio utente (shell), la medesima operazione può essere effettuata tramite i comandi di riga `mkfifo` o `mknod`.

Una volta istanziata la FIFO, i processi vi accedono invocando la comune system call `open()`. L'apertura introduce una semantica di sincronizzazione implicita fondamentale:

- **Apertura Bloccante (Default):** Se un processo apre la FIFO in sola lettura (`O_RDONLY`), la system call `open()` **si blocca immediatamente**, sospendendo il task finché un secondo processo indipendente non esegue una chiamata `open()` sulla medesima FIFO in modalità di scrittura (`O_WRONLY` o `O_RDWR`), e viceversa. Questo meccanismo garantisce che il canale sia pienamente stabilito ad entrambe le estremità prima che i processi possano iniziare a computare dati.
- **Apertura Non Bloccante (`O_NONBLOCK`):** * Se il processo apre il canale in sola lettura (`O_RDONLY | O_NONBLOCK`), la `open()` ha successo immediato e restituisce il descrittore senza attendere lo scrittore.

- Se il processo tenta di aprire il canale in sola scrittura (`O_WRONLY | O_NONBLOCK`), la system call fallisce istantaneamente restituendo il valore -1 e valorizzando `errno` con il codice d'errore `ENXIO` (segnalando che non vi è alcun lettore attivo in ascolto sul canale).
- [L'apertura di un canale FIFO in modalità di lettura e scrittura simultanea `O_RDWR` non presenta un comportamento rigorosamente definito dallo standard POSIX, sebbene sui kernel Linux l'operazione vada a buon fine simulando la presenza costante di una controparte attiva].

18.3 Astrazioni di Alto Livello: `popen` e `pclose`

Per agevolare lo sviluppo evitando la gestione manuale e sequenziale di `pipe()`, `fork()`, chiusura descrittori ed esecuzione tramite `exec()`, lo standard mette a disposizione la funzione di libreria ad alto livello `popen()`:

```
FILE *popen(const char *command, const char *type);
```

La funzione accetta come parametri una stringa contenente un comando di sistema e la modalità operativa (`type`):

- **Modalità Lettura (`type = "r"`):** Il kernel crea una pipe, esegue una `fork()` e lancia il comando delegando l'interpretazione a un'istanza di shell. Lo standard output del comando figlio viene reindirizzato in scrittura sulla pipe, mentre il processo genitore riceve in ritorno un puntatore a un oggetto strutturato di tipo `FILE *` bufferizzato in User Space. Il genitore può estrarre l'output prodotto dal comando semplicemente leggendo dallo stream tramite funzioni standard come `fgets()` o `fscanf()`.
- **Modalità Scrittura (`type = "w"`):** Speculare alla precedente, collega lo standard input del processo figlio all'estremità di scrittura della pipe. Il genitore invia dati al figlio scrivendo direttamente all'interno dello stream restituito da `popen()`.

Al termine delle elaborazioni, lo stream non deve essere dismesso tramite `fclose()`, bensì invocando la funzione dedicata `pclose()`. Questa primitiva si occupa di effettuare il flush protettivo dei buffer residui, chiudere la pipe sottostante e invocare internamente una `wait()` per attendere la formale terminazione del processo figlio, restituendone il codice di uscita al chiamante.

18.4 Altre Primitive di IPC POSIX

Oltre alle pipe, l'ecosistema POSIX mette a disposizione altre famiglie di primitive per consentire lo scambio dati o la sincronizzazione tra processi indipendenti, condividendo la medesima logica di persistenza basata su stringhe identificative a livello kernel:

- **[Code di Messaggi (Message Queues):** Canali di comunicazione strutturati a messaggi discreti definiti dagli standard UNIX, il cui approfondimento di dettaglio viene omesso in questa sezione].

- **Semafori con Nome POSIX:** Istanziati richiamando la funzione `sem_open()`, associano una primitiva di sincronizzazione a un nome logico di sistema (es. `"/my_sem"`). Consentono a processi distinti di coordinarsi effettuando operazioni atomiche di `sem_post()` e `sem_wait()`. Il nome del semaforo viene rimosso dal kernel tramite la funzione `sem_unlink()`. [I semafori anonimi o senza nome utilizzano invece `sem_init()` e richiedono l'ereditarietà da `fork()` per la condivisione].
- **Memoria Condivisa POSIX (Shared Memory):** Consente l'allocazione di un'area di RAM comune a più processi indipendenti. Il canale viene aperto tramite `shm_open()` (restituendo un file descriptor speciale mappato su una stringa), dimensionato mediante la chiamata `ftruncate()` e infine innestato nello spazio indirizzi virtuale del processo invocando `mmap()`. La dismissione dell'oggetto avviene richiamando `shm_unlink()`.

19 – Esercizi

19.1 Atomicità nei Gestori di Segnale

Un problema ricorrente nelle prove d'esame riguarda la manipolazione di variabili globali (es. contatori di eventi) condivise asincronamente tra il flusso principale del programma (**main**) e le routine dei gestori di segnale (**Signal Handlers**).

Quando giunge un segnale (es. **SIGUSR1**), il kernel interrompe istantaneamente il task corrente per deviare l'esecuzione del processore verso l'handler. Se l'handler effettua un incremento standard su una variabile globale condivisa (es. **counter++**), tale operazione non è intrinsecamente sicura: l'istruzione in C si traduce in tre distinte istruzioni assembly (lettura in registro, incremento, scrittura in RAM). Se il flusso principale viene interrotto esattamente nel mezzo di una analoga operazione di manipolazione su quella medesima variabile, lo stato logico del dato viene corrotto.

Poiché all'interno dei gestori di segnale è **severamente vietato acquisire lock complessi o Mutex** (rischiando l'insorgenza di deadlock irreversibili qualora il flusso principale venisse interrotto detenendo il lock stesso), la soluzione ingegneristica consiste nell'utilizzare le funzioni built-in messe a disposizione dal compilatore. Tali primitive implementano cicli di lettura-modifica-scrittura atomici direttamente supportati dall'hardware del processore:

```
__sync_fetch_and_add(&counter, 1);
```

Questa funzione garantisce che l'incremento avvenga in un singolo passaggio hardware indivisibile, blindando la coerenza del contatore globale senza introdurre l'overhead o i rischi dei lock in spazio utente.

19.2 Struttura di una Pipeline Circolare

Un classico scenario d'esame prevede il coordinamento cooperativo di tre processi distinti (**A**, **B**, **C**) configurati in una pipeline ad anello chiuso (**A → B → C → A**) per l'elaborazione sequenziale e progressiva di dati.

Per realizzare questa architettura in modo corretto tramite pipe anonime, il processo radice **A** deve seguire una precisa sequenza di inizializzazione e chiusura dei descrittori al fine di evitare stalli bloccanti indotti dal kernel:

1. **Fase di Setup delle Pipe:** Il processo **A** deve creare preventivamente due distinte pipe anonime richiamando la system call **pipe()**: la pipe destinata al canale **A → B** e la pipe per il canale **C → A**.
2. **Generazione del Processo B:** Il processo **A** esegue una **fork()** generando il processo figlio **B**, **B** eredita entrambe le pipe ad entrambe le estremità.
3. **Isolamento del Canale A → B:** Il processo **A** e il processo **B** devono chiudere le estremità che non competono al loro dialogo diretto:
 - **A** chiude il lato lettura della prima pipe (**A_to_B[0]**), mantenendo aperto solo il lato scrittura.

- **B** chiude il lato scrittura della medesima pipe (**A_to_B[1]**), mantenendo aperto solo il lato lettura.
- 4. **Generazione del Canale B → C:** Prima di dare origine al terzo processo, il processo **B** crea una terza pipe anonima indipendente (**B_to_C**). Successivamente esegue una **fork()** per generare il processo **C**. Il processo **C** eredita l'intero contesto residuo da **B**.
- 5. **Chiusura Massiva Complementare:** Per evitare che il sistema rimanga congelato in attesa di un EOF che non potrà mai manifestarsi, ogni attore deve purificare la propria tabella dei file descriptor:
 - Il processo **B** chiude entrambe le estremità della pipe **C_to_A** (ereditata originariamente da **\$A\$**) poiché non partecipa a quel canale, e chiude il lato lettura di **B_to_C**.
 - Il processo **C** chiude l'estremità residua di **A_to_B**, chiude il lato scrittura di **B_to_C** (mantenendo aperto il lato lettura per ricevere da **B**) e chiude il lato lettura di **C_to_A** (mantenendo aperto il lato scrittura per trasmettere i dati finali ad **A**).

Se un solo processo (es. **B**) dimenticasse di chiudere un'estremità di scrittura di una pipe in cui non deve scrivere (es. **C_to_A[1]**), il processo lettore legato a quel canale (**A**) non riceverà mai il valore **0** (End-of-File) dalla system call **read()** qualora **C** terminasse l'esecuzione. Il kernel rileverà che esiste ancora un potenziale scrittore attivo nel sistema (**B**), mantenendo il lettore permanentemente congelato in uno stato di stallo bloccante indotto dal software.

20 – Gestione dei Dischi, Partizionamento e Mapping Logico dei Volumi

20.1 Basso Livello vs Alto Livello

La preparazione di una periferica di memorizzazione (sia essa un disco magnetico o un disco a stato solido) per renderla utilizzabile da un sistema operativo si articola in due fasi distinte:

1. **Formattazione di Basso Livello (Low-Level Formatting):** Consiste nella preparazione fisica del supporto. Il costruttore suddivide la superficie del disco in tracce e settori, assegnando a ciascun settore un identificativo numerico (ID) univoco. In questa fase vengono scelti i fattori ottimali di *interleaving* e *skewing* in base alla velocità di rotazione del piatto e allo spostamento della testina, e vengono create le aree per i codici di correzione degli errori (ECC) con i relativi pattern fissi di ridondanza. [Ormai questa operazione è eseguita esclusivamente in fabbrica dal produttore ed è raro che l'utente finale possa o debba interagirvi]. Per interleaving si intende il numero di salti tra un settore e il successivo in quanto la testina e il motore hanno velocità diverse, questo permette di ottimizzare il tempo di seek assieme allo skewing che è lo sfasamento tra due cilindri.
2. **Formattazione di Alto Livello / Partizionamento:** Consiste nel rendere la periferica utilizzabile in modo logico dal software. L'utente finale suddivide l'intera area dello storage in porzioni chiamate **Partizioni**, destinate a ospitare file system indipendenti o aree di memoria virtuale (swap). Una partizione è definita formalmente come una porzione di blocchi contigui della periferica di storage.

20.2 MBR (Master Boot Record) (schema vecchio)

Lo schema classico di partizionamento, nato con i primi personal computer, prevede la memorizzazione delle informazioni di controllo nel primissimo settore del disco, denominato **Master Boot Record (MBR)**. Questo settore contiene un piccolo programma assembly per l'avvio della macchina e, nella sua porzione finale, una tabella di partizionamento a quattro elementi (entry).

Ciascuna entry della tabella MBR definisce una **Partizione Primaria** descrivendola con tre campi principali:

- L'indirizzo del blocco/settore di partenza (un numero intero).
- La lunghezza complessiva della partizione in blocchi (dimensione).
- Un identificativo di tipo (ID) e flag di stato, come il flag di partizione *avviabile* (bootable).

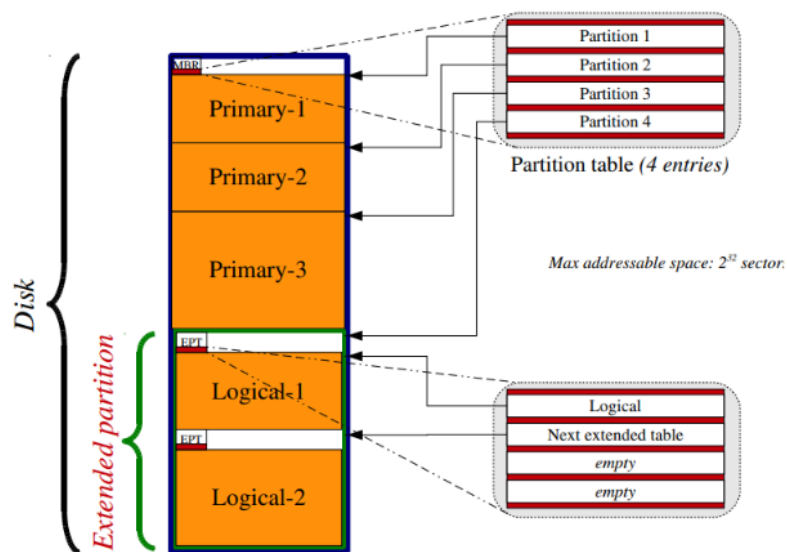
20.2.1 Estensione del Limite delle 4 Partizioni

[Poiché la tabella MBR limita rigidamente a quattro il numero massimo di partizioni primarie, lo schema è stato storicamente esteso]. È possibile configurare una delle entry come puntatore a una **Tabella di Partizione Estesa (Extended Partition Table)**. Questo blocco speciale adotta il medesimo formato a quattro entry ma ne utilizza attivamente solo due: la

prima definisce una **Partizione Logica** (inizio e dimensione), mentre la seconda punta a una successiva tabella estesa. Si viene così a costituire una lista concatenata di partizioni logiche che rimuove il limite hardware originario.

20.2.2 Il Limite dei 32 bit

Il limite dello schema MBR risiede nell'impiego di numeri interi a 32 bit per indicare il blocco di partenza e la dimensione. Un'architettura a 32 bit può indirizzare al massimo 2^{32} settori. Assumendo la dimensione standard settoriale storica di **512 byte**, il limite massimo di capacità gestibile da un MBR si attesta a 2 Terabyte.



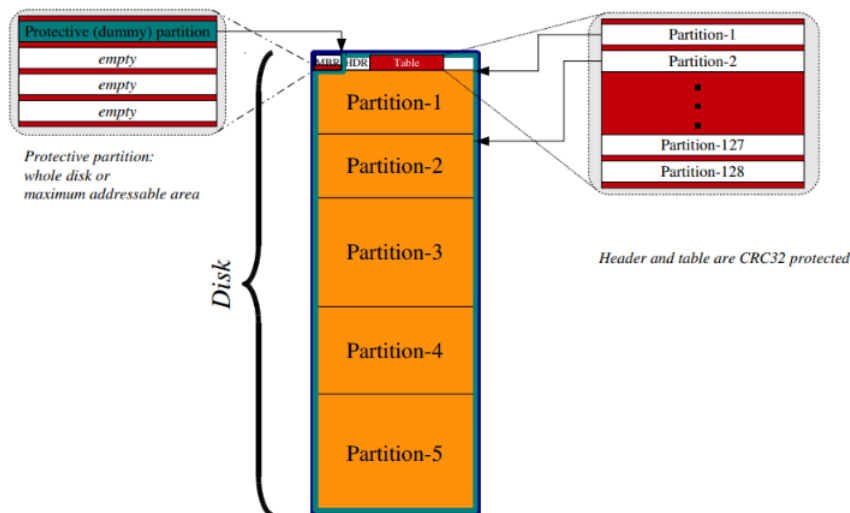
20.3 GPT (GUID Partition Table) (schema moderno)

Per superare i limiti di capacità dell'MBR e garantire maggiore robustezza, i sistemi moderni adottano lo standard **GPT (GUID Partition Table)**. Esso introduce le seguenti evoluzioni architetturali:

- **Dimensione dei campi aumentata:** I bit dedicati a indicare l'inizio e la fine della partizione superano il vincolo dei 32 bit, permettendo di gestire dischi di dimensioni immesse sul mercato.
- **Partizioni arbitrarie non concatenate:** Le informazioni non sono memorizzate come lista concatenata, ma risiedono in una tabella ad elementi multipli preceduta da un *header* che ne descrive la dimensione. Il numero di partizioni non è limitato a un settore.
- **Meccanismi di ridondanza e controllo:** L'header contiene campi per il controllo di integrità tramite algoritmo CRC. Se il CRC calcolato a runtime diverge da quello memorizzato, il sistema rileva la corruzione. Inoltre, la tabella GPT viene salvata in due copie separate: una copia primaria all'inizio del disco e una copia di backup (safe) alla fine dello storage, per il ripristino in caso di danneggiamento.

20.3.1 Allocazione Protettiva (Protective MBR)

Per evitare che sistemi operativi datati o software legacy non conformi a GPT interpretino il disco come non inizializzato (rischiando di sovrascriverne i dati), GPT implementa un meccanismo di protezione nel primo settore. Viene inserita una tabella MBR fittizia (falsa) in cui la prima entry dichiara che l'intero disco è occupato da una partizione di tipo non riconosciuto, mentre le restanti tre entry rimangono vuote. [I vecchi software rileveranno il disco come interamente occupato e si asterranno dal modificarlo].

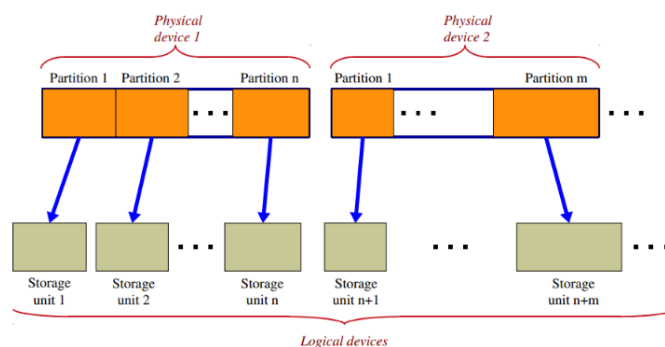


20.4 Mapping Logico dei Volumi (LVM)

È necessario distinguere due concetti fondamentali:

- **Unità Fisica (o Dispositivo Fisico di Storage):** L'intero disco rigido o la sua singola partizione fisica.
- **Unità Logica (o Volume Logico):** La porzione di storage vista dal software e dal file system per il contenimento dei dati.

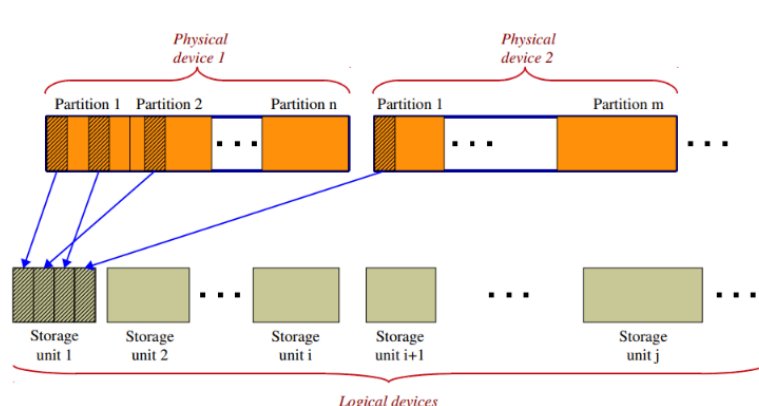
Nei sistemi classici, il mapping tra unità fisica e unità logica è di tipo 1-to-1: a ogni partizione corrisponde un singolo volume. Lo svantaggio primario di tale rigidità emerge in fase di manutenzione: se la partizione allocata per i dati utente (es. la directory `/home`) risulta sottodimensionata, mentre un'altra partizione è semivuota, il ridimensionamento richiede procedure macchinose e rischiose di copia, cancellazione e ricreazione delle tabelle.



Il **Mapping Logico dei Volumi (LVM)** introduce uno strato di astrazione software intermedio. Un'unità logica viene disaccoppiata dalle partizioni fisiche e costruita aggregando delle porzioni di memoria ("fette") prelevate da unità fisiche o dischi differenti.

- **Ridimensionamento Dinamico:** Se un volume logico esaurisce lo spazio, il software di gestione può espanderlo dinamicamente a caldo aggiungendo fette di memoria precedentemente non utilizzate, senza interrompere il file system.
- **Funzionalità Avanzate dello Strato Software:** Poiché l'accesso avviene filtrato da LVM, lo strato software può implementare trasparentemente funzioni di **Crittografia** (i dati sono in chiaro per l'utente ma cifrati sui blocchi fisici) o configurazioni **RAID / Array di Dischi**. L'uso di array permette di aumentare le prestazioni leggendo i byte in parallelo da dischi distinti (striping) o di implementare la correzione degli errori tramite mirroring o blocchi di parità (ridondanza).

[Si evidenzia che i formati di queste tabelle logiche LVM non sono standardizzati; ogni sistema operativo adotta strutture proprietarie non compatibili nativamente (es. un volume LVM avanzato creato su Linux non è leggibile da Windows senza software specifico)].



20.5 Gestione Blocchi Difettosi

Le periferiche di memorizzazione reali possono presentare porzioni di superficie magnetica o celle di memoria difettose all'origine o soggette a usura nel tempo, denominate **Bad Block**. Il sistema operativo gestisce queste anomalie secondo due strategie alternative:

- **Gestione Online (A livello di Controller):** L'operazione è eseguita direttamente dall'elettronica a bordo della periferica (il controller) in modo trasparente per il software e per il sistema operativo. Quando il controller rileva un blocco difettoso, lo marca internamente e ne rimappa l'indirizzo logico reindirizzandolo verso un blocco sano prelevato da una riserva fisica di settori di ridondanza integrati dal produttore. [Il vantaggio è la totale trasparenza; lo svantaggio è che altera la contiguità fisica, degradando l'efficacia degli algoritmi di scheduling].
- **Gestione Offline (A livello di Sistema Operativo):** I settori danneggiati rimangono visibili al sistema operativo. È il codice del file system a farsi carico del problema

registrando i blocchi difettosi all'interno di apposite strutture dati (come una lista dedicata o una mappa di bit / *bitmap*). In fase di allocazione dei file, il sistema operativo interroga queste strutture e salta i blocchi compromessi. Presenta un overhead software maggiore ma permette al kernel di conoscere l'esatta disposizione geometrica dei dati.

20.6 Analisi dei Tempi di Accesso nei Dischi Magnetici

Il tempo totale richiesto per accedere a un blocco su un disco magnetico è composto da due componenti fisiche principali:

1. **Tempo di Spostamento della Testina (Seek Time):** Il tempo necessario per muovere meccanicamente il braccio del disco sopra la traccia desiderata. È una grandezza dell'ordine dei millisecondi ed è la causa principale di latenza.
2. **Latenza di Rotazione (Rotational Latency):** Il tempo impiegato dal piatto per ruotare e posizionare il settore richiesto esattamente sotto la testina. [Questa velocità è fissa e determinata dal produttore hardware; l'utente può solo bilanciare costi e prestazioni in fase di acquisto (es. dischi a 7200 RPM vs 15000 RPM)].

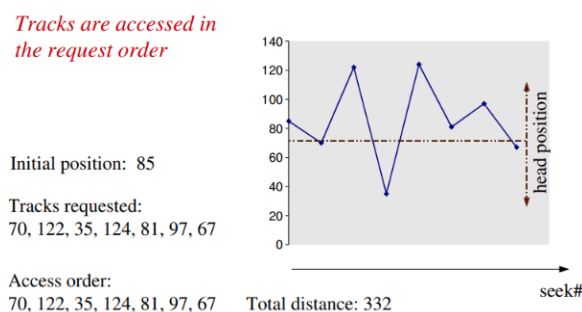
Il software del kernel, vedendo l'elenco complessivo delle richieste di I/O pendenti per una periferica, può ottimizzare le prestazioni **riordinando la coda degli accessi** per minimizzare lo spostamento meccanico della testina (*seek time*). Questa operazione è attuata dalle **Politiche di Scheduling del Disco**.

20.7 Algoritmi di Scheduling

Per valutare l'efficienza di un algoritmo di scheduling, si calcola la **distanza totale percorsa dalla testina**, espressa in numero di tracce attraversate per servire l'intera coda.

20.7.1 FCFS (First-Come, First-Served)

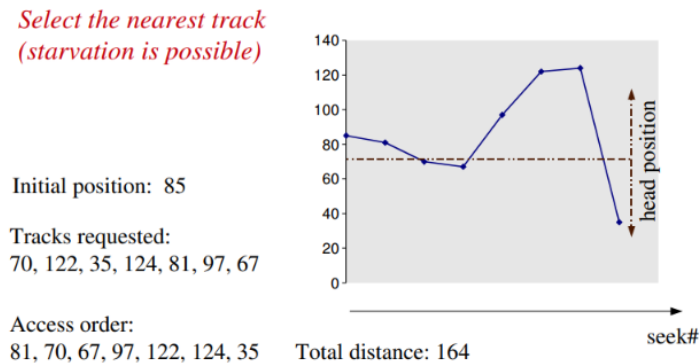
È l'approccio più banale: le richieste vengono servite nell'esatto ordine cronologico con cui giungono alla coda, senza compiere alcuna ottimizzazione geometrica. Non comporta overhead software, ma causa ampi e continui spostamenti del braccio meccanico lungo la superficie del disco, determinando tempi di accesso medi elevati.



20.7.2 SSTF (Shortest Seek Time First)

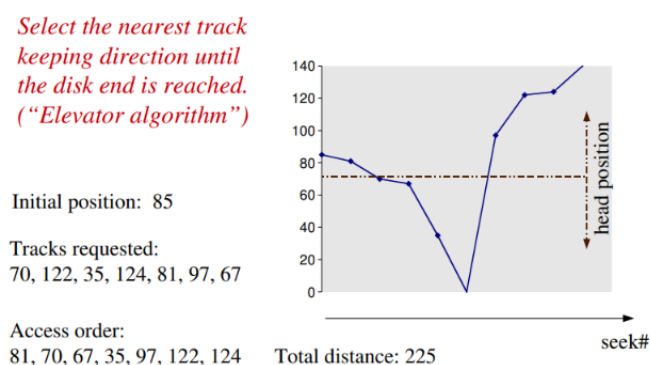
L'algoritmo seleziona di volta in volta, tra tutte le richieste pendenti nella coda, quella posizionata sulla traccia geometricamente più vicina alla posizione corrente della testina. Riduco drasticamente la distanza totale percorsa e il tempo di accesso medio.

- **Il problema della Starvation:** Se continuano a sopraggiungere nuove richieste concentrate attorno alla posizione attuale della testina, l'algoritmo continuerà a preferirle, posticipando indefinitamente il servizio delle richieste attestate sulle tracce più lontane.



20.7.3 Algoritmo SCAN (o dell'Ascensore)

La testina si muove continuamente lungo una direzione (es. verso l'interno del disco), servendo tutte le richieste che incontra sul proprio cammino man mano che interseca le relative tracce. Quando raggiunge l'estremità fisica del supporto (la traccia zero o la massima), il movimento viene invertito e la scansione procede in senso opposto. [Il comportamento è speculare a quello di un ascensore multipiano che raccoglie le chiamate solo nella direzione della corsa corrente]. Risolve il problema della starvation.



20.7.4 Algoritmo C-SCAN (Circular SCAN)

È una variante di SCAN progettata per offrire un tempo di attesa più uniforme. La testina serve le richieste muovendosi in un solo ed unico verso (es. solo scendendo verso la traccia minima). Quando raggiunge l'estremità inferiore, il braccio compie un salto rapido verso la

traccia massima ricominciando la corsa nello stesso verso, senza servire alcuna richiesta durante il tragitto di ritorno.

*Select the nearest track
keeping direction until
the disk end is reached.
Always use the same
direction*

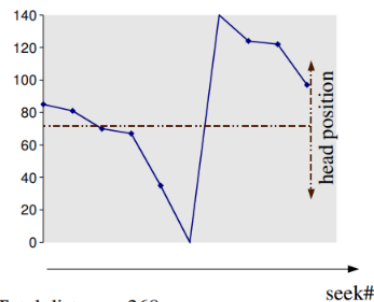
Initial position: 85

Tracks requested:
70, 122, 35, 124, 81, 97, 67

Access order:

81, 70, 67, 35, 124, 122, 97

Total distance: 268



20.7.5 Algoritmi LOOK e C-LOOK

Raffinano la logica di SCAN e C-SCAN introducendo un controllo preventivo sulla coda: la testina non è costretta a viaggiare fino all'estremità fisica assoluta del disco. Se l'algoritmo rileva che nella direzione corrente non vi sono più tracce da servire, inverte il movimento (LOOK) o esegue il salto di ritorno (C-LOOK) immediatamente dall'ultima richiesta utile, risparmiando tempo prezioso.

*As SCAN, but reverse
movement when there
are no more tracks in
the current direction*

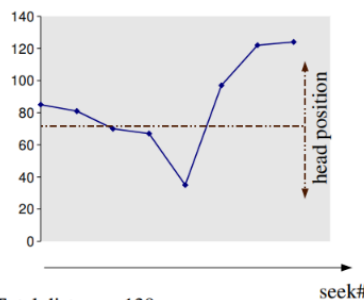
Initial position: 85

Tracks requested:
70, 122, 35, 124, 81, 97, 67

Access order:

81, 70, 67, 35, 97, 122, 124

Total distance: 139



20.7.6 Algoritmi N-Step SCAN e F-SCAN

Sviluppati per evitare che la testina rimanga bloccata su una singola traccia a causa di un flusso continuo di richieste ravvicinate.

- **N-Step SCAN:** Suddivide la coda delle richieste in blocchi di dimensione fissa **N**. Ogni sottomandata viene servita applicando internamente la logica **SCAN**, congelando la ricezione di nuove richieste all'interno di quel ciclo.
- **F-SCAN:** Adotta un approccio a due code distinte. Mentre la prima coda viene interamente servita dall'algoritmo dell'ascensore, tutte le nuove richieste che giungono in tempo reale vengono accumulate nella seconda coda, differenziando l'esecuzione ed eliminando i fenomeni di monopolizzazione del braccio.

21 – Architettura SSD

I **Dischi a Stato Solido (SSD)** abbandonano i componenti magnetici e meccanici a favore di grandi matrici di memoria a semiconduttore non volatile. La tecnologia dominante si basa sulle memorie **NAND Flash**, costituite da transistor a *gate flottante* capaci di intrappolare elettroni per isolamento elettrico, preservando lo stato logico di polarizzazione per anni o decenni anche in assenza di alimentazione elettrica. [Esistono tecnologie emergenti basate su fenomeni fisici differenti, quali le RAM resistive, le memorie a cambio di fase o quelle basate sullo spin degli elettroni].

L'assenza di componenti meccanici in movimento determina un **tempo di accesso (seek time) estremamente rapido**, ridotto alla mera decodifica elettronica dell'indirizzo.

21.1 Pagine e Blocchi

Dal punto di vista logico, il software continua a vedere l'SSD come un array lineare di settori standard. Fisicamente, tuttavia, la memoria Flash è strutturata in due livelli gerarchici rigidi:

- **Pagina (Page):** È l'unità intrinseca di base per le operazioni di **Lettura e Scrittura** (tipicamente ampia 4 KB). La lettura è immediata e può essere eseguita un numero infinito di volte senza degradazione fisica.
- **Blocco (Block):** È un insieme raggruppato di pagine (ad esempio, un blocco composto da 8 o più pagine) ed è l'unità minima per le operazioni di **Cancellazione**.

21.2 Scrittura Fuori Posto (Out-of-Place Write)

La limitazione tecnologica fondamentale delle memorie Flash risiede nell'asimmetria delle transizioni di stato dei bit:

- L'operazione di Scrittura può unicamente commutare lo stato di un bit **da 1 a 0**.
- Per commutare un bit **da 0 a 1**, è necessario applicare tensioni elevate per svuotare il gate flottante dagli elettroni, operazione che prende il nome di Cancellazione e che può essere eseguita **solo ed esclusivamente a livello di intero Blocco**.

Di conseguenza, se il software richiede di modificare il contenuto di una pagina già scritta, il controller non può sovrascriverla direttamente. L'SSD adotta la strategia della **Scrittura Fuori Posto (Out-of-Place Write)**:

1. Il controller scrive i nuovi dati all'interno di una pagina fisicamente diversa che si trova nello stato libero / inizializzato (indicata convenzionalmente come pagina *gialla*, contenente tutti bit a 1).
2. La vecchia pagina fisica viene marcata internamente come *invalida* (non più aggiornata).
3. Il controller aggiorna dinamicamente una propria tabella interna modificando il mapping: associa l'indirizzo della **Pagina Logica** richiesto dal software alla nuova **Pagina Fisica** reale.



21.3 Flash Translation Layer (FTL) e Garbage Collection

Lo strato software/firmware integrato nell'elettronica dell'SSD che si occupa di gestire questa complessa astrazione prende il nome di **Flash Translation Layer (FTL)** o Controller del disco. L'FTL deve garantire costantemente la presenza di pagine libere pronte per accogliere le scritture.

Quando le pagine libere scendono sotto una soglia critica, l'FTL avvia in background la **Garbage Collection (Raccolta dei Rifiuti)**: identifica i blocchi che contengono un'alta concentrazione di pagine marcate come invalide, copia le eventuali poche pagine ancora valide superstiti all'interno di un altro blocco libero e procede alla cancellazione fisica massiva dell'intero blocco originario, riportando tutte le sue pagine allo stato iniziale utile.

21.4 Il Livellamento dell'Usura (Wear Leveling)

Ogni blocco di memoria Flash ha un ciclo di vita limitato, quantificato in un numero massimo di cancellazioni fisiche tollerabili prima della rottura dell'isolamento elettrico del gate (tipicamente nell'ordine delle centinaia di migliaia di cicli). Se un algoritmo cancellasse continuamente lo stesso blocco, quest'ultimo morirebbe precocemente, riducendo la capacità totale del dispositivo.

L'FTL implementa pertanto strategie di **Wear Leveling (Livellamento dell'Usura)**: monitora il numero di cancellazioni subite da ogni singolo blocco e ridistribuisce dinamicamente i dati in modo da uniformare lo stress fisico su tutta la matrice. Se un blocco contiene dati statici che non cambiano mai (es. il codice del sistema operativo), il controller può decidere di spostare quei dati in un blocco molto stressato, liberando un blocco "fresco" per i task di scrittura intensiva.

21.4.1 Impatto sulle Metriche di Scheduling

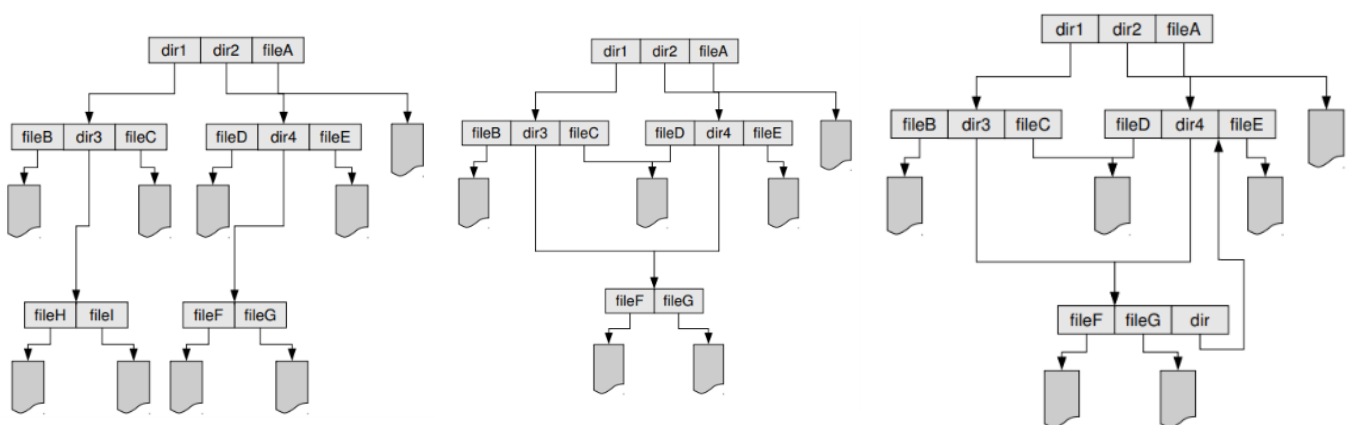
[Mentre per i dischi magnetici l'obiettivo dello scheduling è la minimizzazione della distanza geometrica del braccio meccanico], nei dischi SSD la metrica cambia radicalmente. Poiché tutte le celle hanno il medesimo tempo di accesso elettronico, gli algoritmi di tipo SCAN o SSTF sono del tutto inutili. Lo scheduling negli SSD si focalizza sul **distribuire le scritture in modo uniforme** per assecondare le politiche di Wear Leveling del controller, ottimizzando l'invecchiamento dei blocchi.

22 – File System

22.1 Struttura Logica delle Directory

Il file system organizza i dati persistenti attraverso una struttura gerarchica astratta gestita dal kernel. L'organizzazione formale delle directory si è evoluta nel tempo per rispondere a precise esigenze di condivisione:

- **Struttura ad Albero (Tree):** Modello classico in cui ogni file o directory ha un solo ed unico genitore. Limita la flessibilità, impedendo di raggiungere lo stesso oggetto da percorsi logici differenti.
- **Grafo Aciclico Diretto (Acyclic Graph):** Permette a un file di avere più rinvii o collegamenti da directory diverse (percorsi multipli per lo stesso oggetto). Per garantire l'assenza di cicli infiniti durante le scansioni ricorsive, lo standard Unix applica un vincolo rigoroso: **è vietato creare collegamenti fisici (Hard Link) che puntino a directory**. Un hard link può puntare solo a file regolari, incrementando il contatore di riferimenti nell'i-node.
- **Grafo Generico:** Ammette la presenza di cicli. Nei sistemi moderni è tollerato solo attraverso l'impiego dei **Collegamenti Simbolici (Symbolic Link / Soft Link)**. Un link simbolico non è un puntatore fisico del file system, ma un file speciale contenente una stringa di testo che descrive un percorso. Il kernel gestisce i link simbolici in modo differenziato, interrompendo la navigazione se rileva loop infiniti.



22.2 Layout del Disco per il File System

La partizione logica viene formattata suddividendo lo spazio in strutture dati di gestione e blocchi di memorizzazione.

22.2.1 Il Superblocco (Superblock / Boot Sector)

È posizionato all'inizio della partizione e contiene i metadati vitali che descrivono l'intera struttura del file system : il codice identificativo del tipo di file system (es. ext2, FAT32, NTFS), il numero complessivo di blocchi utilizzati e i puntatori di partenza per localizzare le altre tabelle cruciali (come la tabella degli i-node o la directory radice).

22.2.2 L'Unità di Allocazione: I Cluster

I singoli settori fisici da 512 byte sono spesso troppo piccoli per essere indicizzati singolarmente in modo efficiente, poiché richiederebbero tabelle di gestione enormi. Il file system raggruppa pertanto una quantità fissa di settori contigui (es. 4 o 8 settori) in un'unica unità di allocazione logica, denominata **Cluster** o Blocco del File System.

- **Frammentazione Interna:** Il cluster rappresenta la dimensione minima allocabile per un file. Se un cluster è ampio 16 KB (4 settori da 4 KB) e un file contiene un solo byte di dati reali, esso occuperà comunque l'intero spazio di 16 KB sul disco, determinando uno spreco di spazio interno per frammentazione.

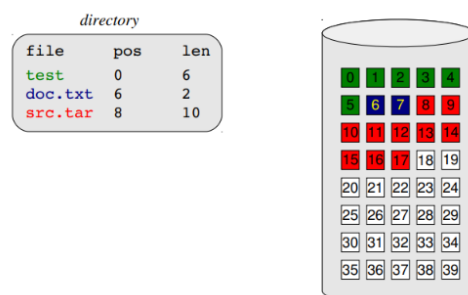
22.3 Strategie di Allocazione dei File

Le modalità con cui i blocchi di un file vengono disposti e tracciati sul supporto fisico si dividono in tre strategie fondamentali:

22.3.1 Allocazione Contigua

Ogni file occupa un blocco di partizioni fisicamente consecutive sul disco. La entry della directory deve memorizzare unicamente l'indirizzo del blocco iniziale e la dimensione totale del file.

- **Vantaggi:** Prestazioni sequenziali eccezionali nei dischi magnetici (la testina non deve muoversi durante la lettura dell'intero file). Il tempo di accesso al singolo blocco (*seek time*) è costante.
- **Svantaggi:** Soffre gravemente di **frammentazione esterna** a seguito di continue cancellazioni e creazioni. Inoltre, il ridimensionamento in append è fortemente limitato: se il blocco successivo è già occupato da un altro file, l'espansione è impossibile e richiede la laboriosa copia dell'intero file in un'altra area libera. È usata quasi esclusivamente per supporti a sola lettura come CD-ROM e DVD (standard ISO 9660).



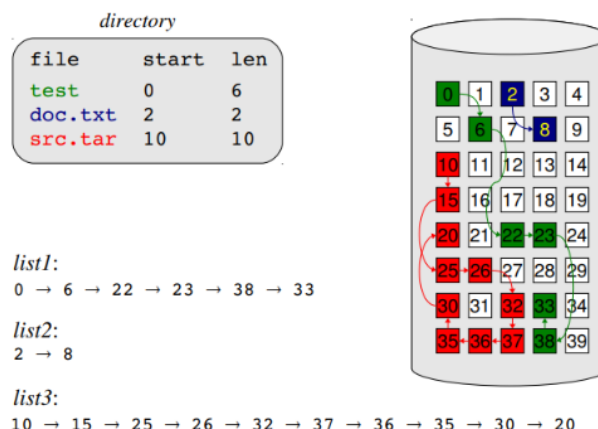
22.3.2 Allocazione Concatenata (Linked Allocation / File Allocation Table)

I blocchi di un file sono distribuiti in modo sparso sulla superficie del disco, e ogni blocco contiene al proprio interno un puntatore logico al blocco successivo (lista concatenata).

Per evitare di sottrarre spazio ai dati all'interno del blocco, lo standard **FAT (File Allocation Table)** centralizza tutti questi puntatori in un'unica grande tabella posizionata all'inizio del

disco. Ciascun elemento della tabella corrisponde a un cluster fisico del disco e memorizza il numero del cluster successivo che compone il file, fino a un marcatore speciale di fine file (EOF).

- **Vantaggi:** Assenza totale di frammentazione esterna; qualsiasi blocco libero può essere allocato semplicemente modificando la catena dei puntatori.
- **Svantaggi:** L'accesso casuale a un blocco intermedio (es. leggere il blocco 50) è lento e lineare, poiché richiede di scansionare l'intera catena a partire dal cluster iniziale. Inoltre, la tabella FAT costituisce una struttura dati estremamente critica: un singolo danneggiamento corrompe l'intera catena logica del file system, motivo per cui i sistemi FAT memorizzano sempre la tabella in copie multiple obbligatorie.



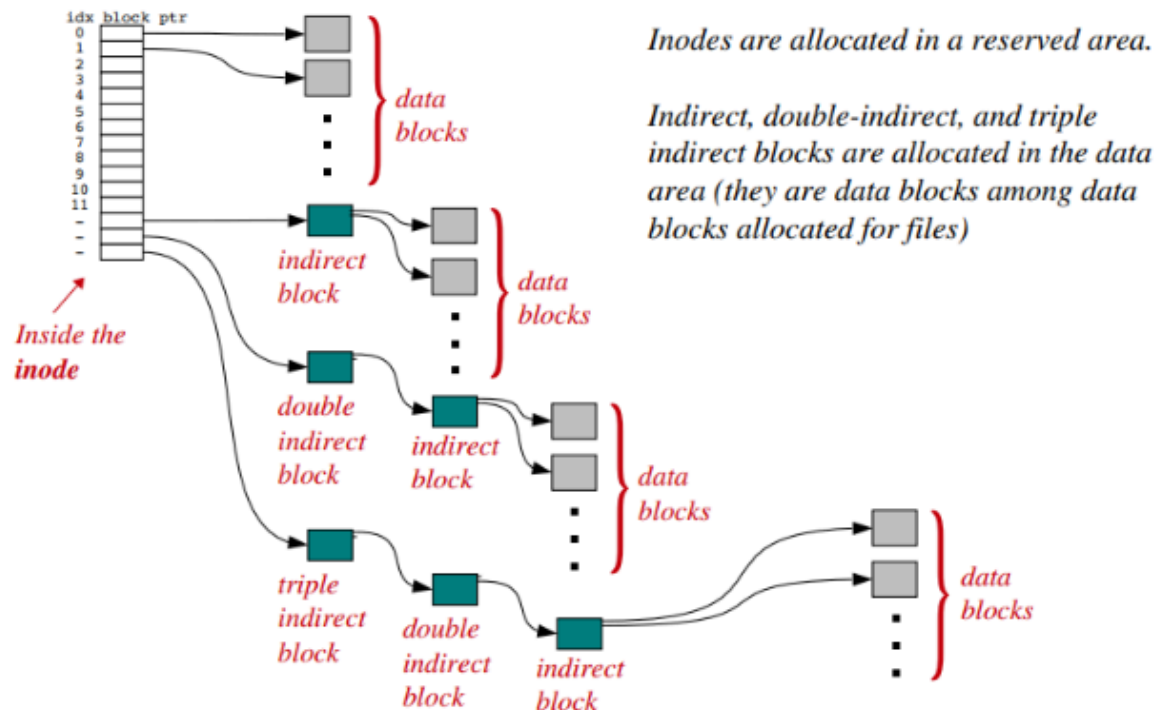
22.3.3 Allocations Indicizzata (I-node con Blocchi Indiretti)

Ogni file è associato a una struttura dati dedicata chiamata **i-node (Index Node)**, che contiene tutti i metadati del file (permessi, dimensioni, timestamp) e una tabella privata di puntatori diretti ai blocchi di dati. Consente un accesso casuale immediato a qualsiasi blocco ($O(1)$) interrogando l'indice della tabella. La dimensione massima dei file è fissata all'inizializzazione della tabella e allo stesso modo è limitato il numero massimo di tabelle allocate. Tutte le allocazioni non utilizzate si traducono in spazio sprecato.

Per evitare che la dimensione dell'i-node diventi eccessiva per file di grandi dimensioni, il sistema adotta l'architettura dei **Blocchi Indiretti** (tipica del file system **ext2** di Linux):

- **Puntatori Diretti:** I primi 12 elementi della tabella dell'i-node puntano direttamente ai blocchi contenenti i dati reali (sufficienti per la maggior parte dei file di piccole dimensioni).
- **Puntatore Indiretto Singolo:** Il 13° elemento punta a un blocco intermedio memorizzato sul disco, il quale non contiene dati ma un intero array di puntatori secondari (es. 1024 puntatori per blocchi da 4 KB).
- **Puntatore Indiretto Doppio:** Il 14° elemento punta a un blocco che contiene puntatori rivolti a blocchi indiretti singoli, espandendo geometricamente la capacità.
- **Puntatore Indiretto Triplo:** Il 15° elemento estende la catena logica su tre stadi gerarchici di rinvio, permettendo al file system di indirizzare file di dimensioni

nell'ordine dei Terabyte, allocando i blocchi di indice solo quando strettamente richiesto dalla crescita del file.



22.4 Implementazione delle Directory e Spazio Libero

Le directory associano il nome testuale di un file al rispettivo indice descrittore (i-node o cluster iniziale). La ricerca può basarsi su due modelli:

- **Scansione Lineare:** I record dei file sono memorizzati sequenzialmente uno dopo l'altro. La ricerca richiede una scansione lineare con complessità temporale $O(N)$, inefficiente per directory contenenti migliaia di file.
- **Tabella di Hash:** Il nome del file viene elaborato da una funzione di hash per calcolare istantaneamente l'indice esatto della entry nella directory, riducendo il tempo di ricerca a una costante ($O(1)$), al netto della gestione software delle eventuali collisioni.

La tracciatura dei blocchi liberi per le nuove allocazioni avviene prevalentemente tramite **Mappa di Bit (Bitmap)**: un vettore in cui ogni singolo bit rappresenta lo stato di un blocco (0 = libero, 1 = occupato). Essendo estremamente compatta, la bitmap può essere caricata interamente in RAM, permettendo al kernel di individuare serie di blocchi liberi consecutivi tramite veloci operazioni bit-a-bit in memoria. Altre metodologie di memorizzazione sono: **linked list** dove ogni blocco punta al successivo, questa struttura può essere vista come file "speciale"; di solito sono necessari più blocchi liberi, si procede con la tecnica del **grouping** che corrisponde a una linked list con associato numero di blocchi liberi contigui.

23 – File System Avanzati

23.1 Consistenza dei Metadati: I File System Journalled

Durante le normali operazioni di scrittura (es. la creazione o l'espansione di un file), il sistema operativo deve compiere modifiche multiple e coordinate sulle strutture del disco: deve allocare i blocchi dati, aggiornare i puntatori nell'i-node e modificare i record della directory coinvolta. Se si verifica un malfunzionamento hardware o uno spegnimento improvviso della macchina esattamente nel mezzo di questa sequenza, il file system si ritrova in uno stato di **inconsistenza grave** (es. blocchi marcati come occupati ma non associati ad alcun file).

I **File System Journalled (con Giornale)** risolvono questa criticità introducendo una logica transazionale:

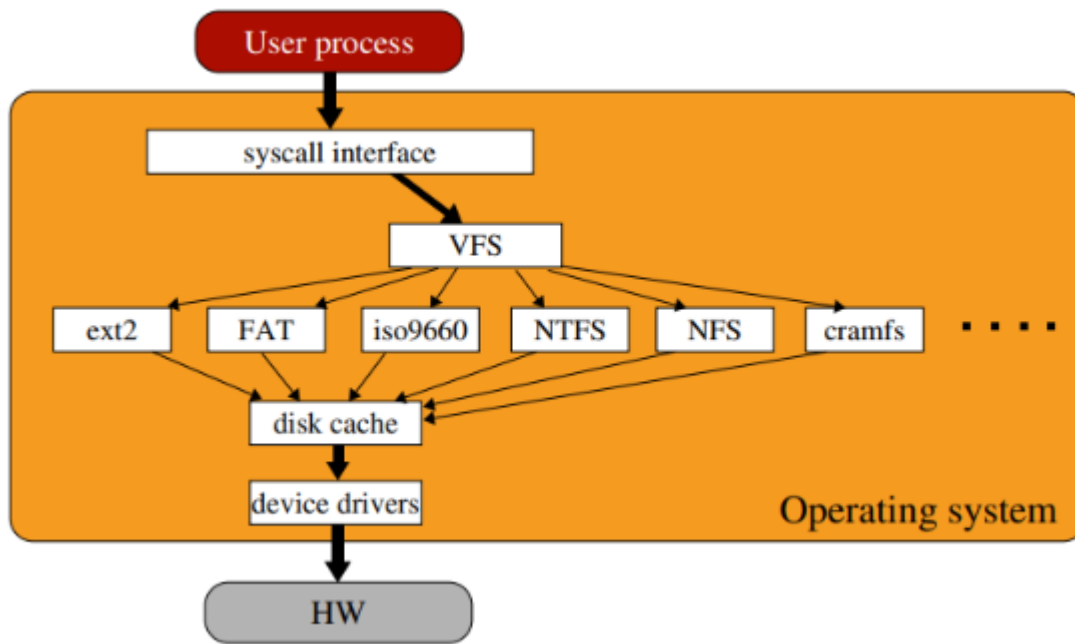
1. **Fase di Scrittura nel Log:** Le intenzioni di modifica e i metadati aggiornati non vengono scritti subito nelle tabelle definitive; vengono preventivamente annotati in un'area riservata e protetta del disco denominata **Registro di Giornale (Journal / Log)**.
2. **Fase di Commit:** Una volta completata con successo la scrittura nel Journal, l'operazione viene considerata sicura. Il kernel provvede a trasferire i dati nelle strutture reali del file system.
3. **Fase di Sblocco:** A trasferimento concluso, l'annotazione nel registro viene rimossa.

La Meccanica di Ripristino: Se la macchina si spegne improvvisamente durante la fase 2, al successivo riavvio il kernel ispeziona il Journal: rileva l'annotazione di commit parziale e provvede a completare l'operazione in modo pulito (*roll-forward*). Se l'interruzione avviene durante la fase 1, l'annotazione nel log risulta incompleta; il kernel scarta la transizione lasciando il file system nello stato coerente originario, azzerando i tempi di scansione del disco.

23.2 Astrazione del Kernel: Il Virtual File System (VFS)

In un sistema operativo moderno possono coesistere contemporaneamente partizioni formattate con file system profondamente eterogenei (es. una partizione radice **ext4**, una chiavetta USB **FAT32** e un supporto ottico **ISO9660**).

Per evitare che le applicazioni debbano integrare logiche separate per interagire con ciascuno standard, il kernel frappa uno strato di astrazione denominato **Virtual File System (VFS)**. Il VFS espone un'interfaccia di sistema unica e uniforme (le system call **open**, **read**, **write**). Quando un programma invoca una **read()**, il VFS intercetta la richiesta, identifica la partizione logica coinvolta e devia il flusso eseguendo dinamicamente la porzione di codice specifica del driver di quel determinato file system, in modo del tutto trasparente per lo sviluppatore.



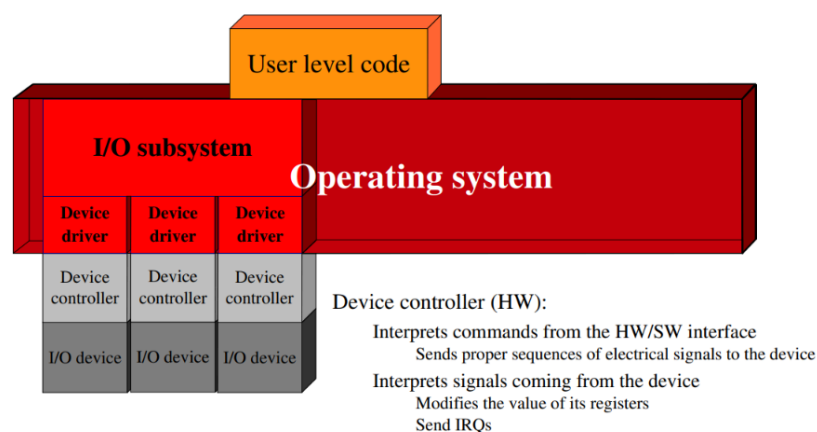
24 - Dispositivi di I/O

24.1 Accesso

Esistono tre metodologie di accesso per un dispositivo di I/O:

- **Polling:** Si chiede ciclicamente alla risorsa se deve comunicare qualcosa al processore, questa metodologia ha un consumo di tempo del processore elevato e non è sensato implementarlo come uno metodo di accesso.
- **I/O guidato da interrupt:** Viene indicato tramite interrupt che lo stato interno è stato modificato, il processore deve gestire un interrupt capendo da quale periferica è arrivato, capire il motivo della variazione di stato ed eventuale terminazione di operazioni I/O che sono state eseguite nel mentre.
- **DMA:** è uno strato software che si occupa delle comunicazioni processore - periferica. Si interessa di mettere in "coda" operazioni di I/O su una periferica che risponderrebbero, una volta terminate, con interrupt. Il DMA li nasconde finché non ha pronti tutti i dati richiesti, solo in quel caso manda un IRQ al processore per gestire i dati richiesti alla periferica. Questo meccanismo è particolarmente utile in caso di periferiche che leggono/scrivono pochi blocchi di dati e si vogliono processare N blocchi. Si richiede un HW molto complesso per l'implementazione e la gestione con politiche di accesso alla memoria ben definite per evitare race condition e garantire la mutua esclusione.

Solitamente la memoria è mappata in indirizzi che vengono suddivisi in base al loro utilizzo. Gli indirizzi $0 \rightarrow K$ sono quelli mappati in memoria, gli indirizzi $X \rightarrow Y$ sono quelli per **Dispositivo1**, quelli $P \rightarrow Q$ per **Dispositivo2**. Avendo gli indirizzi mappati in questo modo, posso usare load/store sia se gli indirizzi sono per la memoria che per l'HW.



Per Device Driver si intende lo strato SW necessario alla traduzione delle richieste in comunicazioni con l'HW, essendo l'HW molto dipendente dal dispositivo, questi blocchi di codice si occupano di tutta la comunicazione con il SO come la gestione di interrupt del device o la comunicazione per gestione di I/O.

25 - Interrupt

25.1 Gestione delle Interruzioni (Interrupt Handling)

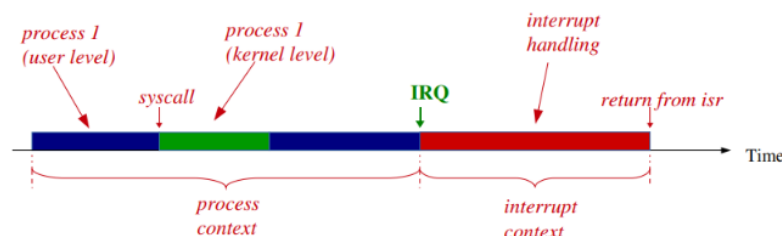
Le interruzioni hardware (Interrupt) sono segnali asincroni inviati dalle periferiche fisiche al processore per richiedere attenzione immediata a seguito del verificarsi di un evento (es. lo scadere di un timer di scheduling, il completamento di un trasferimento DMA o l'arrivo di pacchetti dalla scheda di rete).

Al sopraggiungere di un interrupt, la CPU interrompe istantaneamente l'esecuzione del processo utente corrente per commutare in modalità Kernel Mode e saltare all'indirizzo del gestore dedicato, memorizzato nella tabella dei vettori delle interruzioni.

25.1.1 Il Conflitto Tempestività ed Elaborazione Lunghe

La progettazione dei gestori di interrupt (ISR) deve risolvere un vincolo conflittuale:

- La gestione dell'interruzione deve essere **estremamente rapida** per mantenere il sistema reattivo ed evitare la perdita di successivi segnali hardware.
- Durante l'esecuzione della routine del gestore, il kernel opera spesso con le **interruzioni disabilitate** (zona rossa), congelando la reattività del sistema verso le altre periferiche.
- Tuttavia, l'operazione richiesta dall'evento potrebbe essere strutturalmente complessa o richiedere il trasferimento di grandi quantità di dati da un buffer all'altro.



25.1.2 Suddivisione in Top Half e Bottom Half

Per districare questo conflitto, i sistemi operativi moderni (come Linux) suddividono la gestione di un interrupt in due stadi funzionali distinti:

1. **Top Half (Metà Superiore / Parte Urgente):** È il codice dell'ISR che viene eseguito immediatamente al sopraggiungere del segnale hardware, operando in *Contesto di Interruzione* con gli interrupt disabilitati. Esegue unicamente le operazioni minime ed essenziali: identifica la periferica mittente, preleva i puntatori ai buffer hardware per congelare lo stato e invia un segnale di interrupt software (o virtuale) al sistema per accodare il lavoro pesante. Terminata questa fase (poche istruzioni), riabilita istantaneamente gli interrupt di sistema.
2. **Bottom Half (Metà Inferiore / Parte Differita):** È il codice deputato a eseguire le elaborazioni lente o massive precedentemente pianificate dal Top Half. Viene

eseguito in un secondo momento con gli **interrupt abilitati**, lasciando il sistema pienamente reattivo verso l'hardware.

Il Bottom Half può essere implementato secondo due modalità a seconda delle necessità operative:

- **Tramite Software IRQ / Tasklet:** Il codice viene eseguito comunque all'interno del *Contesto di Interruzione*. È molto veloce ma eredita un vincolo stringente: **il codice non può in nessun caso sospendersi o bloccarsi** (è vietato invocare primitive bloccanti, causare page fault o richiedere memoria dinamica bloccante), pena il crash del kernel.
- **Tramite Thread di Lavoro (Worker Threads):** Il Top Half inserisce la richiesta all'interno di una coda gestita da un processo/thread demone del kernel che gira in **Contesto di Processo**. Quando lo scheduler assegna il tempo CPU a questo thread, esso esegue le elaborazioni lente. Operando in contesto di processo, **questo codice può legittimamente bloccarsi**, sospendersi su primitive di sincronizzazione (mutex), allocare memoria dinamica o effettuare I/O bloccante in totale sicurezza.